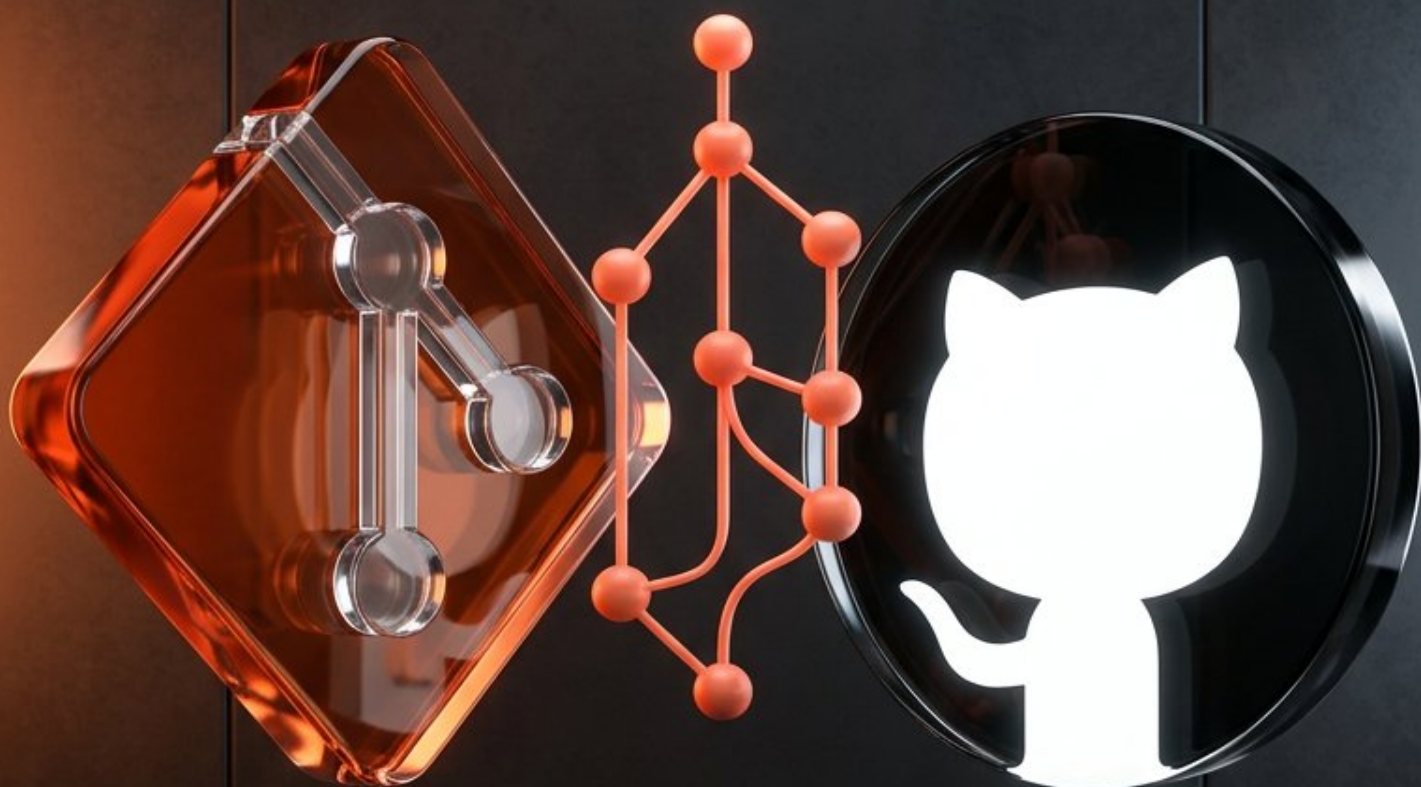




مرجع جامع و حرفه‌ای

Git و GitHub – از صفر تا سطح فوق‌پیشرفته با VS Code



محمدرضا رضائیان

نویسنده و گردآورنده

مرجع فارسی Git و GitHub برای جامعه توسعه‌دهندگان



فهرست مطالب

بخش یکم · بنیادها و مدل ذهنی Git

۴	01	معرفی Git و نقشه راه ۲۰۲۶ – از صفر تا Senior
۸	02	نصب و هویت – ستاپ حرفه‌ای Git در ۲۰۲۶
۱۳	03	سه درخت – Repository و Working Tree، Index
۱۷	04	اولین مخزن – init تا log
۲۱	05	اشیای Git – blob، tree، commit و hash
۲۵	06	شاخه‌ها – اشاره‌گر، نه کپی
۳۰	07	ادغام – fast-forward، merge commit و تعارض
۳۴	08	reflog و Undo – restore، reset، revert
۳۸	09	نادیده‌گرفتن و فایل بزرگ – gitignore، attributes، LFS
۴۱	10	Source Control از صفر – Ctrl+Shift+G

بخش دوم · کار روزانه، VS Code و GitHub

۵۶	11	Source Control فوق‌پیشرفته – Graph تا PR
۶۷	12	دیدن تغییر – bisect و diff، log، blame
۷۰	13	کار موازی – sparse-checkout و stash، worktree
۷۳	14	tracking و Remote – fetch، pull، push
۷۷	15	GitHub – ریپو، README و چهره عمومی پروژه
۸۱	16	Upstream و Clone، Fork – ورود به کد دیگران
۸۴	17	Pull Request – دروازه کیفیت تیم
۸۸	18	Code Review – فرهنگ، نه گیر دادن
۹۱	19	GitHub CLI – gh به جای کلیک اضافه
۹۴	20	Discussions و Issues، Projects – کار تیمی روی GitHub
۹۶	21	استراتژی شاخه – GitHub Flow تا Trunk-Based

۹۸		22	Conventional Commits – پیام به‌عنوان API
----	-------	----	--

بخش سوم · پیشرفته، امنیت، CI و Production

۱۰۰		23	Rebase تعاملی – بازنویسی تاریخچه مثل مجسمه‌ساز
۱۰۴		24	Cherry-pick، revert پیشرفته و rerere
۱۰۷		25	Hooks و کیفیت کامیت – lint قبل از push
۱۱۰		26	هویت و راز – امضا، passkey، 2FA و secret
۱۱۳		27	Rulesets – سیاست شاخه در سطح سازمان
۱۱۶		28	GitHub Actions – از رویداد تا استقرار
۱۲۰		29	امنیت زنجیره تأمین – Dependabot، CodeQL، Secret Scanning
۱۲۳		30	داخلی gc، fsck، packfile و Git – reftable
۱۲۶		31	مخازن بزرگ – Scalar، partial clone و monorepo
۱۲۸		32	Copilot و Git – Coding Agent در عصر عامل‌ها

بخش چهارم · کارگاه، نکات طلایی و مصاحبه

۱۳۱		33	۳۰ نکته و ترفند طلایی Git و GitHub
۱۳۳		34	کارگاه – چهار مینی‌پروژه از صفر تا PR
۱۳۶		35	۲۰ اشتباه رایج و راه‌حل
۱۳۹		36	مصاحبه، واژه‌نامه و نقشه راه بعد از کتاب

01 معرفی Git و نقشه راه ۲۰۲۶ – از صفر تا Senior

Git ابزار ثبت تاریخچه کد است؛ هر کامیت یک عکس کامل از پروژه در آن لحظه است. VS Code و GitHub ظاهر این ابزارند – مغز کار همان Git روی رایانهٔ توست.

🎯 اهداف یادگیری این فصل

بعد از این فصل، بدون ابهام می‌توانی بگویی:

- ♦ Git چیست (برنامه روی رایانهٔ تو) و GitHub چیست (سایت میزبانی)
- ♦ چرا به جای کپی‌کردن پوشه، از Git استفاده می‌کنیم
- ♦ کامیت، شاخه و «عکس لحظه‌ای» یعنی چه
- ♦ این کتاب را با چه ترتیبی بخوانی

🤔 چرا Git مهارت شغلی است؟

در آگهی‌های ۲۰۲۶ تقریباً هیچ نقش نرم‌افزاری بدون Git وجود ندارد. اما شرکت‌های خوب دیگر نمی‌پرسند «تفاوت pull و fetch چیست؟» – مدل ذهنی می‌پرسند:

«اگر روی main مشترک rebase کنی چه فاجعه‌ای رخ می‌دهد؟» «چطور یک باگ را با bisect در تاریخچه ده هزار کامیتی پیدا می‌کنی؟» «Branch Protection با Ruleset چه فرقی دارد؟»

این کتاب برای همان سطح نوشته شده است.

i سه قطعه که تا آخر کتاب تکرار می‌شوند

تا فصل ۶ همین‌ها را با مثال می‌بینی. الان فقط معنی‌شان را بدان:

۱. **کامیت (commit)** – یک عکس ثبت‌شده از همه فایل‌های پروژه در یک لحظه، نه فقط فهرست خطوط عوض‌شده.
۲. **شاخه (branch)** – یک برچسب روی یک کامیت. پوشه را کپی نمی‌کند؛ فقط می‌گوید «الان این عکس، نوک کار ماست».
۳. **سه درخت** – (الف) فایل‌هایی که در ادیتور می‌بینی، (ب) پیش‌نویس کامیت بعدی، (ج) آخرین عکس ثبت‌شده. هر فرمان Git فقط یکی از این‌ها را عوض می‌کند. وقتی بپرسی «این فرمان کدام را عوض کرد؟» دیگر مبهم نمی‌ماند.

📖 تاریخچه – از BitKeeper تا Git ۳.۰

- ♦ ۲۰۰۵: لینوس توروالدز Git را در دو هفته ساخت؛ هستهٔ لینوکس دیگر نمی‌توانست به BitKeeper تکیه کند.

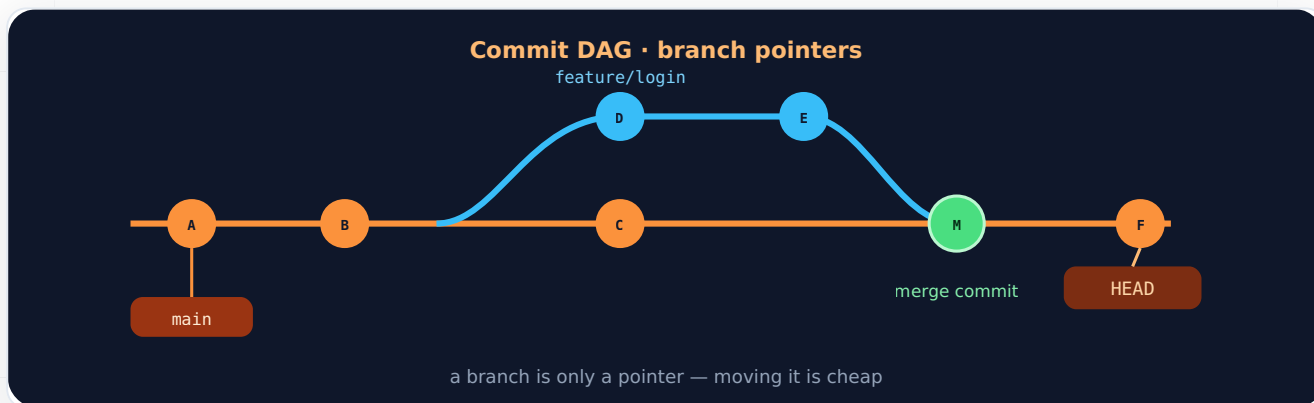
- ♦ ۲۰۰۸: GitHub متولد شد و «اجتماع روی کد» را ساخت؛ Fork و Pull Request زبان مشترک صنعت شد.
- ♦ ۲۰۱۹: `git switch` و `git restore` آمدند تا `checkout` دوچهره را بشکنند.
- ♦ ۲۰۲۵ (Git ۲.۵۱): `switch` و `restore` دیگر آزمایشی نیستند؛ refable و SHA-256 برای مخازن جدید آماده‌اند.
- ♦ ۲۰۲۶: Git ۳.۰ در افق است — پاکسازی فرمان‌های قدیمی، hash پیش‌فرض SHA-256، و backend سریع‌تر برای میلیون‌ها ref.
- ♦ گیت‌هاب ۲۰۲۵-۲۰۲۶: Agent HQ، Copilot Coding Agent، Immutable Releases، Rulesets، و Actions تو در تو.

♦ دو مسیر در هر فصل — هیچ‌کدام را حذف نمی‌کنیم

برای عوض کردن شاخه هرگز `checkout` نمی‌نویسیم: `git switch` شاخه، `git restore` فایل. هر مهارت را دو بار می‌بینی: فرمان در ترمینال و دکمه در VS Code (پنل Source Control و افزونه GitHub Pull Requests). مفهوم را در ترمینال بفهم؛ کار روزانه را در VS Code انجام بده.

Git چه چیزی نیست؟ 🏗️

تصور غلط	واقعیت روشن
Git همان GitHub است	Git برنامه روی رایانه توست. GitHub فقط یکی از سایت‌های میزبان است (GitLab و Gitea هم هستند). بدون اینترنت هم Git کار می‌کند.
Git فقط پشتیبان فایل است	Git علاوه بر فایل، چرا عوض شد (پیام کامیت) و چه کسی / کی را هم نگه می‌دارد.
شاخه = کپی کل پروژه	شاخه یک برچسب کوتاه روی یک کامیت است؛ ساختنش آنی است و پوشه را دوباره نمی‌کند.
کامیت = فقط diff	کامیت یک snapshot است: عکس کامل درخت فایل‌ها در آن لحظه. Diff بعداً از مقایسه دو عکس به‌دست می‌آید.



شکل ۱ – هر دایره یک کامیت است. پیکان به کامیت قبلی (والد) می‌رود. حلقه وجود ندارد؛ به این گراف DAG می‌گویند. شاخه فقط نام روی یکی از دایره‌هاست.

DAG یعنی Directed Acyclic Graph: گراف جهت‌دار بدون دور. اگر این یک جمله را حفظ کنی، merge (دو والد) و rebase (گره‌های جدید) دیگر رمز نیستند.

🗺️ اکوسیستم ۲۰۲۶ – چه چیزی یاد بگیریم؟

لایه	انتخاب ۲۰۲۶	چرا؟	جایگزین
VCS	+Git ۲.۵۱	switch/restore پایدار، reftable	Mercurial فقط میراث
Host	GitHub	PR، Actions، Security، Copilot	GitLab / Gitea
ادیتور	GitHub Pull + VS Code Requests	Source Control، Graph Review داخل IDE	JetBrains / Neovim
CLI همراه	GitHub CLI (gh)	PR، Issue، Actions از ترمینال	hub قدیمی
کیفیت کامیت	Conventional Commits	semantic changelog خودکار، release	پیام آزاد
CI	GitHub Actions	OIDC، reusable workflow	Jenkins میراث
امنیت	Rulesets + signing + CodeQL	سیاست متمرکز، عرضه قابل اثبات	فقط branch protection کلاسیک

🧠 چگونه این کتاب را بخوانیم؟

♦ ۱. فرمان را حفظ نکن – بپرس کدام لایه عوض شد

بعد از هر فرمان از خودت بپرس: فایل روی دیسک عوض شد، پیش‌نویس کامیت عوض شد، یا تاریخچه ثبت‌شده؟ (اسامی دقیق این سه لایه در فصل ۳ است.)

♦ ۲. ترمینال و VS Code را با هم یاد بگیر

تقریباً هر کار دو شکل دارد: یک فرمان و یک دکمه. اگر فقط یکی را بلد باشی، در تیم گیر می‌کنی.

♦ ۳. عمداً خراب کن – برگشت دارد

حذف تاریخچه ثبت‌شده سخت است، ولی حرکت اشتباه HEAD معمولاً تا حدود ۹۰ روز در `reflog` مانده. فصل ۸ دقیقاً می‌گوید کدام Undo کدام لایه را لمس می‌کند.

🚀 چک‌لیست فهم عمیق

- ☐ می‌توانی مدل را روی کاغذ بکشی؟
- ☐ می‌توانی همان کار را در VS Code انجام دهی؟
- ☐ می‌توانی بگویی کدام فرمان تاریخچه را بازنویسی می‌کند؟
- ☐ می‌توانی در مصاحبه ۲ دقیقه توضیح دهی؟

📖 مسیر پیشنهادی

- هفته ۱:** فصل ۱-۱۰ – مدل ذهنی + Source Control از صفر (`Ctrl+Shift+G`)
- هفته ۲:** فصل ۱۱-۲۲ – همکاری واقعی روی GitHub
- هفته ۳:** فصل ۲۳-۳۲ – `rebase`، `Actions`، امنیت
- هفته ۴:** فصل ۳۳-۳۶ – کارگاه، اشتباهات، مصاحبه

“ سؤالات مصاحبه (با پاسخ)

Git – Junior و GitHub چه فرقی دارند؟ Git موتور نسخه‌بندی توزیع‌شده روی ماشین توست. GitHub سرویس میزبانی، `PR`، `Issue`، `Actions` و هویت تیمی است.

Mid – چرا Git معمولاً از SVN سریع‌تر حس می‌شود؟ چون کل تاریخچه روی رایانه توست و هر فایل با هاش محتوایش شناخته می‌شود. شبکه فقط وقتی لازم است که بخواهی با دیگران همگام شوی – نه برای هر مقایسه.

Senior – DAG یعنی چه و چرا مهم است؟ تاریخچه یک خط نیست؛ یک گراف است. `Merge` کامیتی با دو والد می‌سازد. `Rebase` کامیت‌های جدید می‌سازد (هاش عوض می‌شود). اگر گراف را ببینی، می‌فهمی چرا `force-push` روی `main` کار دیگران را می‌شکند.

✓ | جمع‌بندی و اتصال

- ♦ Git روی رایانه تو تاریخچه را با عکس لحظه‌ای و برچسب شاخه نگه می‌دارد.
 - ♦ GitHub میزبان + صفحه `PR` + قوانین تیم + اجرای خودکار تست است.
 - ♦ مسیر این کتاب: `restore / switch` ، VS Code ، بعد GitHub ، بعد امنیت و CI.
- فصل بعد:** نصب Git، نام و ایمیل، کلید SSH و امضای کامیت – تا هویت تو در تاریخچه درست ثبت شود.

02 نصب و هویت – ستاپ حرفه‌ای Git در ۲۰۲۶

هر کامیت نام و ایمیل را داخل خودش ذخیره می‌کند. اگر این دو را غلط بگذاری، تاریخچه مال کس دیگری به نظر می‌رسد. این فصل را یک‌بار درست تنظیم کن.

🎯 اهداف یادگیری

- ♦ Git را روی Windows / macOS / Linux نصب و نسخه را چک کنی
- ♦ `user.name` و `user.email` را درست – و ترجیحاً محلی – تنظیم کنی
- ♦ SSH Key بسازی، به GitHub اضافه کنی و با `gh auth` وارد شوی
- ♦ کامیت‌ها را با SSH sign کنی (جایگزین مدرن GPG)

🔧 نصب Git ۲.۵۱+

```
🍏 🍏 🍏 نصب - لینوکس / macOS / Windows

# Debian / Ubuntu
sudo apt update && sudo apt install -y git gh

# macOS
brew install git gh

# Windows (پاورشل به‌عنوان Admin)
winget install Git.Git GitHub.cli

git --version    # باید ۲.۵۱ یا جدیدتر باشد
gh --version
```

در VS Code از Command Palette (`Ctrl+Shift+P`) بنویس `Git: Show Git Output` . اگر Git پیدا نشود، مسیر را در `git.path` تنظیم کن.

📁 هویت – دو لایه

دو هویت جدا وجود دارد و باید یکی باشند:

۱. هویت Git – همان `user.name` و `user.email` که داخل هر کامیت نوشته می‌شود.
۲. حساب GitHub – ایمیلی که در Settings گیت‌هاب ثبت کرده‌ای.

اگر ایمیل کامیت با ایمیل حساب یکی نباشد، کنار کامیت در GitHub آواتار خاکستری می‌بینی و نشان **Verified** (تأیید امضا) نمی‌آید.

```
پیگردندی هویت

# سراسری - فقط اگر یک هویت داری
git config --global user.name "rezaian-dev"
git config --global user.email "mrezaian.dev@gmail.com"

# بهتر: هویت داخل هر مخزن (توصیه تیمی)
git config user.name "rezaian-dev"
git config user.email "mrezaian.dev@gmail.com"

git config --global init.defaultBranch main
git config --global pull.rebase false
git config --global fetch.prune true
git config --global rerere.enabled true
git config --global core.editor "code --wait"
git config --global alias.st status
git config --global alias.lg "log --oneline --graph --decorate --all"
```

ایمیل مخفی GitHub ⚠

اگر نمی‌خواهی ایمیلت در تاریخچه عمومی دیده شود، از noreply گیت‌هاب استفاده کن:

12345678+rezaian-dev@users.noreply.github.com

در Settings → Emails گزینه **Block command line pushes that expose my email** را روشن کن.

SSH – در جلویی GitHub 🔑

HTTPS با token کار می‌کند اما SSH برای کار روزانه تمیزتر است.

```
Ed25519 ساخت کلید

ssh-keygen -t ed25519 -C "mrezaian.dev@gmail.com"
# Enter - passphrase بگذار

eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519

# کلید عمومی را کپی کن
cat ~/.ssh/id_ed25519.pub
```

در GitHub: **Settings → SSH and GPG keys → New SSH key**. بعد:

```
تست اتصال

ssh -T git@github.com
# Hi rezaian-dev! You've successfully authenticated...
```

در VS Code وقتی Clone می‌کنی، اگر remote با **git@github.com:** شروع شود از همین کلید استفاده می‌شود.

🔥 امضای کامیت با SSH – استاندارد ۲۰۲۶

GPG هنوز معتبر است اما **SSH signing** ساده‌تر است و همان کلید را دوباره مصرف می‌کند.

```
فعال‌سازی امضا

git config --global gpg.format ssh
git config --global user.signingkey ~/.ssh/id_ed25519.pub
git config --global commit.gpgsign true
git config --global tag.gpgsign true
```

همان کلید عمومی را در GitHub به‌عنوان **Signing Key** هم اضافه کن. بعد از آن، کامیت‌هایت نشان **Verified** می‌گیرند.

VS Code +

در Settings بگذار:

```
git.enableCommitSigning: true
```

دکمه Commit از همین تنظیم پیروی می‌کند – لازم نیست هر بار پرچم **-S** بزنی.

🐙 ورود با GitHub CLI

```
gh auth

gh auth login
# GitHub.com → SSH → Login with browser

gh auth status
gh api user --jq .login # rezaian-dev
```

از این به بعد **gh repo create**، **gh pr create** و **gh run list** بدون token دستی کار می‌کنند.

```
گیت - .vscode/settings.json
{
  "git.enabled": true,
  "git.autofetch": true,
  "git.confirmSync": false,
  "git.enableSmartCommit": false,
  "git.enableCommitSigning": true,
  "git.untrackedChanges": "separate",
  "git.branchProtection": ["main", "master"],
  "git.branchProtectionPrompt": "alwaysCommitToNewBranch",
  "git.rebaseWhenSync": false,
  "scm.defaultViewMode": "tree",
  "diffEditor.ignoreTrimWhitespace": false
}
```

افزونه گیت VS Code – اتاق فرمان روزانه

Git داخل VS Code سه لایه دارد. این کتاب هر فرمان را با هر دو مسیر یاد می‌دهد: ترمینال و این افزونه‌ها.

نقش	شناسه افزونه	لایه
Source Control، Stage، Commit Branch، Sync	همراه VS Code	Git توکار
Clone از GitHub، Checkout PR Merge، ریویو اینلاین	GitHub.vscod-pull-request-github	GitHub Pull Requests
DAG رنگی، blame خطی، مقایسه کامیت‌ها	/ mhutchie.git-graph eamodio.gitlens	Git Graph یا GitLens
وضعیت workflow کنار ادیتور	github.vscod-github-actions	GitHub Actions

نصب از خود ادیتور:

۱. Extensions → `Ctrl+Shift+X`

۲. جست‌وجو: `GitHub Pull Requests` → Install

۳. جست‌وجو: `Git Graph` → Install

۴. Accounts (پایین Activity Bar) → `Sign in to GitHub`

۵. `Ctrl+Shift+G` باید پنل Source Control را باز کند – خانهٔ روزانهٔ Git در این کتاب؛ فصل ۱۰ و ۱۱ آن را تا تسلط کامل می‌روند.

Terminal

```
git switch -c git commit -m "..."  
gh pr git push -u origin HEAD feat/x  
create --fill
```

VS Code Git

Commit: جعبهٔ پیام → `Ctrl+Enter` **شاخه:** کلیک روی `main` در نوار وضعیت **Sync:** آیکون `↺` کنار نام شاخه **PR:** آیکون GitHub در Activity Bar

قانون این کتاب: مفهوم را در ترمینال بفهم؛ کار روزانه را در VS Code انجام بده. Senior هر دو را بلد است.

“سؤالات مصاحبه (با پاسخ)”

Junior – چرا SSH بهتر از HTTPS است؟ کلید روی دیسک می‌ماند، passphrase در agent کش می‌شود، و token PAT را هر چند ماه عوض نمی‌کنی.

Mid – تفاوت `user.email` و حساب `GitHub`؟ Git فیلد را داخل شیء `commit` می‌نویسد. `GitHub` با همان ایمیل کامیت را به پروفایل وصل می‌کند.

Senior – چرا امضا مهم است؟ بدون امضا هر کسی می‌تواند با `user.name` تو کامیت جعلی بسازد. Verified یعنی کلید ثبت‌شده در `GitHub` آن شیء را امضا کرده.

✓ | چک‌لیست فهم

☐ `git --version` حداقل ۲.۵۱ است

☐ `user.name` / `user.email` با `GitHub` یکی است

☐ `ssh -T git@github.com` سلام می‌دهد

☐ کامیت تست با نشان Verified ساخته‌ای

☐ VS Code Signing و branch protection روشن است

فصل بعد: سه درخت `Git` – جایی که ۸۰٪ سردرگمی متولد می‌شود.

03 سه درخت – Repository و Working Tree، Index

Git فایل‌ها را هم‌زمان در سه جا نگه می‌دارد: آنچه در ادیتور می‌بینی، پیش‌نویس کامیت بعدی، و آخرین عکس ثبت‌شده. اگر این سه را قاطی کنی، `reset` و `Save` و `Commit` برای یک به نظر می‌رسند.

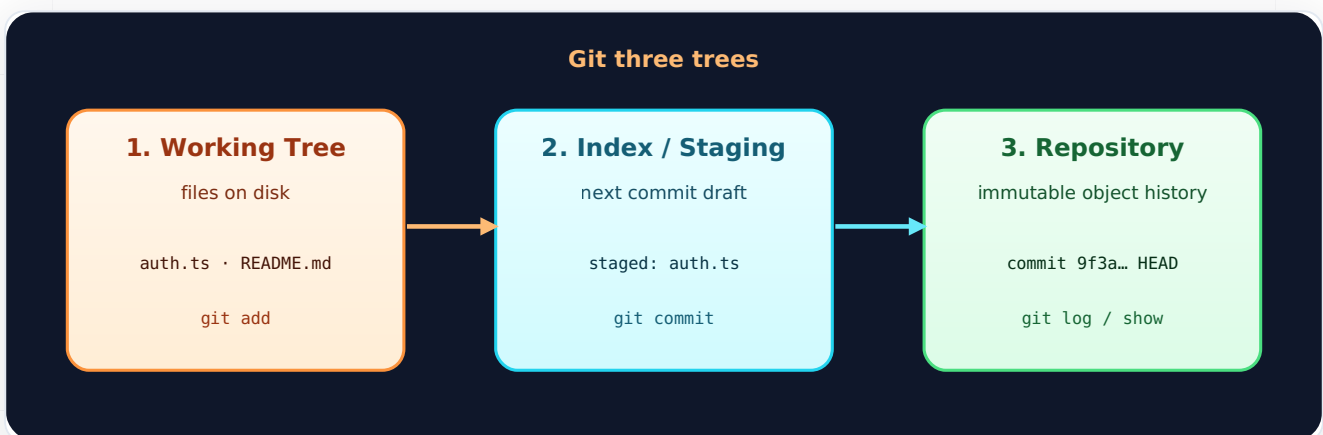
اهداف یادگیری

- سه درخت را نام ببری و نقش هر کدام را بکشی
- وضعیت فایل را در VS Code بخوانی: U، M، A، D
- تفاوت `unstaged` و `staged` را با مثال نشان دهی
- بگویی `add`، `restore` و `commit` دقیقاً چه چیزی را کپی می‌کنند

مدل ذهنی

Git فایل‌ها را در سه لایه نگه می‌دارد:

- Working Tree** – فایل‌هایی که در Explorer می‌بینی و با `Save` ذخیره می‌کنی. هنوز تاریخچه نیست.
- Index (Staging Area)** – پیش‌نویس کامیت بعدی. `git add` یا دکمه Stage فایل را از دیسک به این پیش‌نویس کپی می‌کند.
- Repository (HEAD)** – آخرین عکس ثبت‌شده. فقط `commit` این لایه را جلو می‌برد.



شکل ۲ – `add` از Working به Index می‌رود؛ `commit` از Index یک شیء `commit` می‌سازد.

i فرمول طلایی

$\text{diff}(\text{Working}, \text{Index}) + \text{diff}(\text{Index}, \text{HEAD}) = \text{status}$

دو ستون تغییرات در VS Code دقیقاً همین دو `diff` هستند.

وضعیت را با چشم مسلح ببین

```
git status -sb
# ## main
# M src/auth.ts      # نشده stage هنوز، Working، یعنی تغییر در M + فاصله
# M src/api.ts       # M در ستون اول یعنی M
# MM README.md       # هم بعدش دوباره ویرایش شده staged هم
# ?? notes.txt       # untracked - Git اصلاً نمی‌شناسدش
```

در VS Code همین اطلاعات چنین دیده می‌شود:

نشانه	معنی	اقدام
U	Untracked	Stage یا به <code>.gitignore</code> بسپار
M	Modified	Stage / Discard
A	Added	فایل جدید staged
D	Deleted	حذف staged یا بازیابی
C	Conflict	Resolve در ادیتور

آزمایش ۳۰ ثانیه‌ای 🧪

سه درخت را حس کن

```
echo "v1" > hello.txt
git add hello.txt      # Working = Index (v1 هر دو) HEAD ندارد
git commit -m "v1"     # هر سه = v1

echo "v2" > hello.txt
git add hello.txt      # Working=v2 Index=v1 HEAD=v1
git commit -m "v2"     # Working=v2 Index=v2 HEAD=v1
git commit -m "v2"     # هر سه = v2
```

حالا می‌فهمی چرا «کامیت نکردم ولی Save کردم» تاریخچه را عوض نمی‌کند – Save فقط Working Tree است.

فرمان‌ها روی کدام درخت؟

فرمان	Working	Index	Repo
ویرایش در VS Code / Save	✓		
Stage / <code>git add</code>		✓	
<code>git commit</code>			✓
<code>git restore file</code>	✓		
<code>git restore --staged</code>		✓	
<code>git reset --soft</code>			✓ (HEAD حرکت)
<code>git reset --mixed</code>		✓	✓
<code>git reset --hard</code>	✓	✓	✓

reset --hard ✖

`--hard` هر سه درخت را با کامیت هدف یکی می‌کند و تغییرات ذخیره‌نشده را می‌کشد. در VS Code معادل **Discard All Changes** + حرکت HEAD است. تا فصل ۸ بدون تور ایمنی (`reflog`) نزدیکش نشو.

در VS Code کجایند این درخت‌ها؟

Working Tree = **Explorer** ♦

$\text{diff(Working, Index)}$ = **Source Control → Changes** ♦

diff(Index, HEAD) = **Source Control → Staged Changes** ♦

Repository = **Timeline / Git Graph** ♦

وقتی فایلی را Stage می‌کنی، از لیست Changes به Staged می‌پرد – یعنی کپی snapshot آن فایل به Index.

عادت Junior

. `git add` کورکورانه. فایل `env`، آرتیفکت بیلد و رمز داخل کامیت می‌رود.

عادت Senior

`git add -p` یا Stage انتخابی در VS Code (روی `hunk`). کامیت را مثل پاراگراف می‌نویسی نه مثل سطل زباله.

“ سؤالات مصاحبه (با پاسخ)

Junior – Staging یعنی چه؟ سبد خرید قبل از پرداخت. commit پرداخت است.

Mid – چرا Index وجود دارد؟ تا یک Working شلوغ را به چند کامیت معنادار بشکنی.

Senior – Index از نظر داده چیست؟ یک درخت مرتب‌شده از cache entry ها (مسیر، stat، OID، mode) که commit آن را به یک tree object تبدیل می‌کند.

✓ | چک‌لیست فهم

☐ سه درخت را بدون نگاه به کتاب نام می‌بری

☐ M در ستون اول و دوم sb -status را بلدی

☐ می‌دانی Save ≠ Commit

☐ Stage انتخابی را به add ترجیح می‌دهی

فصل بعد: اولین مخزن واقعی – init ، add ، commit ، log .

04 اولین مخزن – log تا init

در این فصل یک پوشه را به مخزن Git تبدیل می‌کنی، دو کامیت با پیام روشن می‌نویسی، و با `log` می‌بینی تاریخچه دقیقاً چه شکلی شده.

اهداف یادگیری

- ♦ مخزن را از صفر با `git init` یا از VS Code بسازی
- ♦ پیام کامیت خوب بنویسی
- ♦ `log` را با گراف بخوانی
- ♦ تفاوت `--allow-empty` و کامیت واقعی را بدانی

ساخت مخزن

- ♦ روش ۱ – ترمینال (پایه)

```
تمیز init

mkdir git-lab && cd git-lab
git init -b main
# Initialized empty Git repository in .../.git/

echo "# Git Lab" > README.md
git add README.md
git status
git commit -m "chore: bootstrap repository"
```

پرچم `-b main` شاخه پیش‌فرض را `master` نمی‌گذارد. اگر فراموش کردی: `git config --global init.defaultBranch main`

- ♦ روش ۲ – VS Code (توصیه این کتاب)

۱. پوشه را با `File → Open Folder` باز کن
۲. نوار کناری آیکون Git را بزن (`Ctrl+Shift+G`)
۳. دکمه `Initialize Repository`
۴. فایل بساز، در `Changes` ظاهر می‌شود
۵. پیام را بنویس و `Ctrl+Enter`

اگر پنل خالی دیدی و دکمه Initialize نبود، یا پوشه Git ندارد یا Git در PATH نیست.

📖 پیام کامیت – قرارداد این کتاب

از **Conventional Commits** استفاده می‌کنیم (فصل ۲۲ عمیق می‌شود):

```
قالب
type(scope): summary
بدنهٔ اختیاری - چرا، نه چه
```

انواع رایج: `feat`، `fix`، `docs`، `chore`، `refactor`، `test`، `ci`.

خوب

`feat(auth): add refresh-token rotation`

چرا: نوع + محدوده + فعل امری + نتیجه.

بد

`update` یا `fix bug` یا `asdf`

تاریخچهٔ بی‌معنی، `git log` را به زباله‌دان تبدیل می‌کند.

در VS Code پیام را در جعبهٔ بالا می‌نویسی. `Ctrl+Enter` کامیت می‌کند. اگر **Smart Commit** روشن باشد و چیزی stage نشده باشد، همه چیز را قورت می‌دهد – در فصل ۲ آن را خاموش کردیم.

📖 خواندن تاریخچه

```
Senior به سبک log
git log --oneline --decorate --graph --all
git log -1 -p # آخرین کامیت + diff
git show HEAD # معادل بالا
git log --author="rezaian" --since="2 weeks ago"
git shortlog -sn # چه کسی چقدر کامیت کرده
```

در VS Code:

- ♦ **Timeline** زیر فایل → تاریخچهٔ همان فایل
- ♦ **Git Graph** / **GitLens** → کل DAG
- ♦ کلیک روی کامیت در Source Control → جزئیات

تور سریع

```
ls .git
# HEAD config description hooks info objects refs

cat .git/HEAD          # ref: refs/heads/main
cat .git/refs/heads/main
# 9f3a1c... ← هش کامیت فعلی
```

هنوز لازم نیست packfile را بفهمی (فصل ۳۰). فقط بدان: هر چیز ماندگار اینجاست؛ پاک کردن `git` یعنی قتل تاریخچه.

✦ amend برای ۵ دقیقه اول

اگر همین الان پیام را غلط نوشتی و هنوز push نکرده‌ای:

```
git commit --amend -m "docs: fix readme title"
```

در VS Code: تیک Amend کنار دکمه Commit (منوی کشویی ✓).

⚠ هرگز amend بعد از push

--amend هش را عوض می‌کند. اگر دیگران آن کامیت را گرفته‌اند، تاریخچه دوباره می‌شود. قانون: فقط کامیت محلی push نشده.

“ سؤالات مصاحبه (با پاسخ)

Junior – git init چه می‌سازد؟ پوشه `.git` با `refs`، `objects`، `HEAD` و `config`.

Mid – چرا پیام امری می‌نویسیم؟ کامیت را مثل وصله بخوان: «این وصله refresh-token را اضافه می‌کند.» نه «اضافه کردم.»

Senior – تفاوت git log -p و git show ؟ show یک شیء را نمایش می‌دهد؛ log -p پیمایش تاریخچه با diff هر کامیت است.

✓ | چک لیست فهم

- ☐ مخزن با شاخهٔ `main` ساخته‌ای
- ☐ حداقل دو کامیت Conventional داری
- ☐ `git lg` (یا alias) گراف را نشان می‌دهد
- ☐ می‌دانی `.git` قلب مخزن است

فصل بعد: اشیای Git — blob، tree، commit و جادوی hash.

05 اشیای Git – blob، tree، commit و hash

Git هر فایل و هر کامیت را با یک اثرانگشت (هش) می‌شناسد. اگر حتی یک حرف پیام کامیت عوض شود، اثرانگشت عوض می‌شود – برای همین «ویرایش تاریخچه» در واقع ساختن کامیت جدید است، نه دستکاری فایل قدیمی.

🎯 اهداف یادگیری

- ♦ چهار نوع شیء را نام ببری: blob، tree، commit، tag
- ♦ توضیح دهی چرا بازنویسی کامیت هش را عوض می‌کند
- ♦ با `cat-file` و `rev-parse` یک کامیت را کالبدشکافی کنی
- ♦ تفاوت SHA-1 و SHA-256 (مسیر ۳.۰ Git) را بدانی

🧠 محتوا آدرس‌دهی

```
hash("blob " + size + "\0" + content) → شناسه فایل
```

اگر یک بیت عوض شود، شناسه عوض می‌شود. دو فایل یکسان در کل تاریخچه یک blob اند – این است دلیل ارزان بودن شاخه و سریع بودن Git.



شکل ۳ – commit به tree اشاره می‌کند؛ tree به blobها و treeهای فرزند.

کالبدشکافی یک کامیت

تا بایت‌های فایل HEAD از

```
git rev-parse HEAD
git cat-file -p HEAD
git cat-file -p 'HEAD^{tree}'      # حتماً نقل قول بگذار در zsh
git ls-tree -r HEAD
```

خروجی commit شبیه این است:

commit آناتومی

```
tree 4c2e9a...
parent a91b77...
author rezaian-dev <mrezaian.dev@gmail.com> 1757400000 +0330
committer rezaian-dev <mrezaian.dev@gmail.com> 1757400000 +0330

feat(auth): add refresh-token rotation
```

چهار چیز هویت کامیت را می‌سازند: tree، والد(ها)، هویت نویسنده/زمان، پیام. `--amend` روی هر کدام دست بگذارد، هاش جدید است.

annotated tag

تگ سبک در برابر تگ کامل

```
git tag v1.0.0                # ref: فقط یک
git tag -a v1.0.0 -m "release" # با امضا و پیام tag شیء
git cat-file -p v1.0.0
```

برای انتشار محصول همیشه **annotated** (و در فصل ۲۶ امضاشده) بساز.

SHA-1 در برابر SHA-256

SHA-256 (آینده)	SHA-1 (میراث)	
hex ۶۴	hex ۴۰	طول
در Git ۲.۵۱+ برای مخزن جدید آماده	هنوز پیش‌فرض بسیاری از hostها	وضعیت ۲۰۲۶
در حال گسترش – قبل از مهاجرت چک کن	پشتیبانی کامل	GitHub
<code>git init --object-format=sha256</code>	–	آزمایش

⚠ مهاجرت عجولانه نکن

مخزن SHA-256 را نمی‌توانی به ریموت SHA-1 پوش کنی. تا وقتی GitHub/تیم تو آماده نشده، روی SHA-1 بمان و فقط مفهوم را بدان.

Reftable (`git init --ref-format=reftable`) لایه **refs** را سریع می‌کند، نه hash را. برای monorepoهای عظیم مفید است.

✦ کوتاه‌هش

Git معمولاً ۷–۱۲ کاراکتر اول را می‌پذیرد (`9f3a1c`). در مخازن خیلی بزرگ ممکن است مبهم شود؛ `core.abbrev` را بالا ببر یا هش کامل را در اسکریپت‌ها استفاده کن.

“ سؤالات مصاحبه (با پاسخ)

blob – Junior چیست؟ محتوای فایل، بدون نام. نام در tree ذخیره می‌شود.

Mid – چرا rename در Git «تشخیص» است نه شیء؟ چون blob همان است؛ tree فقط مسیر را عوض کرده. `git log --follow` این جابه‌جایی را حدس می‌زند.

commit – Senior تغییرناپذیر است یعنی چه؟ شیء در objects دیتابیس هرگز ویرایش نمی‌شود. «ویرایش تاریخچه» یعنی ساختن اشیای جدید و حرکت دادن اشاره‌گر شاخه.

✓ چک لیست فهم

- ☐ چهار نوع شیء را بلدی
 - ☐ `cat-file -p HEAD` را تفسیر می‌کنی
 - ☐ می‌دانی `amend` چرا هش را عوض می‌کند
 - ☐ SHA-256 را با احتیاط می‌شناسی
- فصل بعد:** شاخه — ارزان‌ترین اشاره در مهندسی نرم‌افزار.

06 شاخه‌ها – اشاره‌گر، نه کپی

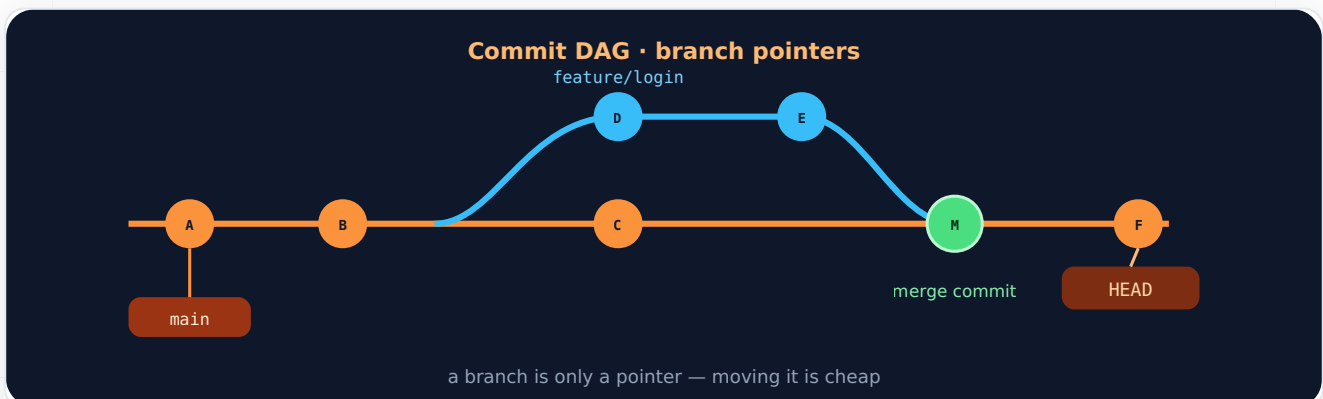
شاخه کپی پروژه نیست. یک فایل کوچک داخل `.git/refs/heads/` است که فقط هاش یک کامیت را نگه می‌دارد (حدود ۴۰ کاراکتر). ساختن شاخه یعنی نوشتن همین فایل.

🎯 اهداف یادگیری

- ♦ شاخه را به‌عنوان اشاره‌گر به یک commit توضیح دهی
- ♦ با `switch` و `switch -c` کار کنی
- ♦ HEAD detached را تشخیص و نجات دهی
- ♦ قرارداد نام‌گذاری تیم را رعایت کنی

🧠 شاخه چیست؟

مسیر `refs/heads/main` فقط هاش کامیت نوک شاخه را نگه می‌دارد. ساخت شاخه = نوشتن یک فایل جدید با یک هاش. عوض کردن شاخه = عوض کردن اینکه HEAD به کدام فایل اشاره کند.



شکل ۴ – HEAD به شاخه اشاره می‌کند؛ شاخه به کامیت. وقتی کامیت می‌کنی، هر دو جلو می‌روند.

اشاره‌گرها را ببین

```
git branch -vv          # شاخه‌های محلی + tracking
git show-ref --heads
cat .git/HEAD           # ref: refs/heads/main
```

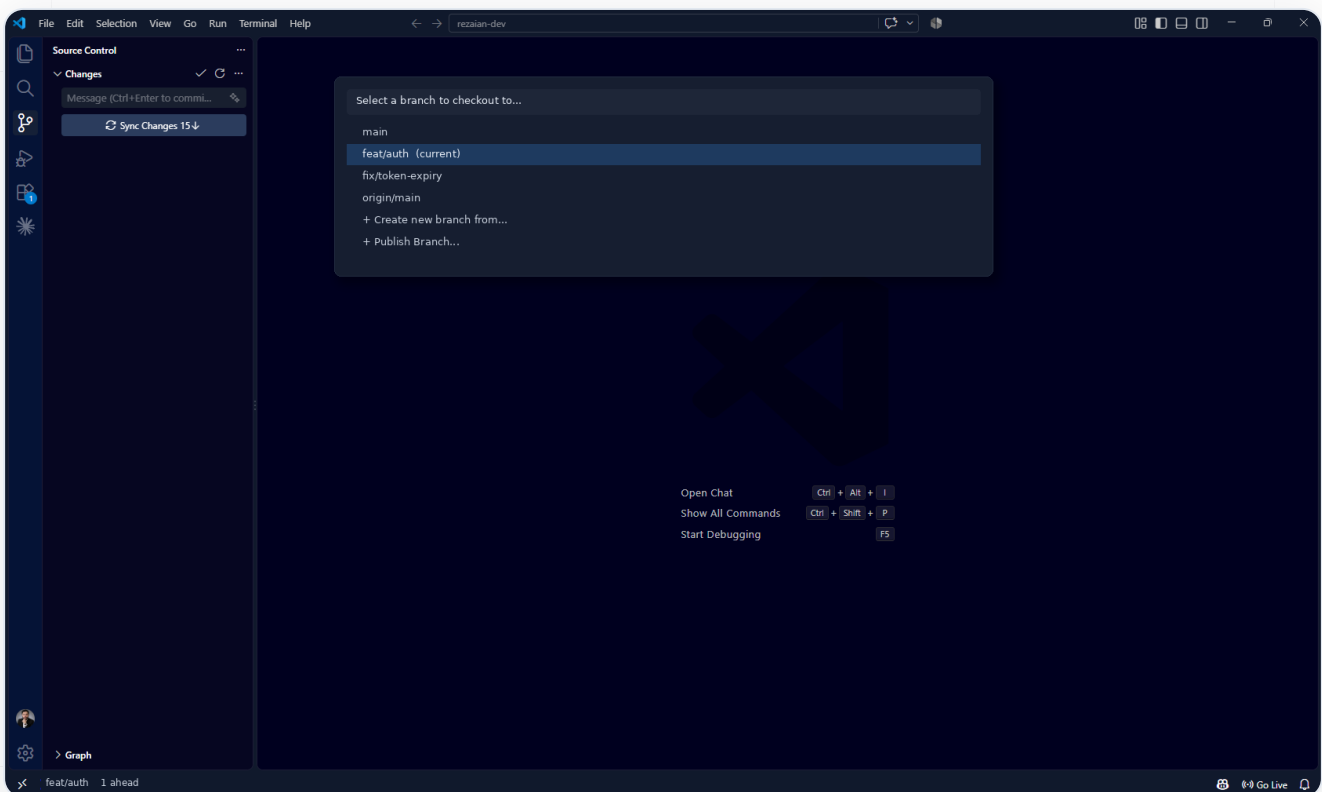
ساخت و تعویض – مسیر مدرن

Terminal

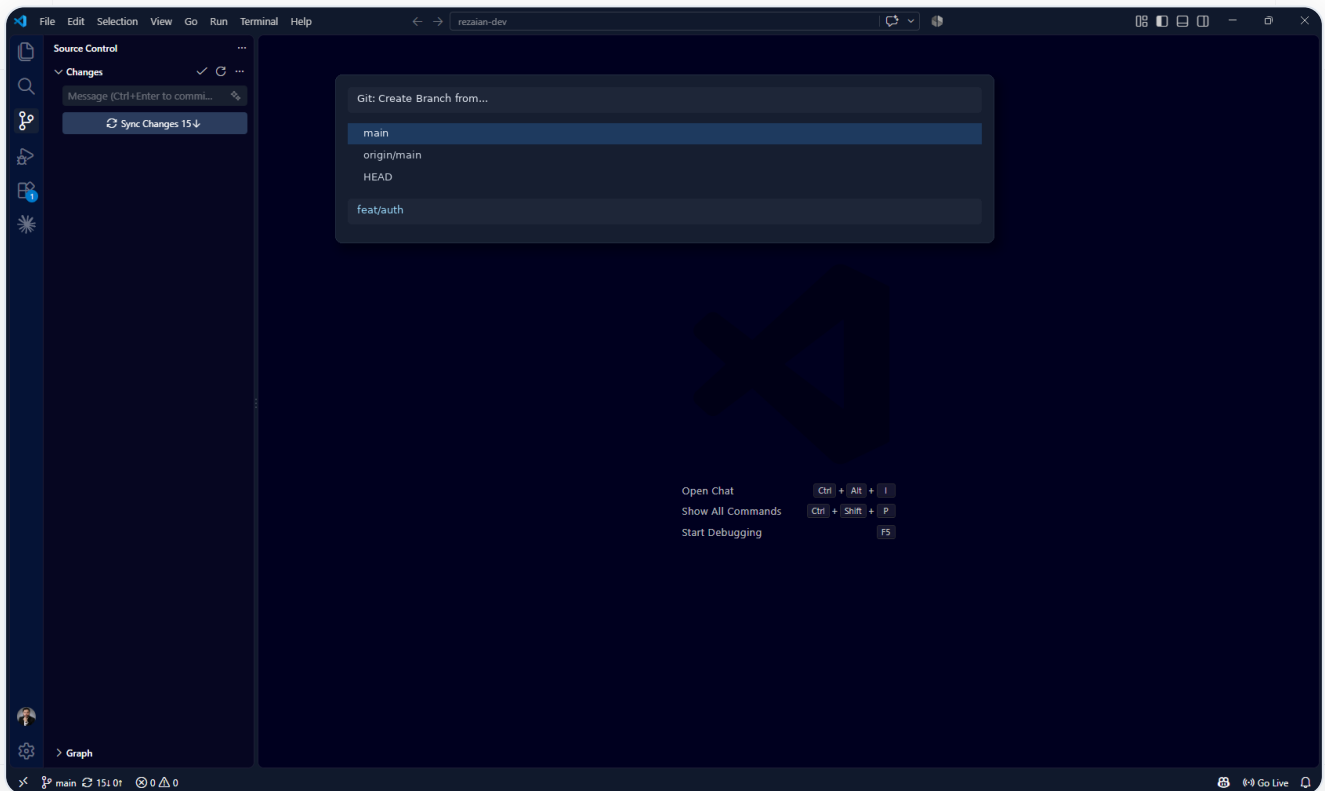
```
git switch -c feat/auth  
  
git switch main  
  
git switch -  
merge فقط اگر  
شده؛ وگرنه Git رد می‌کند.  
git branch -d feat/auth حذف اجباری
```

VS Code Git

۱. روی نام شاخه در نوار وضعیت (پایین-چپ) کلیک کن
۲. feat/auth → ...Create new branch from
۳. برای برگشت: همان منو → main
۴. Git: / Git: Create Branch :Palette
Checkout to...
۵. Publish Branch وقتی آمادهٔ remote شدی



شکل ۴ب – کلیک روی نام شاخه در نوار وضعیت: تعویض، ساخت شاخهٔ جدید، Publish.



شکل ۴ – Ctrl+Shift+P سپس Git: Create Branch from ... معادل `git switch -c`.

۱ چرا دیگر checkout نه؟

`checkout` هم شاخه عوض می‌کرد هم فایل برمی‌گرداند. دو معنا = دو برابر باگ انسانی. از Git ۲.۵۱ فرمان‌های `switch` و `restore` پایدار و توصیه‌شده‌اند.

نام‌گذاری

پیشوند	استفاده	مثال
feat/	قابلیت جدید	feat/jwt-refresh
fix/	باگ	fix/token-expiry
chore/	ابزار / وابستگی	chore/bump-actions
hotfix/	تعمیر تولید	hotfix/500-login
release/	آماده‌سازی نسخه	release/1.4.0

از فاصله، اموجی و نام‌های خیلی کلی (`update` , `ali`) پرهیز کن. GitHub شاخه را در URL می‌گذارد.

HEAD detached – وحشت بی‌مورد 🪂

وقتی مستقیم روی یک کامیت checkout/switch کنی (نه روی شاخه)، HEAD به هش می‌چسبد نه به ref.

```
detached نجات کارهای  
  
git switch --detach HEAD~3      # حداقل ۴ کامیت لازم است  
git switch -c rescue  
git switch main
```

در VS Code اگر Graph را بزنی و یک کامیت قدیمی را Checkout کنی، بنر زرد **detached HEAD** می‌آید. فوری شاخه بساز.

⚠ شاخه منتشرشده را بازنویسی نکن

main و شاخه‌هایی که دیگران track می‌کنند مقدس‌اند. بازنویسی یعنی force-push و درد تیم. قانون را در فصل ۲۳ با گوشت و پوست حس می‌کنی.

🧠 دیدن همه چیز

```
نقشه  
  
git branch -a          # محلی + remote  
git log --oneline --graph --decorate --all
```

در VS Code افزونه Graph همین نقشه را رنگی می‌کشد – فصل ۱۱ کامل روی آن کار می‌کنیم.

“ سؤالات مصاحبه (با پاسخ)

Junior – ساخت شاخه فایل‌ها را کی می‌کند؟ خیر. فقط یک اشاره‌گر جدید.

Mid – تفاوت branch و tag؟ هر دو اشاره‌گرند؛ شاخه با هر commit حرکت می‌کند، تگ معمولاً ثابت می‌ماند.

Senior – HEAD symbolic چیست؟ فایل HEAD معمولاً `ref: refs/heads/main` است نه خود هش. Detached یعنی HEAD مستقیم هش را نگه می‌دارد.

چک لیست فهم ✓

- ☐ شاخه را فایل اشاره گر می دانی نه پوشه
 - ☐ `switch -c` را به جای checkout استفاده می کنی
 - ☐ از detached بدون وحشت خارج می شوی
 - ☐ نام شاخه Conventional است
- فصل بعد:** ادغام – وقتی دو تاریخچه باید یکی شوند.

07 ادغام – fast-forward، merge commit و تعارض

Merge یعنی دو خط تاریخچه را یکی کنی. اگر هر دو خط همان خطوط فایل را جور دیگری عوض کرده باشند، Git نمی‌داند کدام درست است – به آن تعارض می‌گویند و باید خودت انتخاب کنی.

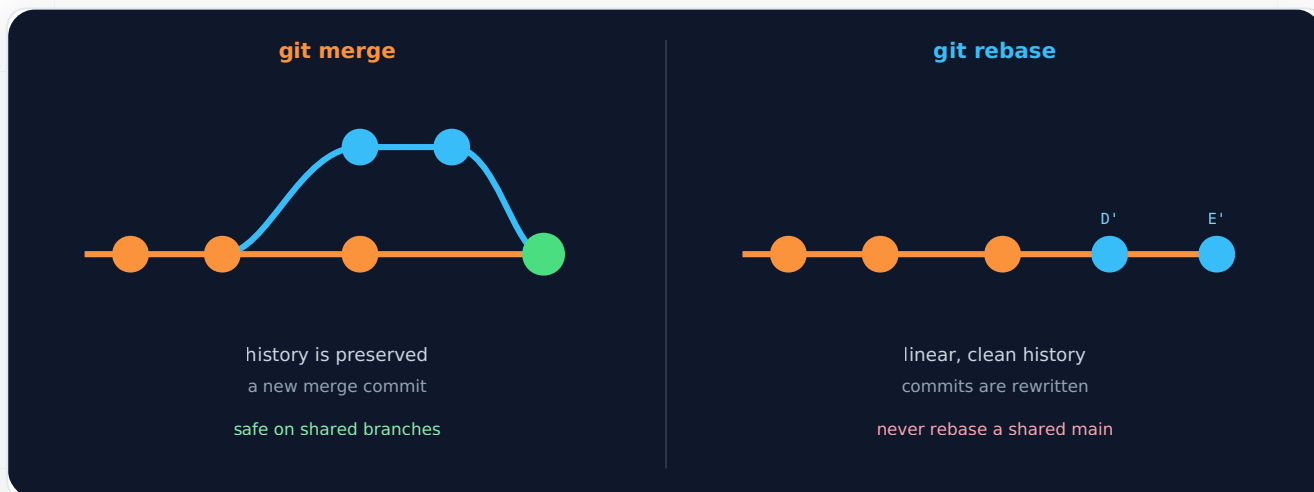
🎯 اهداف یادگیری

- ♦ fast-forward را از merge commit تشخیص دهی
- ♦ تعارض را در VS Code با دکمه‌های Accept حل کنی
- ♦ `--no-ff` و `--squash` را بدانی کی استفاده کنی
- ♦ abort کنی وقتی اوضاع خراب شد

🔗 سه شکل ادغام

```
سناریوی آموزشی
git switch main
git merge feat/auth
```

نوع	کی رخ می‌دهد	نتیجه
Fast-forward	از وقتی شاخه را ساختی، <code>main</code> هیچ کامیت جدیدی نگرفته	Git فقط برچسب <code>main</code> را روی همان کامیت‌های شاخه جلو می‌کشد؛ کامیت ادغام ساخته نمی‌شود
Merge commit	هر دو طرف بعد از جدایی کامیت جدید دارند	یک کامیت جدید با دو والد ساخته می‌شود
Squash merge	معمولاً دکمه GitHub	همه کامیت‌های شاخه در یک کامیت روی هدف خلاصه می‌شوند؛ جزئیات میانی در <code>main</code> نمی‌ماند



شکل ۵ – merge تاریخچه را حفظ می‌کند؛ rebase آن را خطی می‌کند (جزئیات در فصل ۲۳).

GitHub و --no-ff

بسیاری از تیم‌ها **Create a merge commit** را می‌خواهند تا هر PR یک گره قابل‌ردیابی داشته باشد. معادل محلی:
`git merge --no-ff feat/auth`

تعارض – آناتومی

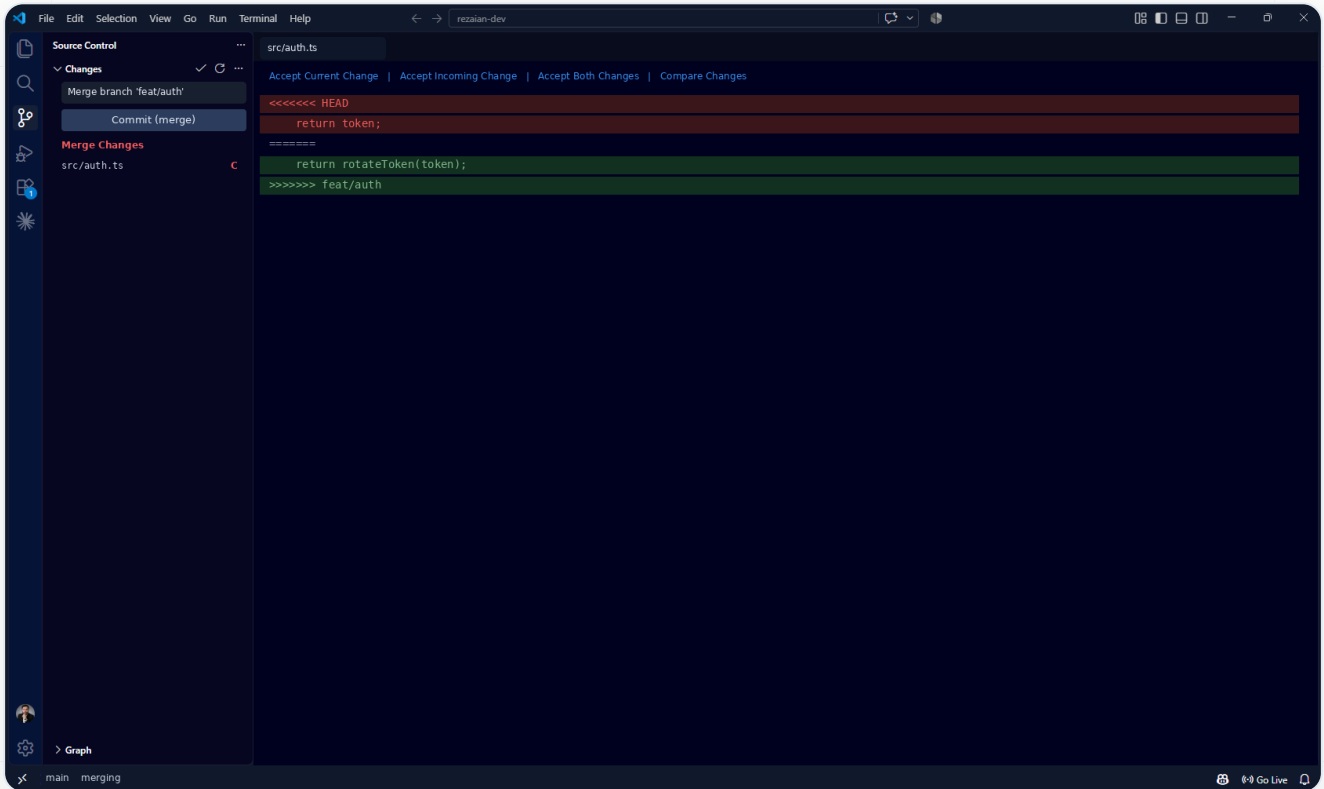
تعارض وقتی است که Git نمی‌تواند دو تغییر روی یک hunk را ترکیب کند.

نشانه‌های تعارض در فایل

```
<<<<<< HEAD
return token;
=====
return rotate(token);
>>>>>> feat/auth
```

سه نسخه در ذهن داشته باش:

- ◆ **Ours / Current** = شاخه‌ای که روی آن هستی (**HEAD**)
- ◆ **Theirs / Incoming** = شاخه‌ای که merge می‌کنی
- ◆ **Base** = جد مشترک (برای merge سه‌طرفه)



شکل ۶ – CodeLens: Accept Current, Accept Incoming, Accept Both, Compare Changes

◆ مسیر حل در VS Code

۱. Source Control بخش **Merge Changes** را نشان می‌دهد
۲. فایل را باز کن – نوار CodeLens بالای تعارض
۳. انتخاب کن یا دستی ویرایش کن
۴. فایل را **Stage** کن (علامت تعارض تمام می‌شود)
۵. Commit ادغام را کامل می‌کند

```
ترمینال همارز

git merge --abort          # پشیمانی قبل از commit
git merge --continue       # بعد از حل
git diff --name-only --diff-filter=U
```


Accept Both کورکورانه ⚠️

هر دو را پذیرفتن گاهی کد syntactically درست و semantically غلط می‌سازد. همیشه تست را اجرا کن.

استراتژی‌های پیشرفته merge 🧳

```
ابزارها

git merge -X ours feat/x      # در تعارض، ما برنده
git merge -X theirs feat/x
git log --merge               # فقط کامیت‌های درگیر تعارض
git checkout --conflict=diff3 -- file
```

diff3 جد مشترک را هم نشان می‌دهد – برای تعارض سخت طلایی است. در VS Code: **merge-diff3** **conflict.diff3": true** (یا Merge Editor).

Merge Editor VS Code سه‌ستونه (Incoming / Current / Result) را از Command Palette با **Merge Conflict: Open Merge Editor** باز می‌کند.

“سؤالات مصاحبه (با پاسخ)”

Junior – fast-forward یعنی چه؟ حرکت اشاره‌گر بدون کامیت جدید، چون تاریخچه خطی بوده.

Mid – squash در main تاریخچه را «دروغ» می‌کند؟ چون چند کامیت واقعی به یک کامیت تبدیل می‌شوند؛ **bisect** و **blame** جزئیات را از دست می‌دهند – ولی **PR** تمیز می‌ماند.

Senior – recursive در برابر **ort**؟ از Git ۲.۳۴ موتور پیش‌فرض **ort** است؛ سریع‌تر و پیش‌بینی‌پذیرتر از **recursive** قدیمی.

✓ چک‌لیست فهم

- ☐ سه نوع ادغام را بلدی
- ☐ تعارض را در VS Code حل کرده‌ای
- ☐ **merge --abort** را حفظی بلدی
- ☐ می‌دانی **squash** چه چیزی را پنهان می‌کند
- فصل بعد: Undo** – نجات بدون وحشت.

08 reflog و Undo – restore, reset, revert

Undo در Git یک فرمان نیست – چهار کار مختلف است. قبل از اجرا باید بدانی فایل روی دیسک را برمی‌گردانی، پیش‌نویس را خالی می‌کنی، اشاره‌گر شاخه را حرکت می‌دهی، یا یک کامیت جدید خنثی می‌سازی.

🎯 اهداف یادگیری

- ◆ نقشه Undo را از حفظ بکشی
- ◆ `restore` را جایگزین checkout فایل کنی
- ◆ سه حالت reset را از هم جدا کنی
- ◆ با `reflog` کامیت «گمشده» را زنده کنی

Undo map · which command, which tree?

Working Tree

git restore file
VS Code Discard
restores the file

Index

git restore --staged
Unstage in VS Code
removes from staging

Last commit

git commit --amend
git reset --soft HEAD~1
if not pushed yet

public history

git revert
revert the PR
never rewrite public history

golden rule: if others saw it, revert — never reset

شکل ۷ – قانون طلایی: تاریخچه عمومی را فقط با revert لمس کن.

Index و Working Tree 🖋️

بازگردانی فایل – مدرن

```
git restore src/auth.ts           # Working ← Index
git restore --staged src/auth.ts  # Index ← HEAD
git restore --source=HEAD --staged --worktree src/auth.ts
# هر دو درخت فایل = HEAD (checkout -- file)
git restore --source=main~3 README.md # نسخه قدیمی را بیاور
```

Terminal

```
git restore --staged git restore file
git git reset --soft HEAD~1 file
reflog برای نجات
```

VS Code Git

- = Discard Changes ↔ آیکون → Changes ♦
- restore restore --staged = Unstage - → Staged ♦
- = Commit → Undo Last Commit → ... منوی ... ♦
- reset --soft
- Timeline فایل را بعد از Discard هم گاهی نجات می‌دهد ♦
- Discard All خطرناک است ♦

سه چهرهٔ reset

فرض: A--B--C (HEAD) و می‌خواهی به B برگردی.

فرمان	HEAD	Index	Working	استفاده
--soft B	B	محتوای C	محتوای C	کامیت را بشکن، تغییرات بماند staged
--mixed B (پیش‌فرض)	B	B	محتوای C	کامیت را بشکن، تغییرات unstaged
--hard B	B	B	B	همه‌چیز برود

مثال‌های امن 

```
git reset --soft HEAD~1 # پیام/فایل‌ها را دوباره بچین
git reset HEAD~1 # unstaged همان، اما
# git reset --hard HEAD~1 # فقط اگر مطمئنی
```

در VS Code منوی ... در Source Control: Commit → Undo Last Commit معمولاً --soft .

✖ هرگز reset --hard روی main مشترک

اگر push شده، تاریخچهٔ دیگران را تکه‌پاره می‌کنی. به‌جایش git revert .

کامیت معکوس بساز

```
git revert HEAD          # آخرین را خنثی کن
git revert -m 1 <merge-sha> # merge (شاخه هدف = والد ۱) خنثی کردن
```

Revert یک کامیت جدید می‌سازد که تغییرات را برعکس اعمال می‌کند. تاریخچه دروغ نمی‌گوید.

در GitHub: روی PR دگمه **Revert** همین کار را تمیزتر انجام می‌دهد.

reflog – ماشین زمان اشاره‌گرها

هر حرکت HEAD در reflog ثبت می‌شود (معمولاً ۹۰ روز).

زنده کردن کامیت گمشده

```
git reflog
# e3a1... HEAD@{0}: reset: moving to HEAD~1
# 9f3a... HEAD@{1}: commit: feat(auth): ...

git switch -c rescue HEAD@{1}
# یا
git reset --hard 9f3a1c
```

i چه چیزی را reflog نجات نمی‌دهد؟

فایلی که هرگز **add/commit** نشده و Discard شده. Git آن را ندیده. از Local History خود VS Code (Timeline) کمک بگیر.

در VS Code: Timeline گاهی نسخه‌های محلی ذخیره می‌کند حتی بدون Git – آخرین تور ایمنی برای Untracked.

“سؤالات مصاحبه (با پاسخ)”

Junior – تفاوت restore و reset؟ restore فایل‌محور است؛ reset اشاره‌گر شاخه را حرکت می‌دهد.

Mid – soft در برابر mixed؟ هر دو HEAD را حرکت می‌دهند؛ soft ایندکس را نگه می‌دارد.

Senior – revert یک merge چرا 1 -m می‌خواهد؟ چون merge دو والد دارد. باید بگویی «خط اصلی کدام است» تا diff معکوس معنا داشته باشد.

چک لیست فهم ✓

- ☐ نقشه Undo را بلدی
- ☐ `restore` را به جای checkout فایل می زنی
- ☐ `--hard` را برای کار عمومی ممنوع می دانی
- ☐ یکبار با reflow نجات داده ای

فصل بعد: gitignore, attributes و LFS.

09 نادیده‌گرفتن و فایل بزرگ – gitignore، attributes، LFS

اگر فایل رمز (مثل `.env`) یک‌بار `git add` شود، حتی بعد از حذف از پوشه در تاریخچه می‌ماند. این فصل می‌گوید چه چیزی را از اول وارد Git نکنی و فایل بزرگ را کجا بگذاری.

اهداف یادگیری

- ♦ `.gitignore` مؤثر بنویسی و الگوی negating را بفهمی
- ♦ فایل track شده را بعداً ignore کنی
- ♦ Git LFS را برای باینری بزرگ راه بیندازی
- ♦ `.gitattributes` را برای eol و diff تنظیم کنی

gitignore – الگوها

```
.gitignore - نمونه Node/Python

# وابستگی
node_modules/
.venv/
dist/
coverage/

# راز
.env
.env.*
!.env.example

# سیستم
.DS_Store
Thumbs.db

# می‌کنند commit را vscode - ادیتور - اختیاری: بعضی تیم‌ها
.idea/
```

اولویت: الگوی محلی فایل → پوشه والد → ریشه → `.git/info/exclude` → `core.excludesFile` جهانی.

```
چرا هنوز دیده می‌شود؟

git check-ignore -v src/.env      # کدام قانون خورد؟
git rm --cached secrets.json      # بردار، فایل روی دیسک بماند Index از
```

✖ فایل track شده ignore نمی‌شود

اگر قبلاً add شده، باید `git rm --cached` کنی. فقط نوشتن در gitignore کافی نیست.

در VS Code فایل‌های ignore شده در Explorer کم‌رنگ‌اند. Source Control آن‌ها را نشان نمی‌دهد مگر **Show Ignored**.

🌐 قالب جهانی

سراسری excludes

```
git config --global core.excludesFile ~/.gitignore_global
echo ".DS_Store" >> ~/.gitignore_global
```

📦 Git LFS – باینری را در تاریخچه خفه نکن

PDF، مدل سه‌بعدی، وزن شبکه عصبی و فیلم با هر ویرایش کل blob را تکرار می‌کنند.

LFS

```
git lfs install
git lfs track "*.pdf"
git lfs track "*.psd"
git add .gitattributes
git add docs/book.pdf
git commit -m "chore: store pdf in LFS"
git lfs ls-files
```

روی GitHub فضای LFS جداگانه دارد (سه‌میه). برای کتاب و بازی رایج است؛ برای کد منبع لازم نیست.

 .gitattributes

```
* text=auto eol=lf
*.sh text eol=lf
*.bat text eol=crlf
*.png binary
*.pdf filter=lfs diff=lfs merge=lfs -text
package-lock.json merge=binary # قفل JSON در هم merge جلوگیری از
```

`eol=lf` جنگ CRLF ویندوز/لینوکس را در تیم‌های چندسیستمی می‌کشد. در VS Code `"files.eol": "\n"`.

🔖 قالب github

فایل `.gitattributes` با `linguist-vendored` می‌تواند آمار زبان مخزن را درست کند – مفید برای `.monorepo`.

“سؤالات مصاحبه (با پاسخ)”

Junior – node_modules را **commit** کنیم؟ هرگز. قفل (`package-lock`) را بله، پوشه را خیر.
Mid – تفاوت exclude و gitignore؟ `info/exclude` شخصی و غیرقابل پوشش است؛ `gitignore` تیمی است.
Senior – LFS pointer چیست؟ فایل متنی کوچک با OID و اندازه که به جای باینری در Git ذخیره می‌شود؛ محتوا روی LFS store است.

✅ چک لیست فهم

☐ `.env` در تاریخچه نیست

☐ `check-ignore -v` را بلدی

☐ برای PDF از LFS استفاده می‌کنی

☐ `eol` تیم یکسان است

فصل بعد: Source Control در VS Code از صفر – میانبر `Ctrl+Shift+G`، با عکس نزدیک.

10 Source Control از صفر – Ctrl+Shift+G

خانهٔ روزانهٔ Git در VS Code پنل Source Control است. میانبر ثابت این کتاب Ctrl+Shift+G است. اینجا هر کلیک را به ترتیب می‌نویسیم و برای هر تفاوت («چرا مال من این‌طور است؟») دلیل دقیق می‌آید.

🎯 اهداف یادگیری

- ♦ پنل را با کلیک مشخص باز کنی و بدانی بعد از آن باید چه ببینی
- ♦ اگر دکمه یا لیست تو با عکس کتاب یکی نبود، در همان لحظه علت را بدانی
- ♦ یک فایل را از Save تا Commit، کلیک‌به‌کلیک، تمام کنی
- ♦ Stage، Unstage، Discard و Stage Selected Ranges را بدون حدس بزنی

🧭 قبل از اولین کلیک – عکس کتاب مال یک حالت است

عکس‌های این فصل از یک VS Code واقعی‌اند: ویندوز، زبان انگلیسی، تم تیره، مخزنی که Working Tree تمیز است و origin پانزده کامیت جلوتر. صفحهٔ تو باید دکمه‌های همان‌جا را داشته باشد، نه لزوماً همان عدد ۱۵ یا همان نام فایل.

چیزی که در عکس می‌بینی	مال تو ممکن است	چرا؟
برچسب انگلیسی Source Control ، Sync Changes ، Commit	فارسی: «کنترل منبع»، «ثبت»، «همگام‌سازی تغییرات»	زبان نمایش VS Code. موقعیت دکمه یکی است؛ فقط نوشته عوض می‌شود.
تم تیرهٔ سرمه‌ای	تم روشن یا تم دیگر	رنگ عوض می‌شود؛ نام فرمان‌ها عوض نمی‌شود.
Sync Changes 15↓	Commit یا Publish Branch یا هیچ	بستگی دارد تغییرات محلی داری یا نه، و آیا origin هست یا نه – پایین، سه حالت.
حرف M کنار src/auth.ts	فایل و حرف دیگر، یا لیست خالی	مثال کتاب است. تو فایل خودت را می‌بینی.
Graph پایین پنل	Graph نیست	نسخهٔ قدیمی VS Code، یا بخش جمع شده؛ فلش > را باز کن.
میانبر Ctrl+Shift+G	در مک گاهی باید Cmd نباشد؛ همین Ctrl است	در VS Code روی مک هم پیش‌فرض Ctrl+Shift+G است نه Cmd.

i زبان عکس‌ها انگلیسی است عمداً

در شکل‌ها متن فارسی نمی‌گذاریم (برعکس می‌شود). اگر VS Code تو فارسی است، همان دکمه را از روی **جایگاه** پیدا کن نه از روی املای کتاب.

🖱️ باز کردن پنل – سه کلیک معادل، یکی کافی است

یکی از این سه را بزن؛ هر سه باید همان ستون چپ را باز کنند:

۱. صفحه کلید: **Ctrl+Shift+G** (ویندوز، لینوکس، مک)

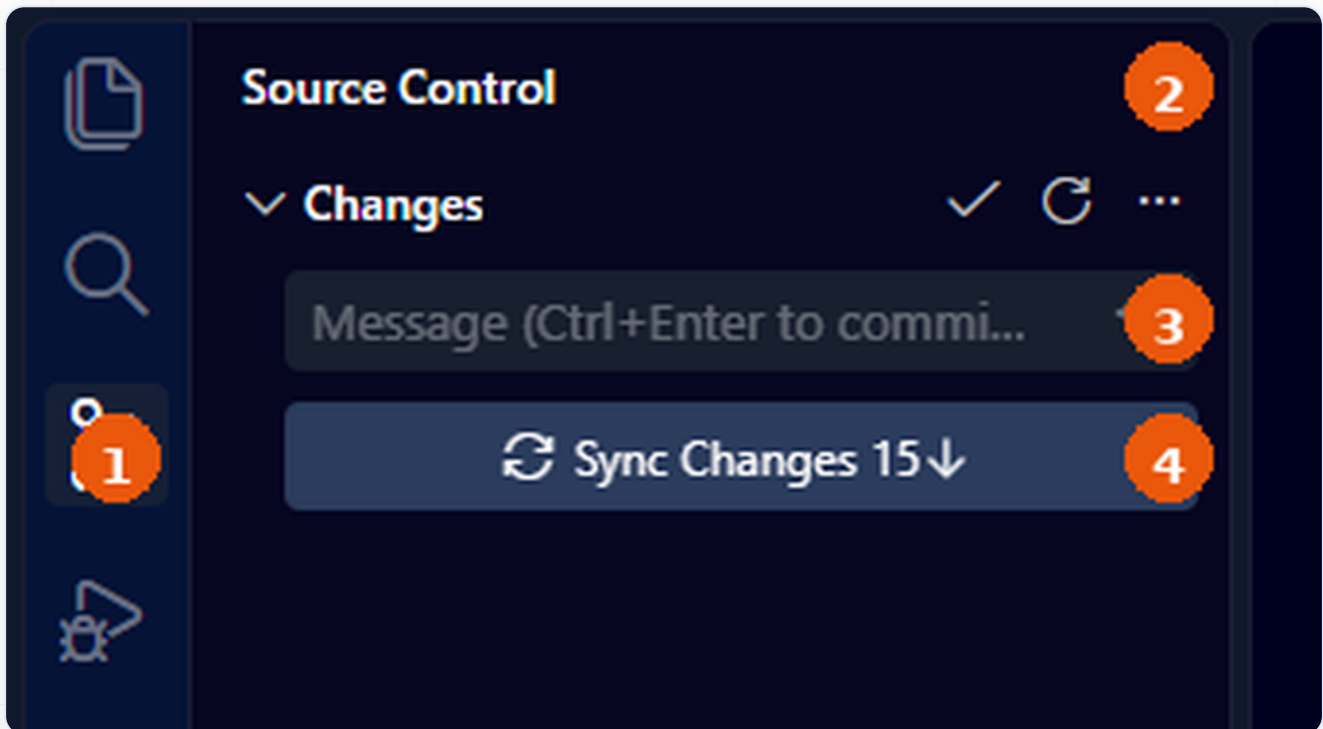
۲. ماوس: در نوار باریک چپ (Activity Bar) روی آیکون شاخه/Git کلیک کن – در عکس کتاب شماره ۱

۳. اگر هیچ‌کدام کار نکرد: **Ctrl+Shift+P** → تایپ کن **View: Show Source Control** → Enter

بعد از باز شدن باید ببینی: ستون سمت چپ با عنوان Source Control. ادیتور سمت راست سر جایش می‌ماند.

اگر مال تو این‌طور نیست:

چه دیدی	علت	چه کار کن
هیچ پنلی باز نشد	فوکوس روی ترمینال است و میانبر قورت داده شد	یک بار روی ادیتور کلیک کن، دوباره Ctrl+Shift+G
پنل Explorer باز شد نه Git	آیکون اشتباه (دو سند بالا)	آیکون سوم/چهارم Activity Bar؛ یا همان Palette
ستون آمد ولی نوشته Git not found / Git not installed	VS Code فایل git.exe را پیدا نکرده	→ Settings . Git: Show Git Output → Palette git.path را به مسیر نصب Git بده. از نو VS Code را باز کن
ستون آمد، کاملاً خالی، بدون Message	پوشه باز نیست	File → Open Folder روی پوشه پروژه



شکل ۹ – برش نزدیک. شماره‌ها را با فهرست زیر تطبیق بده. دکمه آبی اینجا Sync است نه Commit؛ برای تو ممکن است برعکس باشد.

1. **کلیک روی آیکون Git** در Activity Bar. اگر روی آیکون **عدد** بود یعنی فایل تغییر کرده داری. اگر عدد نبود، یا Working Tree تمیز است یا نشان روی آیکون دیگری است (در عکس کامل کتاب، نشان آبی روی Extensions است نه Git – این باگ نیست).
2. **عنوان Source Control** و سه نقطه **...** بالا-راست. یک کلیک روی **...** منوی Fetch / Pull / Push / Stash را باز می‌کند (فصل ۱۱).
3. **یک کلیک داخل جعبه Message** – مکان‌نما باید چشمک بزند. این‌جا متن کامیت را می‌نویسی. **Ctrl+Enter** فقط وقتی کار می‌کند که فوکوس همین جعبه باشد.
4. **دکمه بزرگ زیر جعبه** – برچسبش **ثابت نیست**. VS Code بر اساس وضعیت مخزن یکی از این‌ها را می‌نویسد: **Initialize Repository**، **Commit**، **Sync Changes**، یا **Commit (merge)**.

پایین همان ستون، بدون عوض کردن پنل:

♦ **Graph** – یک کلیک روی > یا روی کلمه Graph. اگر نبود، VS Code را به روز کن یا **Git: View** → **Palette** یا **History**.

♦ **نوار وضعیت پایین-چپ پنجره** (نه داخل ستون): نام شاخه. یک کلیک روی نام شاخه = فهرست عوض کردن شاخه.



شکل ۹ – یک کلیک روی main. عدد ↓ و ↑ مال این مخزن است؛ مال تو و هم طبیعی است.

اگر **نوار وضعیت شاخه نشان نمی‌دهد**: یا پوشه Git ندارد، یا Git پیدا نشده. اول همان جدول «اگر مال تو این‌طور نیست» را بالا ببین.

🙄 | دکمه بزرگ زیر Message – از روی برچسب بفهم کجایی

VS Code یک دکمه اصلی دارد. برچسب را بخوان؛ حدس نزن.

♦ حالت ۱ – نوشته Initialize Repository / Publish to GitHub

یعنی پوشه‌ای که Open Folder کرده‌ای فایل `.git` ندارد.

SOURCE CONTROL

The folder currently open doesn't have a Git repository.

Initialize Repository

Publish to GitHub

Or open a folder that already contains a `.git` directory.

شکل ۹ – فقط وقتی `.git` نیست این دو دکمه را می‌بینی.

کلیک‌ها:

۱. `File → Open Folder` – پوشه خود پروژه را انتخاب کن، نه درایو و نه پوشه والد.

۲. `Ctrl+Shift+G`.

۳. یک کلیک روی **Initialize Repository**. معادل `git init`. بعد از کلیک باید جعبه Message ظاهر شود و اگر فایلی باشد، لیست Changes پر شود.

۴. همان لحظه فایل `.gitignore` بساز (`Ctrl+N` ، ذخیره با همین نام) تا `node_modules` در Changes نیاید.

۵. **Publish to GitHub** را فقط اگر می‌خواهی ریپو روی GitHub ساخته شود بزن. اگر بزنی و وارد حساب نباشی، پنجره ورود مرورگر باز می‌شود – این خطا نیست.

اگر بعد از **Initialize** هنوز همان دو دکمه را می‌بینی: پوشه اشتباه باز است. در Explorer باید `.git` را (گاهی مخفی) ببینی. `File → Open Folder` را تکرار کن.

اگر **Publish** خاکستری است یا خطا می‌دهد: **Accounts** پایین `Activity Bar → Sign in to GitHub` بدون ورود، **Publish** کار نمی‌کند؛ **Initialize** کار می‌کند.

♦ حالت ۲ – نوشته Sync Changes (شاید با عدد ↓ یا ↑)

یعنی مخزن هست، فایل ذخیره‌شده تغییر کرده برای کامیت نداری، ولی با `origin` هم‌تراز نیستی.

♦ `۱۵↓` = روی سرور ۱۵ کامیت هست که تو نداری → Sync تقریباً `pull` است.

♦ `↑۳` = تو ۳ کامیت داری که سرور ندارد → Sync تقریباً `push` است.

♦ هر دو = واگرایی؛ Sync ممکن است `merge` بسازد.

لیست Changes خالی در این حالت موفقیت است (`nothing to commit, working tree clean`)، نه اینکه Git خراب باشد.

اگر **Sync** را زدی و از تو تأیید خواست: تنظیم `git.confirmSync` روشن است – **Yes** بگو یا در Settings خاموش کن.

اگر **Sync** خطا داد (`rejected / diverged`): فصل ۱۱ – اول `Fetch`، بعد `Pull` یا `Force Pull (Rebase)` زن.

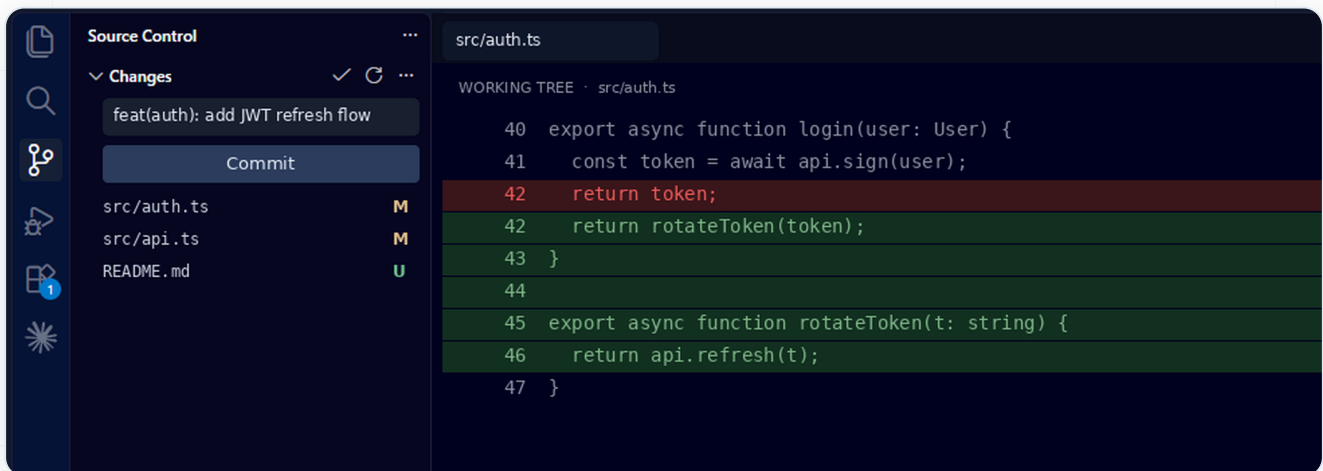
♦ حالت ۳ – نوشته Commit (یا Commit با فلش ▼)

یعنی حداقل یک فایل در Changes یا Staged هست.

اگر تو **Commit** می‌بینی و کتاب **Sync**: تو فایل ذخیره‌نشده یا تغییر کرده داری؛ کتاب در آن عکس Working Tree تمیز نشان می‌دهد. هر دو درست‌اند.

مسیر کامل کامیت – کلیک به کلیک

هدف: یک فایل را عوض کنی و در تاریخچه ثبت شود.



شکل ۱۰ – بعد از مرحله ۴ باید چیزی شبیه این ببینی: پیام، فایل‌ها با diff، M/U، سبز/قرمز.

۱. فایل را در ادیتور عوض کن و **Ctrl+S** بزن.

باید ببینی: روی آیکون Git عدد می‌آید؛ فایل در Changes ظاهر می‌شود.

اگر نیامد: Save نشده، یا فایل داخل **.gitignore** است، یا پوشه دیگری Open است.

۲. **Ctrl+Shift+G**.

باید ببینی: نام فایل + حرف **M** یا **U**.

۳. یک کلیک روی نام فایل در **Changes**.

باید ببینی: سمت راست، diff. خط قرمز = حذف، سبز = اضافه.

اگر سمت راست همان **Welcome VS Code** ماند: روی نام فایل کلیک نشده؛ روی فضای خالی ستون کلیک کرده‌ای.

۴. یک کلیک روی **+** سمت راست همان فایل (وقتی نشانگر روی ردیف است ظاهر می‌شود). معادل **git add**.

باید ببینی: فایل از **Changes** به **Staged Changes** می‌پرد.

اگر **+** نبود: نشانگر را روی ردیف فایل ببر. در بعضی تم‌ها **+** فقط hover است. یا **Git: Stage** → **Palette**.

۵. یک کلیک داخل جعبه **Message** و بنویس مثلاً **feat(auth): add JWT refresh flow**.

اگر جعبه غیرفعال است: هنوز **Staged** نداری و **Smart Commit** خاموش است – مرحله ۴ را بزن. یا فوکوس روی ادیتور است.

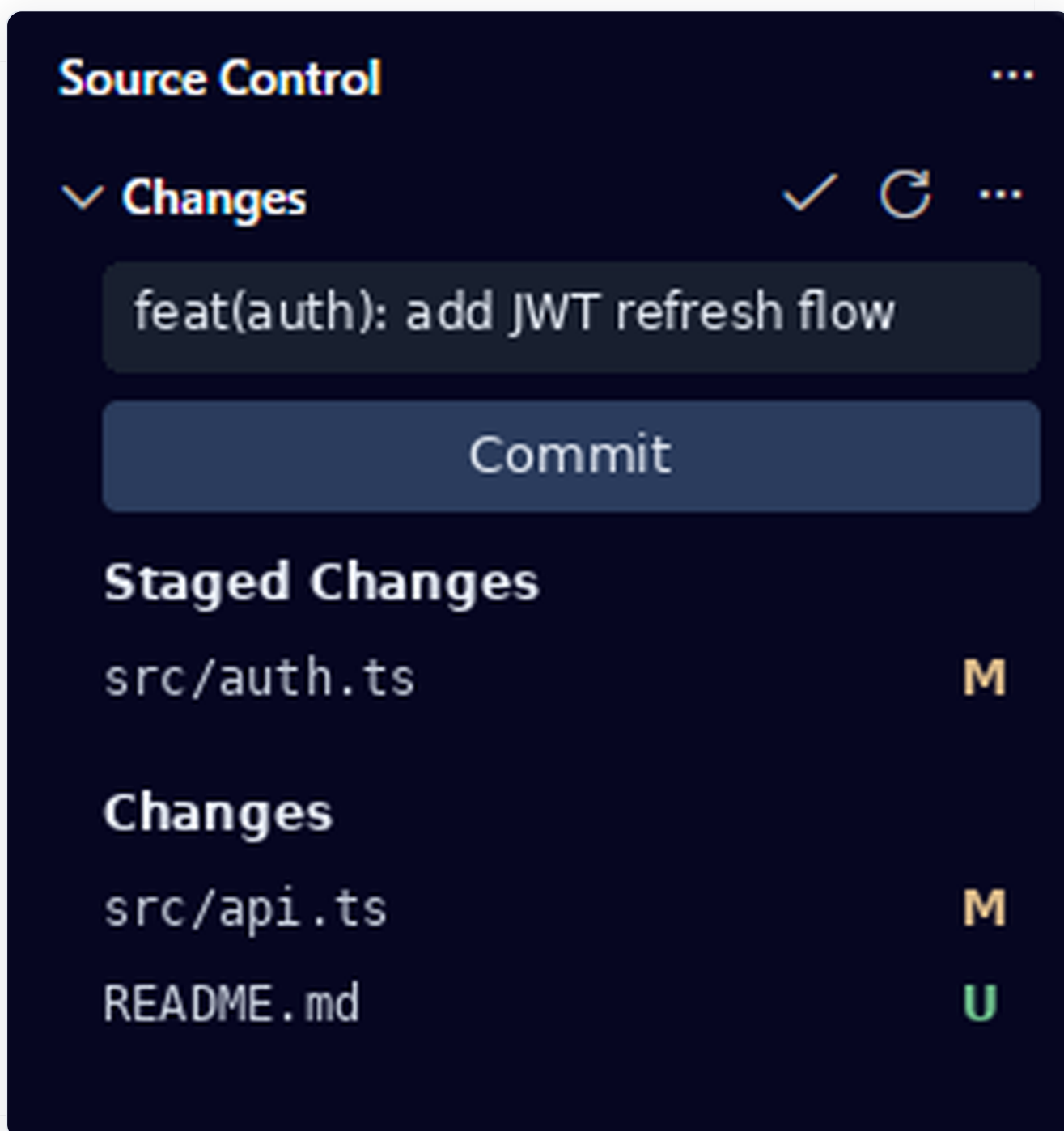
۶. **Ctrl+Enter** (مک: **Cmd+Enter**) یا یک کلیک روی دکمه **Commit**.

باید ببینی: لیست خالی می‌شود، دکمه ممکن است به **Sync/Publish** عوض شود، نوار وضعیت **↑1** بگیرد اگر remote داری.

اگر کلیک **Commit همه فایل‌ها را برد نه فقط یکی:** `git.enableSmartCommit` روشن است. Settings → این گزینه را Off کن. از این به بعد بدون Stage، دکمه Commit کار نمی‌کند یا هشدار می‌دهد.

اگر خطا **Signing failed / secret key**: فصل ۲ – امضا روشن است ولی کلید SSH signing روی این ماشین نیست. موقتاً `git.enableCommitSigning` را Off کن یا کلید را بساز.

اگر خطا **Please tell me who you are**: `user.name` و `user.email` ست نشده – فصل ۲.



شکل ۱۰ب – بالا می‌رود داخل کامیت؛ پایین هنوز نمی‌رود. اگر فقط یک لیست می‌بینی، یا چیزی Stage نکرده‌ای یا Untracked جداست.

می‌خواهی	کلیک دقیق	باید ببینی
فایل برود برای کامیت	+ روی ردیف در Changes	ردیف می‌پرد به عنوان Staged Changes
پشیمان از Stage	- روی ردیف در Staged	برمی‌گردد به Changes؛ فایل روی دیسک همان است
همه را Stage	+ روی نوار عنوان Changes (نه روی فایل)	همه می‌روند بالا
تغییرات فایل روی دیسک دور ریخته شود	راست‌کلیک روی ردیف Changes → Discard Changes → تأیید	فایل از لیست می‌رود؛ محتوای ادیتور به نسخه قبلی برمی‌گردد

اگر **Staged Changes** را به‌عنوان عنوان جدا نمی‌بینی: هنوز + نزدی. با Smart Commit روشن، ممکن است بدون این عنوان هم Commit همه را بگیرد – برای همین خاموشش کردیم.

اگر **Discard** را زدی و فایل برگشت: Untracked (U) را Discard کردن یعنی حذف فایل از دیسک. VS Code معمولاً «Are you sure?» می‌پرسد. اگر پرسید و Yes گفتی، از Timeline برگردان (پایین فصل).

اگر **Untracked** ها ته یک لیست قاطی‌اند نه گروه جدا: Settings → `git.untrackedChanges` را بگذار `separate`.

hunk – سه کلیک آبی روی خود diff

hunk = چند خط سبز/قرمز پشت‌سرهم. معادل `git add -p`.

```
src/auth.ts
Stage Change | Stage Selected Ranges | Discard Change
40 export async function login(user: User) {
41   const token = await api.sign(user);
42   return token;
42   return rotateToken(token);
43 }
44
45 export async function rotateToken(t: string) {
46   return api.refresh(t);
47 }
```

شکل ۱۰ – این سه لینک آبی فقط وقتی فایل را از Changes باز کرده باشی بالای hunk ظاهر می‌شوند.

۱. در Changes یک کلیک روی فایل.

۲. در diff، خطوطی که می‌خواهی را با ماوس رنگ کن (یا هیچی انتخاب نکن اگر کل hunk را می‌خواهی).

۳. یک کلیک روی یکی از سه لینک آبی بالای hunk:

◆ **Stage Change** = کل این hunk

◆ **Stage Selected Ranges** = فقط خطوط انتخاب شده

◆ **Discard Change** = همین hunk از روی دیسک برگردد

باید ببینی: اگر فقط بخشی Stage شد، فایل هم در Staged می ماند هم در Changes (حرف گاهی M دو جا). این درست است.

اگر لینک آبی نبود:

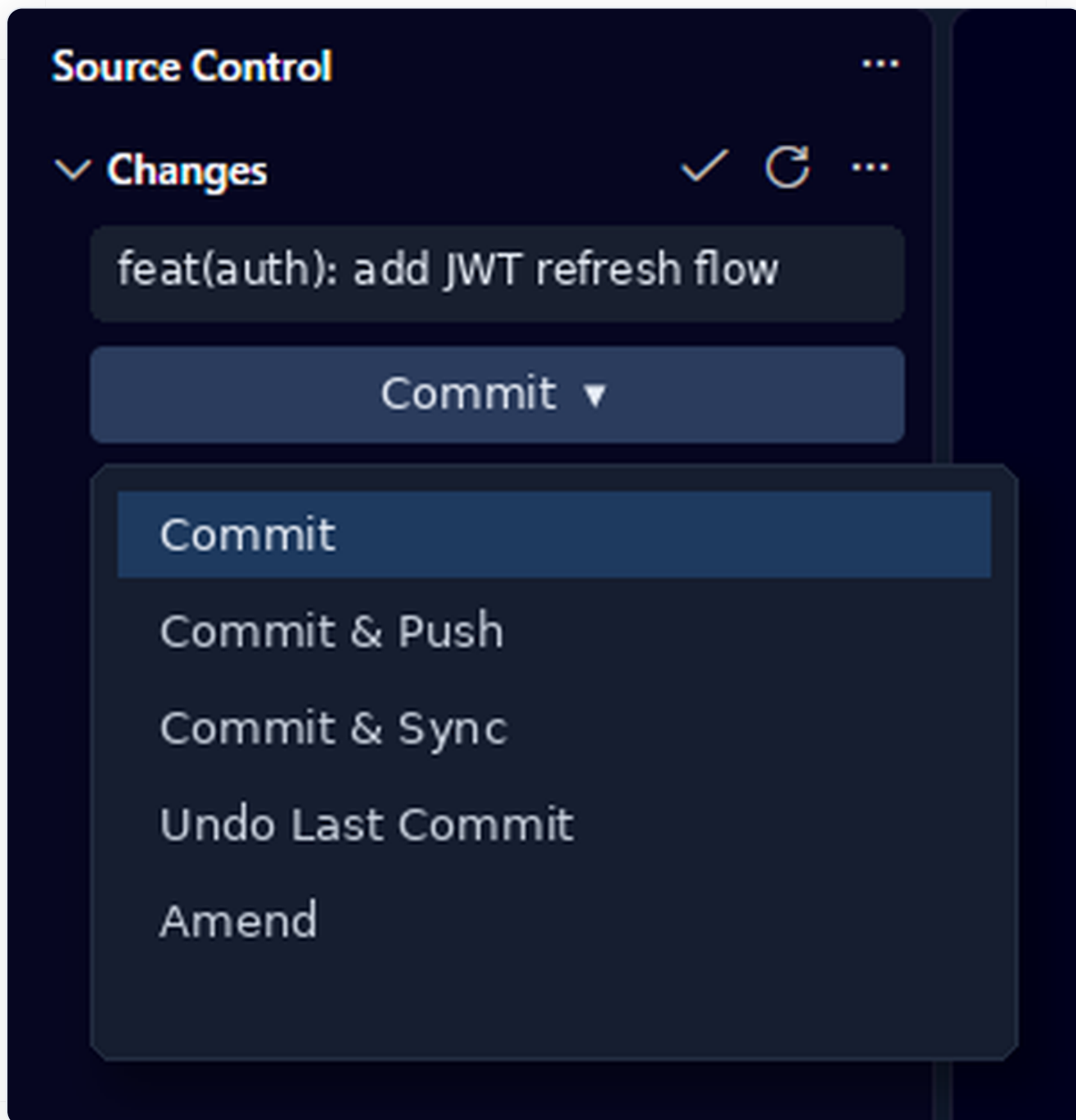
◆ diff را از Timeline باز کرده ای نه از Changes – ببند، از Source Control روی فایل کلیک کن.

◆ روی نمای پیش نمایش Markdown هستی نه متن – روی تب فایل خام کلیک کن.

◆ تنظیم `diffEditor.renderSideBySide` فرق ندارد؛ لینک ها باید باشند. VS Code را به روز کن.

▼ فلش کنار Commit – یک کلیک، پنج فرمان

یک کلیک روی فلش ▼ سمت راست دکمه Commit (نه خود Commit).



شکل ۱۰ – اگر فلش را نمی‌بینی، دکمه هنوز Commit نیست (Sync/Publish است) یا عرض ستون خیلی کم است؛ ستون را بکش عریض‌تر کن.

کلیک روی	چه می‌شود	اگر مال تو خاکستری / نیست
Commit	فقط ثبت محلی	باید Staged یا Smart Commit داشته باشی
Commit & Push	ثبت + ارسال به origin	origin نباشد → اول Publish Branch
Commit & Sync	ثبت + pull/push	tracking نداشته باشی، Push جدا بزن
Undo Last Commit	آخرین کامیت باز می‌شود، فایل‌ها Staged می‌مانند	کامیت نداری
Amend	آخرین کامیت را با Stage فعلی و پیام جدید عوض می‌کند	بعد از push نزن

⚠ Amend بعد از push

هش عوض می‌شود. اگر همکار آن کامیت را گرفته، تاریخچه دوباره می‌شود.

🗂 میانبر وقتی فوکوس کجاست؟

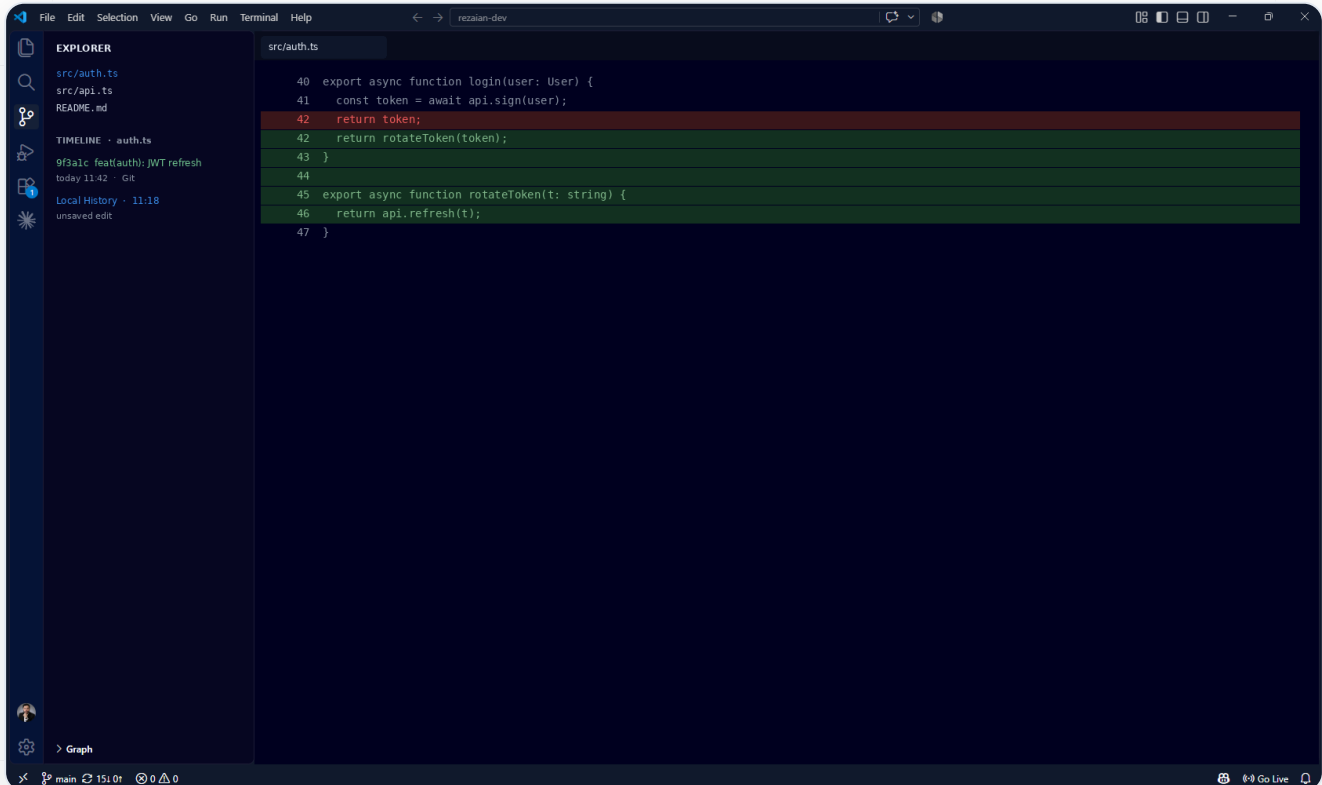
Ctrl+Enter فقط داخل جعبه Message کامیت می‌زند. اگر فوکوس روی ادیتور باشد، ممکن است کار دیگری بکند یا هیچ.

یک کلیک داخل Message، بعد **Ctrl+Enter**. اگر عادت شد: بعد از Stage، کلیک Message، میانبر.

کار	کلیک / کلید	شرط
باز کردن پنل	Ctrl+Shift+G	—
کامیت	Ctrl+Enter	فوکوس روی Message
Palette	Ctrl+Shift+P Git: سپس	—
Diff	Enter روی ردیف فایل	فوکوس لیست Changes
Stage همه	Git: Stage All Changes	—

Timeline – این یکی داخل Source Control نیست

کلیک‌ها: آیکون Explorer بالا-چپ Activity Bar (دو سند) → فایل را در درخت انتخاب کن → پایین همان ستون تب Timeline را باز کن (اگر جمع است، عنوان Timeline را کلیک کن).



شکل ۱۸ – اگر Timeline خالی است، فایل را انتخاب نکرده‌ای یا Git در این پوشه نیست.

یک کلیک روی یک ردیف Timeline آن نسخه را مقایسه می‌کند. Local History برای Discard فایل Untracked است.

تمرین – هر کدام را تا «باید ببینی» پیش برو

تمرین ۱. پوشه خالی Open Folder → باید Initialize ببینی → کلیک Initialize → `README.md` بساز و Save → باید U در Changes → `docs: add readme` Message → `Ctrl+Enter` → باید لیست خالی شود.

تمرین ۲. سه فایل را Save کن → فقط روی یکی `+` → `Commit` → باید دو فایل در Changes مانده باشد. اگر هر سه رفتند، Smart Commit را Off کن و تمرین را تکرار کن.

تمرین ۳. یک فایل را چند خط عوض کن → از Changes باز کن → فقط بعضی خط‌ها را Stage Selected Ranges بزن → باید فایل هم بالا هم پایین باشد.

تمرین ۴. Discard یک `M` → از Timeline برگردان.

“ سؤالات مصاحبه (با پاسخ)

Junior – چرا Save کافی نیست؟ Save فقط فایل روی دیسک را عوض می‌کند. تاریخچه با کلیک Commit ساخته می‌شود.

Mid – چرا مال من دکمه Commit است و در عکس Sync؟ چون تو تغییر ذخیره‌شده داری و عکس کتاب Working Tree تمیز + origin جلوتر را نشان می‌دهد. دکمه بر اساس وضعیت عوض می‌شود.

Senior – Publish to GitHub دقیقاً بعد از چند کلیک چه می‌سازد؟ ریپو، `origin`، و `push -u`. بدون Sign in، همان کلیک پنجره ورود باز می‌کند نه خطا.

✓ | چک لیست فهم

- ☐ سه راه باز کردن پنل را بلدی و اگر باز نشد جدول بالا را بلدی
 - ☐ از روی برچسب دکمه بزرگ می‌گویی در کدام حالتی
 - ☐ مسیر Save → کلیک فایل → + → Message → Ctrl+Enter → بدون حدس می‌روی
 - ☐ اگر صفحه با عکس یکی نبود، ستون «چرا؟» را می‌دانی نه اینکه کتاب غلط است
- فصل بعد:** همان پنل، سطح فوق پیشرفته – با همین قرارداد «کلیک / باید ببینی / اگر فرق داشت».

11 Source Control فوق‌پیشرفته – Graph تا PR

Graph، شاخه، Sync، تعارض، rebase و PR همه از همان `Ctrl+Shift+G` می‌آیند. مثل فصل ۱۰: هر کار یعنی چند کلیک مشخص، چه باید ببینی، و اگر مال تو فرق داشت یعنی چه.

🎯 اهداف یادگیری

- ♦ Graph را با کلیک باز کنی و از راست‌کلیک روی کامیت نترسی
- ♦ Fetch / Pull / Push / Sync را از روی منو جدا بزنی و بدانی چرا دکمه تو با کتاب یکی نیست
- ♦ تعارض و rebase را تا آخرین کلیک (Continue یا Abort) تمام کنی
- ♦ اگر Incoming، Graph، یا افزونه PR نبود، علت را بدانی

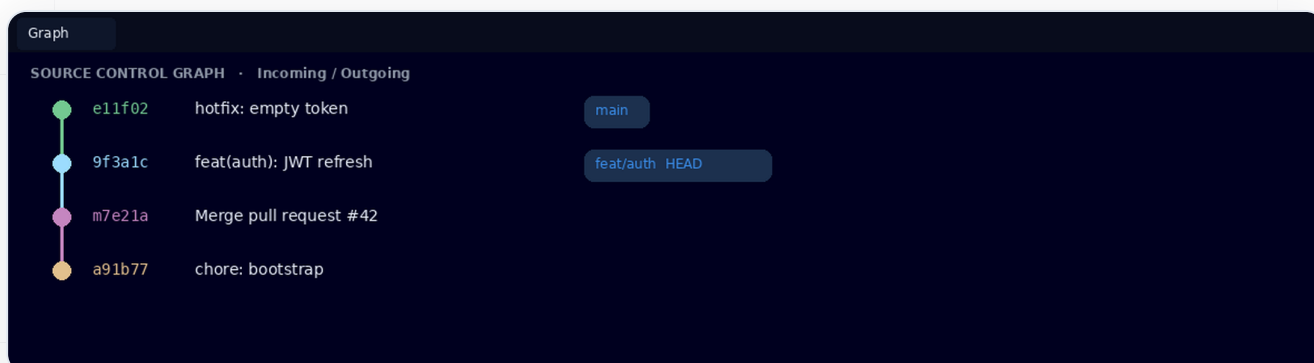
📊 Graph – باز کردن و راست‌کلیک

کلیک‌ها برای باز شدن:

۱. `Ctrl+Shift+G`.

۲. ته ستون Source Control روی **Graph** یک کلیک (اگر > دیدی، همان را باز کن).

۳. اگر Graph نبود: `Enter` → `Git: View History` → `Ctrl+Shift+P`.



شکل ۱۱ – برش نزدیک. رنگ‌ها و هش‌ها مثال‌اند؛ تو کامیت‌های مخزن خود را می‌بینی.

باید ببینی: خط‌های رنگی، هش کوتاه، پیام کامیت، گاهی برچسب `HEAD / main`.

اگر مال تو این طور نیست:

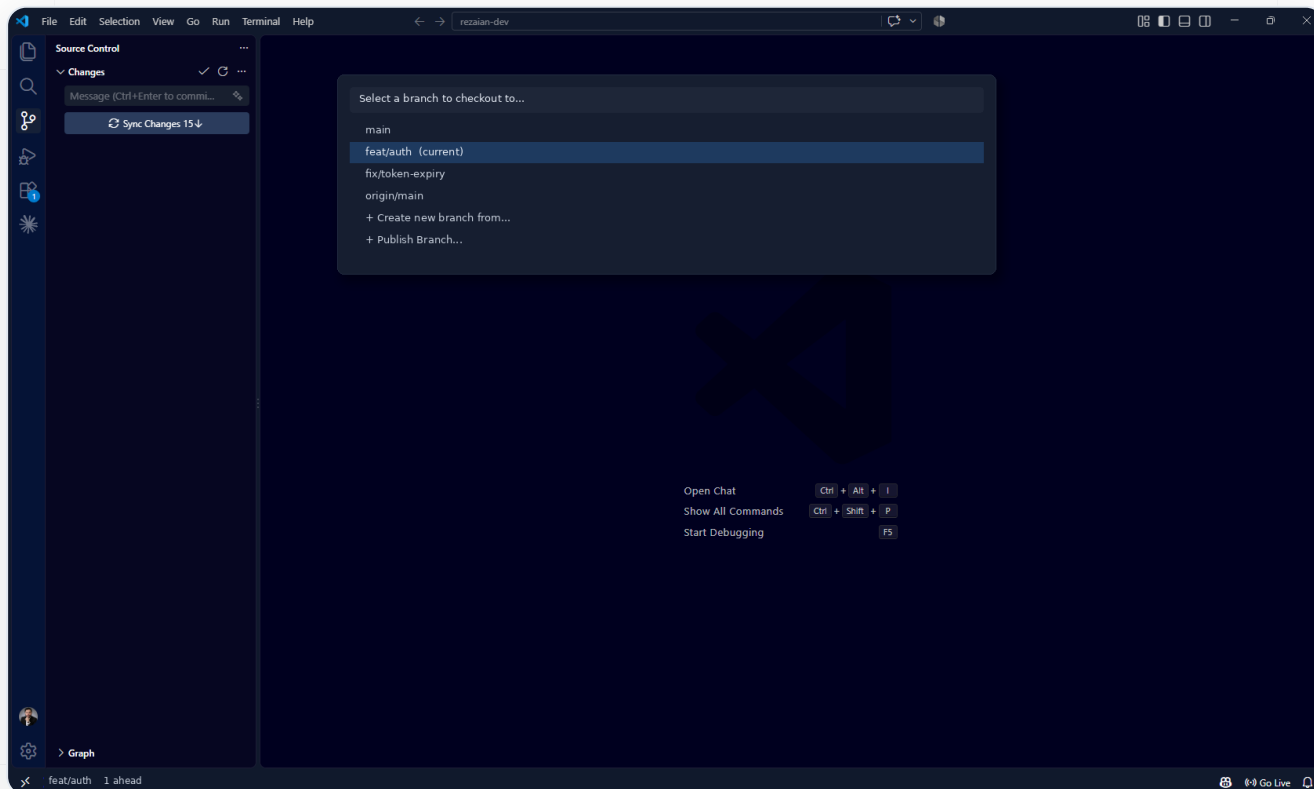
چه دیدی	علت	چه کار کن
اصلاً Graph در ستون نیست	VS Code قدیمی تر از حدود ۱.۹۷	VS Code را به روز کن؛ یا افزونه Git Graph / GitLens
Graph باز شد ولی یک نقطه است	مخزن فقط یک کامیت دارد	طبیعی است
Incoming خالی است با اینکه همکار push کرده	Fetch نشده	منوی Fetch → (پایین)
راست کلیک منوی کوتاه دارد	روی پیام کلیک کرده ای نه روی گره، یا افزونه منو را عوض کرده	روی دایره کامیت راست کلیک کن

راست کلیک روی یک دایره / ردیف کامیت، بعد یک کلیک روی فرمان:

فرمان	بعد از کلیک چه می شود	اگر ترسیدی
Checkout	HEAD می رود روی آن کامیت	اگر شاخه نبود، نوار زرد detached می آید → فوری Create Branch
Create Branch	جعبه نام می پرسد	Enter بزن؛ معادل <code>git switch -c</code>
Cherry Pick	یک کامیت جدید روی شاخه فعلی	تعارض آمد = فصل تعارض همین صفحه
Revert	یک کامیت خنثی کننده می سازد	برای main مشترک همین را بزن نه Reset
Reset Current Branch to Here	می پرسد Soft/Hard	Hard را روی شاخه مشترک نزن
Compare with HEAD	diff باز می شود	بی خطر

عوض کردن و ساختن شاخه – از نوار وضعیت

کلیک ۱: پایین-چپ پنجره، روی نام شاخه (مثلاً `main`).
باید ببینی: فهرست بالا وسط ادیتور (Quick Pick).



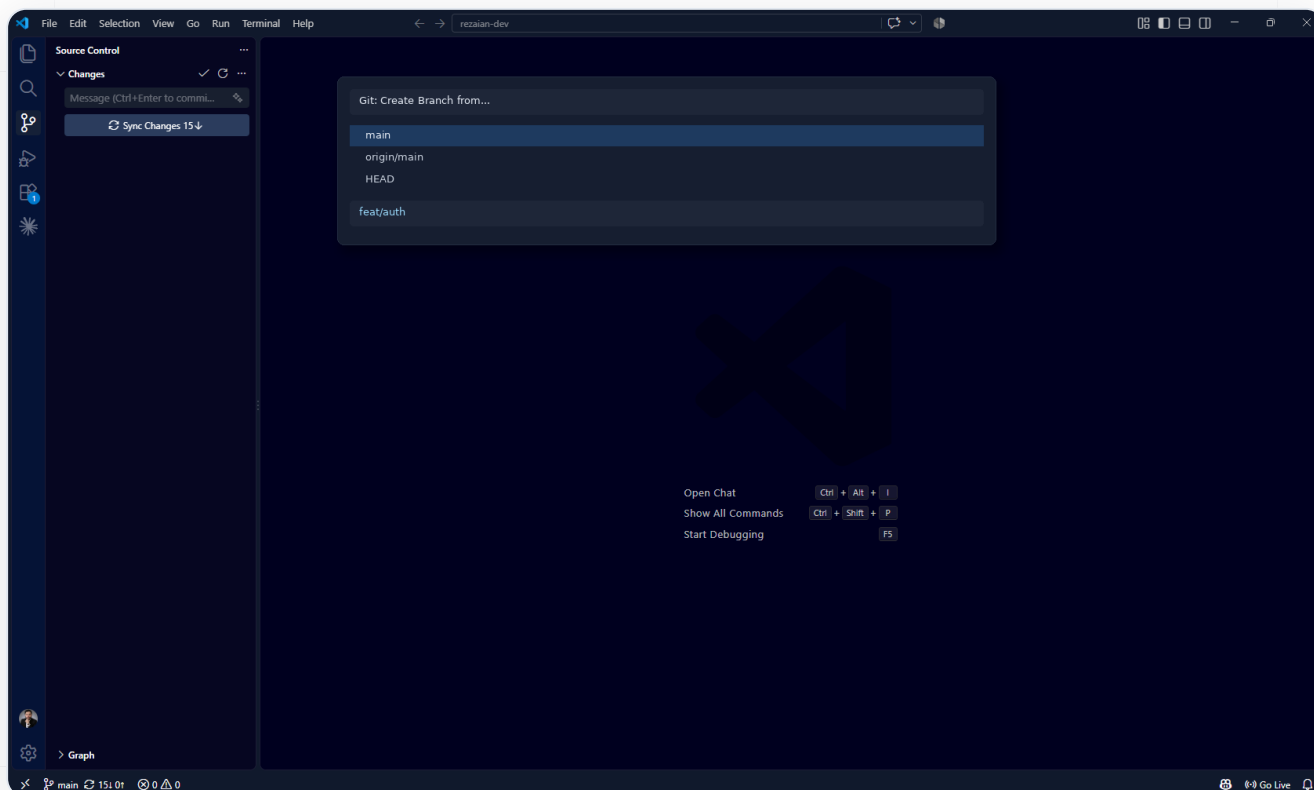
شکل ۱۱ – اگر این فهرست نیامد، روی نام شاخه کلیک نشده یا Git پیدا نشده.

کلیک ۲ – تعویض: روی نام شاخه دیگر.

باید ببینی: نوار وضعیت همان نام را می‌گیرد؛ فایل‌های ادیتور عوض می‌شوند.

اگر خطا **Your local changes would be overwritten**: تغییرات ذخیره‌شده داری. اول Commit یا Stash (منوی `...` → Stash)، بعد دوباره نام شاخه را کلیک کن.

کلیک ۲ – ساختن: روی **+ Create new branch from...** → یک کلیک روی پایه‌ای که می‌خواهی (**main**) → نام را تایپ کن مثلاً **feat/auth** → Enter.

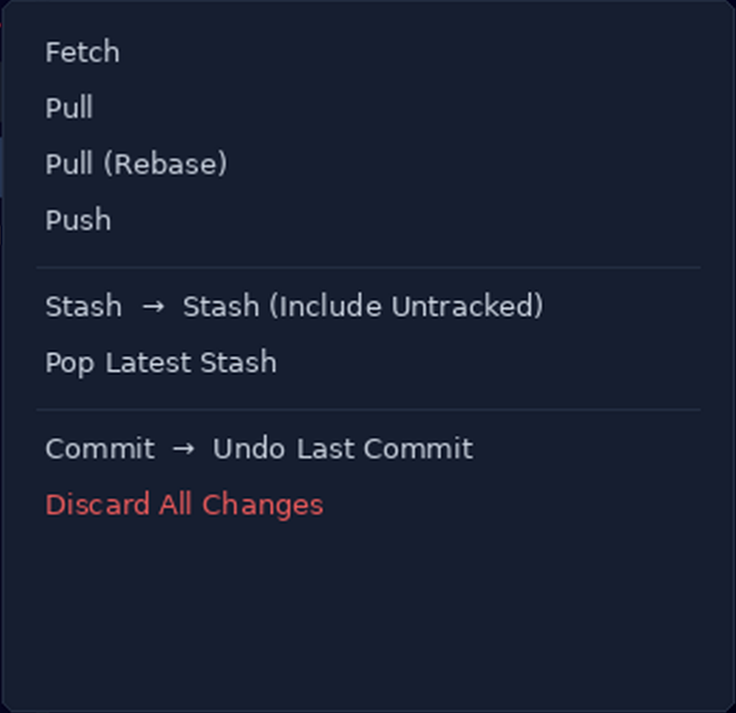


شکل ۱۱ – همین کار از Palette با **Git: Create Branch from...** هم می‌شود.

کلیک **Publish Branch**: در همان فهرست یا روی دکمه ابری نوار وضعیت. اولین **push** است (**-u**).
اگر **Publish** می‌خواهد وارد **GitHub** شوی: **Accounts** پایین **Sign in** → **Activity Bar**. بدون **origin**، دکمه اصلی پنل به جای **Sync** می‌ماند **Publish**.

اگر شاخه جدید در فهرست نیست: هنوز **Enter** ساختن را نزدی، یا **refresh** لازم است: **...** → **Refresh**.

...



شکل ۱۱ – ترتیب پیشنهادی: اول Fetch، نگاه به $\downarrow \uparrow$ ، بعد Pull یا Sync یا Push میانبر هر دوست.

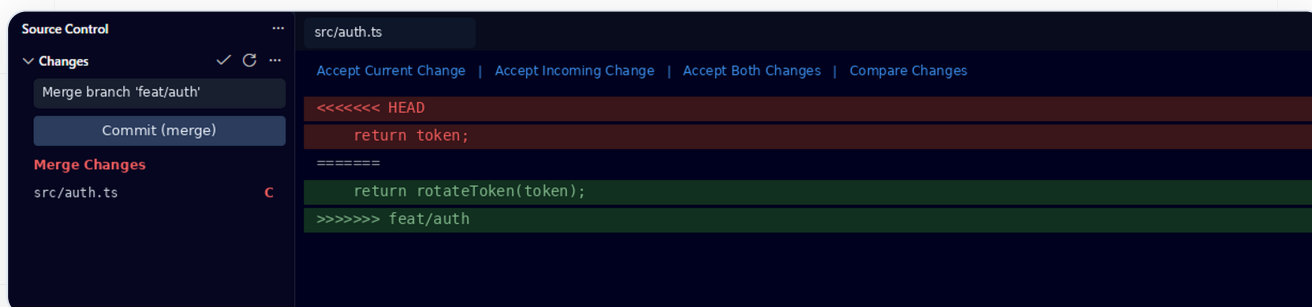
کلیک روی	باید بعدش ببینی	مال تو فرق دارد اگر
Fetch	عدد ↓ در نوار وضعیت زیاد شود، Graph Incoming پر شود؛ فایل‌ها عوض نمی‌شوند	اینترنت/SSH نیست؛ یا origin نداری
Pull	فایل‌ها ممکن است عوض شوند؛ ↓ صفر شود	تعارض → بخش بعد. اگر rebaseWhenSync روشن باشد ممکن است rebase شروع شود
Pull (Rebase)	اگر تعارض باشد نوار زرد Rebase	روی main مشترک نزن
Push	↑ صفر شود	Force = rejected کسی push کرده؛ Fetch کن نه Force
Sync	ترکیب pull/push	اگر confirmSync روشن باشد یک Yes اضافه می‌پرسد – این «خراب بودن» نیست
Stash (Include Untracked)	Changes خالی می‌شود	بدون Include، فایل‌های U می‌مانند و switch شکست می‌خورد
Undo Last Commit	فایل‌ها برمی‌گردند به Staged	اگر push کرده باشی، بعداً Push rejected می‌شود
Discard All Changes	همه فایل‌های M از لیست می‌روند	پنجره Are you sure؟ Yes – یعنی از دست رفتن کار ذخیره‌نشده

نوار وضعیت **↓2 ↑1** یعنی ۱ کامیت برای push و ۲ برای pull. عدد تو با کتاب یکی نمی‌شود و نباید بشود.

تعارض – از حرف C تا Commit (merge)

تعارض فقط بعد از Merge / Pull / Cherry-pick / Rebase می‌آید. اگر هیچ‌کدام را نزدی و C دیدی، احتمالاً merge ناتمام قبلی مانده: **Git: Abort Merge** → Palette.

باید ببینی: در Source Control عنوان **Merge Changes** و فایل با حرف **C**. دکمه اصلی ممکن است بشود **Accept Commit (merge) / Merge**.



شکل ۱۱-۱ – نوار آبی چهار فرمان فقط روی فایل C باز شده از Source Control است.

کلیک‌ها:

۱. یک کلیک روی فایل C.

۲. بالای بلوک قرمز/سبز، یک کلیک روی یکی از:

♦ **Accept Current Change** = نسخه شاخه‌ای که روی آن بودی (Ours / **HEAD**)

♦ **Accept Incoming Change** = نسخه‌ای که داشتی وارد می‌کردی (Theirs)

♦ **Accept Both Changes** = هر دو پشت‌سرهم؛ بعد خودت ویرایش کن

♦ **Compare Changes** = دو ستون

۳. **Ctrl+S**.

۴. یک کلیک روی **+** همان فایل (Stage). حرف C باید برود.

۵. یک کلیک روی **Commit (merge)** یا **Ctrl+Enter** با پیام پیش‌فرض merge.

اگر نوار **Accept** نبود: **Palette** → **Merge Conflict: Open Merge Editor** — سه ستون Incoming / Result. Result را درست کن → **Accept Merge**.

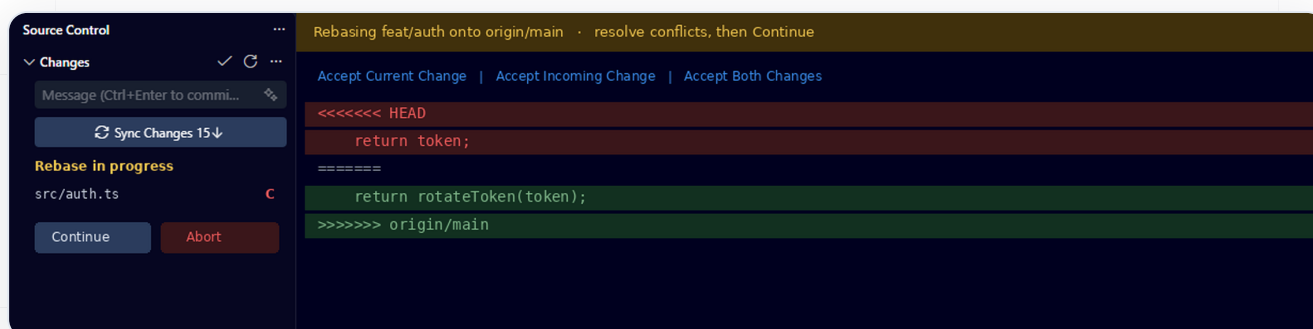
اگر هنوز C می‌ماند: **Save** و **Stage** را نزدی. یا چند تعارض در همان فایل است — همه را **Accept** کن.

پشیمانی قبل از مرحله ۵: **Palette** → **Git: Abort Merge**. بعد از **Commit** دیگر **abort** نیست؛ باید **revert**.

Rebase ناتمام — نوار زرد را تا انتها دنبال کن

بعد از **Pull (Rebase)** یا **Git: Rebase Branch...** اگر تعارض باشد:

باید ببینی: در پنل «Rebase in progress»، در بالا ادیتور نوار زرد، دکمه‌های **Continue** / **Abort**.



شکل ۱۱ — تا این نوار هست، دکمه معمولی **Commit** را نزن. VS Code عمده کامیت عادی را قفل یا همراه‌کننده می‌کند.

کلیک‌ها:

۱. فایل C را مثل تعارض حل کن → **Save** → **+** (Stage).

۲. یک کلیک **Continue** (= **git rebase --continue**).

۳. اگر تعارض بعدی آمد، ۱ و ۲ را تکرار کن تا نوار زرد برود.

اگر **Continue** خاکستری است: هنوز همه فایل‌های C را Stage نکرده‌ای، یا Message rebase خالی است – جعبه Message را خالی نگذار.

پشیمانی: یک کلیک **Abort**. همه چیز برمی‌گردد به قبل از rebase.

اگر وسط **rebase VS Code** را بستی و باز کردی: نوار زرد باید برگردد. اگر نماند، **Git: Continue** → Palette، **Rebase** یا **Abort Rebase**.

چند مخزن در یک پنجره

کلیک‌ها: **File** → **Add Folder to Workspace** → پوشه دوم که خودش **.git** دارد.

باید ببینی: در Source Control چند عنوان Repository. یک کلیک روی عنوان، آن مخزن را جمع/باز می‌کند.

SOURCE CONTROL

REPOSITORIES

v git-lab feat/auth

M src/auth.ts

> docs-site main

> hotfix-500 hotfix/500

Each folder is its own Git repo.

Worktrees appear as extra roots.

شکل ۱۱ – اگر فقط یکی می‌بینی، پوشه دوم Git ندارد یا هنوز Add Folder نکرده‌ای.

اگر **Commit** روی مخزن غلط نشست: عنوان همان Repository را باز کن، بعد Message را بزن. VS Code روی مخزن «فعال» کامیت می‌کند.

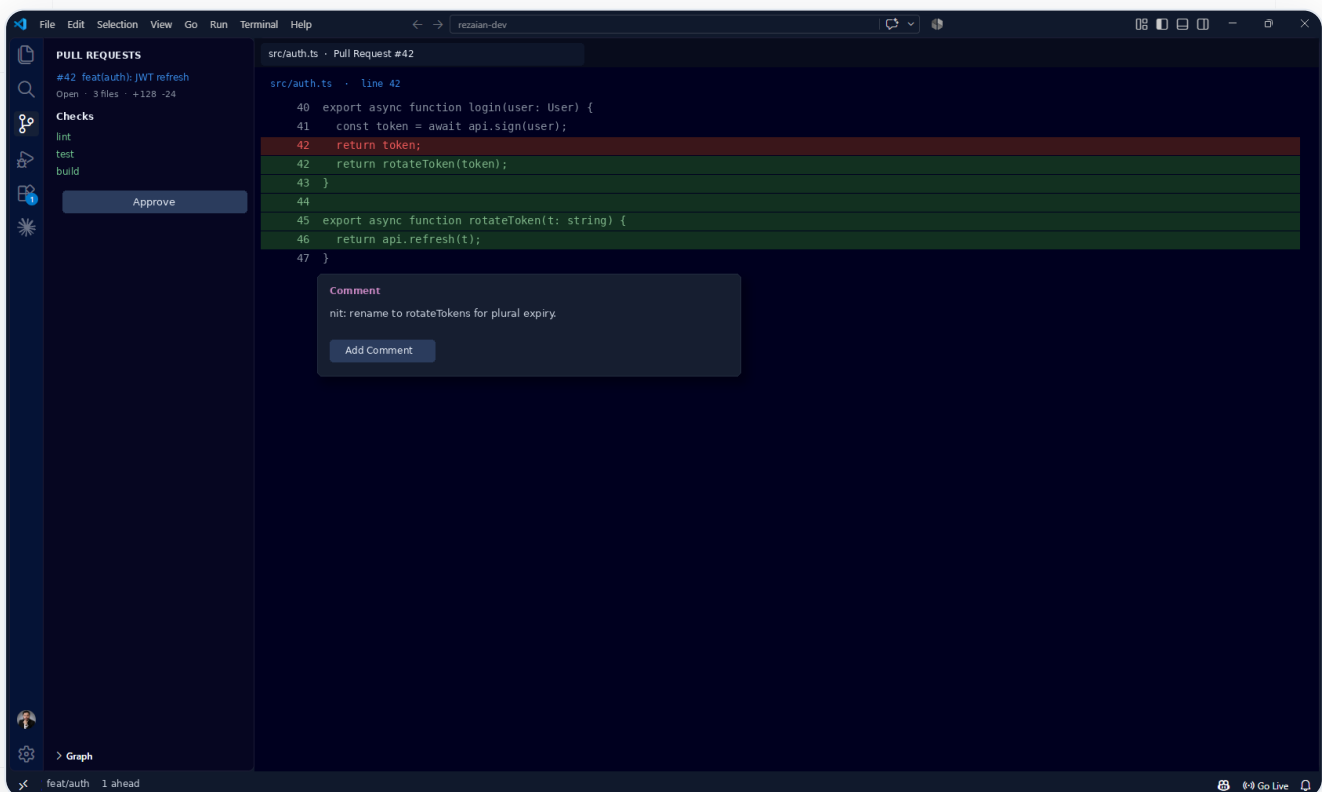
worktree: در ترمینال `git worktree add -b hotfix/500 ../hotfix main` بعد `File → Add Folder` روی `../hotfix`. یک شاخه در دو worktree همزمان باز نمی‌شود — پیام خطا را با Stash یا عوض کردن شاخه حل کن.

Pull Request — افزونه، نه خود Git توکار

Source Control توکار PR نمی‌سازد. باید افزونه **GitHub Pull Requests** نصب باشد.

کلیک نصب: Activity Bar آیکون Extensions (یا `Ctrl+Shift+X`) → جست‌وجو **GitHub Pull Requests** → Accounts → Install → پایین Activity Bar → **Sign in to GitHub** → مرورگر Allow.

اگر آیکون **GitHub** در **Activity Bar** نیامد: افزونه نصب نشده یا غیرفعال است. `View → Appearance → Activity` Bar را چک کن.



شکل ۱۲ — این نما مال افزونه است نه ستون Git. اگر ندیدی، `Sign in` نشده.

ساخت PR از شاخه فعلی:

۱. اول `Push / Publish Branch` از Source Control (وگرنه GitHub شاخه را ندارد).

۲. `Ctrl+Shift+P` → **GitHub Pull Requests: Create Pull Request**.

۳. عنوان و پایه (main) را تأیید کن → Create.

اگر Create خطا می‌دهد **no upstream** Publish Branch را جا انداختی.

ریویو: آیکن GitHub → یک کلیک روی Checkout → PR → در Files Changed روی خط کلیک → کامنت → Approve.

اگر Checkout PR فایل‌های تو را **overwrite** می‌کند: اول Commit یا Stash از ستون Source Control.

🔧 | اگر رفتار پنل «عجیب» است، اول این تنظیمات را چک کن

Ctrl+, → جست‌وجوی هر کلید:

```
.vscode/settings.json - رفتار SCM

{
  "git.enabled": true,
  "git.autofetch": true,
  "git.confirmSync": true,
  "git.enableSmartCommit": false,
  "git.enableCommitSigning": true,
  "git.untrackedChanges": "separate",
  "git.branchProtection": ["main", "master"],
  "git.branchProtectionPrompt": "alwaysCommitToNewBranch",
  "git.rebaseWhenSync": false,
  "git.autoStash": true,
  "scm.defaultViewMode": "tree",
  "scm.repositories.visible": 8,
  "diffEditor.ignoreTrimWhitespace": false,
  "diffEditor.hideUnchangedRegions.enabled": true
}
```

دید	احتمالاً این تنظیم
Commit بدون + همه را برد	enableSmartCommit روشن — Off کن
روی main کامیت زدی، VS Code شاخه جدید ساخت	branchProtection — عمدی است
Sync ناگهان rebase شروع کرد	rebaseWhenSync روشن
قبل از Pull فایل‌ها ناپدید شدند و بعد برگشتند	autoStash — stash موقت، ترس ندارد
Untracked ته لیست قاطی است	untrackedChanges را separate کن
هر Sync یک دیالوگ Yes/No	confirmSync — سلیقه است

Palette — وقتی کلیک را پیدا نمی‌کنی

Ctrl+Shift+P، تایپ Git:، Enter روی فرمان. اگر فرمان در فهرست نیست: یا پوشه مخزن ندارد، یا Git پیدا نشده (Git: Show Git Output).

- Fetch · Pull (Rebase) · Pull · Push · Undo Last Commit · Commit · Stage All Changes
- Rebase Branch... · Merge Branch... · Checkout to... · Create Branch from... · Sync
- View History · Abort Merge · Abort Rebase · Continue Rebase · Stash · Publish Branch
- Show Git Output

“سؤالات مصاحبه (با پاسخ)”

Mid – چرا Sync من merge ساخت و مال همکار rebase؟ تنظیم git.rebaseWhenSync فرق دارد. از روی دکمه حدس نزن؛ منوی ... را بخوان.

Senior – Checkout PR چه کلیک‌هایی زیرش است؟ fetch، Sign in، همان switch، ref، به آن شاخه. اگر Changes خالی نبود، VS Code اول Stash/Commit می‌خواهد.

چک‌لیست فهم

- ☐ Graph را باز کرده‌ای و اگر نبود علت را بلدی
 - ☐ از روی ... یک Fetch جدا زده‌ای نه فقط Sync
 - ☐ یک فایل C را تا Commit (merge) یا Abort برده‌ای
 - ☐ نوار زرد rebase را Continue یا Abort کرده‌ای نه Commit عادی
 - ☐ اگر PR نبود، فرق افزونه با Source Control توکار را می‌دانی
- فصل بعد: log، blame، diff و bisect — دیدن تغییر با چشم مسلح.

12 دیدن تغییر – bisect و diff، log، blame

برای فهمیدن «چه عوض شد» فایل کامل را نخوان – تفاوت دو نسخه را بخوان. این فصل سه نوع diff، فیلتر log، و پیدا کردن کامیت خراب با bisect را جدا می‌کند.

اهداف یادگیری

- ♦ diff سه سطحی (staged / unstaged / بین کامیت‌ها) را جدا کنی
- ♦ log را فیلتر کنی تا در ده هزار کامیت گم نشوی
- ♦ با bisect باگ را در $\log_2(n)$ قدم پیدا کنی

سه diff

سه نگاه

```
git diff                # Working vs Index
git diff --staged       # Index vs HEAD
git diff main...HEAD    # سه نقطه: از جد مشترک
git diff main..HEAD     # دو نقطه: تفاوت نوک‌ها
```

i سه نقطه در برابر دونقطه

A...B یعنی «تغییرات B از وقتی از A جدا شد» – دقیقاً همان چیزی که PR نشان می‌دهد. **A..B** نوک A را با نوک B مقایسه می‌کند و ممکن است تغییرات نامرتب A را هم بیاورد.

در VS Code کلیک روی فایل `git diff = Changes`. کلیک روی `git diff --staged = Staged`. **Open Changes** با **Alt** گاهی side-by-side را عوض می‌کند.

بهرتر diff

```
{
  "diffEditor.renderSideBySide": true,
  "diffEditor.ignoreTrimWhitespace": false,
  "diffEditor.hideUnchangedRegions.enabled": true
}
```

`hideUnchangedRegions` در فایل‌های بزرگ ۲۰۲۶ نجات‌دهنده است.

جست‌وجوی تاریخچه

```
git log --grep="refresh" -i
git log -S "rotateToken" -p          # pickaxe: کی این رشته آمد/رفت؟
git log -G "function rotate" -p      # regex روی diff
git log --since="2026-01-01" --until="2026-03-01" -- src/auth.ts
git log --no-merges --first-parent  # خط اصلی محصول
```

`--first-parent` روی `main` پر از merge، داستان محصول را می‌خواند نه داستان شاخه‌ها را.

bisect – کارآگاه باینری

پیدا کردن کامیت خراب

```
git bisect start
git bisect bad          # خراب است HEAD
git bisect good v1.2.0  # این تگ سالم بود
# Git می‌کند - تست کن checkout وسط را
git bisect good         # یا bad
git bisect reset
```

اسکرپت‌پذیر:

خودکار bisect

```
git bisect start HEAD v1.2.0
git bisect run npm test
```

در VS Code می‌توانی هر گام را از Graph با Checkout انجام دهی؛ اما `bisect run` در ترمینال سریع‌تر است.

✦ w- و ignore-revs

برای blame تمیز: فایل `.git-blame-ignore-revs` با هش کامیت‌های `format-only`، بعد:

```
git config blame.ignoreRevsFile .git-blame-ignore-revs
```

“ سؤالات مصاحبه (با پاسخ)

git diff Junior – بدون آرگومان چه می‌کند؟ Working در برابر Index.

Mid – pickaxe چیست؟ -S کامیتهایی را می‌آورد که تعداد رخداد یک رشته را عوض کرده‌اند.

Senior – چرا first-parent برای main مهم است؟ چون هر merge یک دنیای موازی است؛ برای رگرسیون محصول معمولاً فقط خط اصلی اهمیت دارد.

✓ | چک‌لیست فهم

☐ A...B را از A..B جدا می‌کنی

☐ -S را برای یافتن تولد یک تابع بلدی

☐ یک bisect موفق انجام داده‌ای

فصل بعد: stash، worktree و sparse-checkout – چند کار موازی بدون هرج و مرج.

13 کار موازی – stash، worktree و sparse-checkout

اگر وسط یک فیچر ناقص باگ تولید آمد، دو راه تمیز داری: تغییرات را موقتاً کنار بگذار (stash) یا همان مخزن را در پوشه دوم روی شاخه دیگر باز کن (worktree). کامیت الکی روی فیچر نزن.

اهداف یادگیری

- ♦ stash را با پیام نام‌گذاری کنی و درست برگردانی
- ♦ worktree را برای کار همزمان روی دو شاخه راه بیندازی
- ♦ sparse-checkout را در monorepo بفهمی

stash – قفسه موقت

تمیز stash

```
git stash push -u -m "wip: jwt tests"
git stash list
git stash show -p stash@{0}
git stash pop                # اعمال + حذف
git stash apply stash@{1}    # اعمال، نگه داشتن
git stash drop stash@{1}
git stash branch rescue stash@{0}
```

-u فایل Untracked را هم می‌گیرد. بدون آن notes.txt جدید جا می‌ماند و switch ممکن است شکست بخورد.

Terminal

```
git stash git stash push -u -m "wip"
git stash apply stash@{1} pop
```

VS Code Git

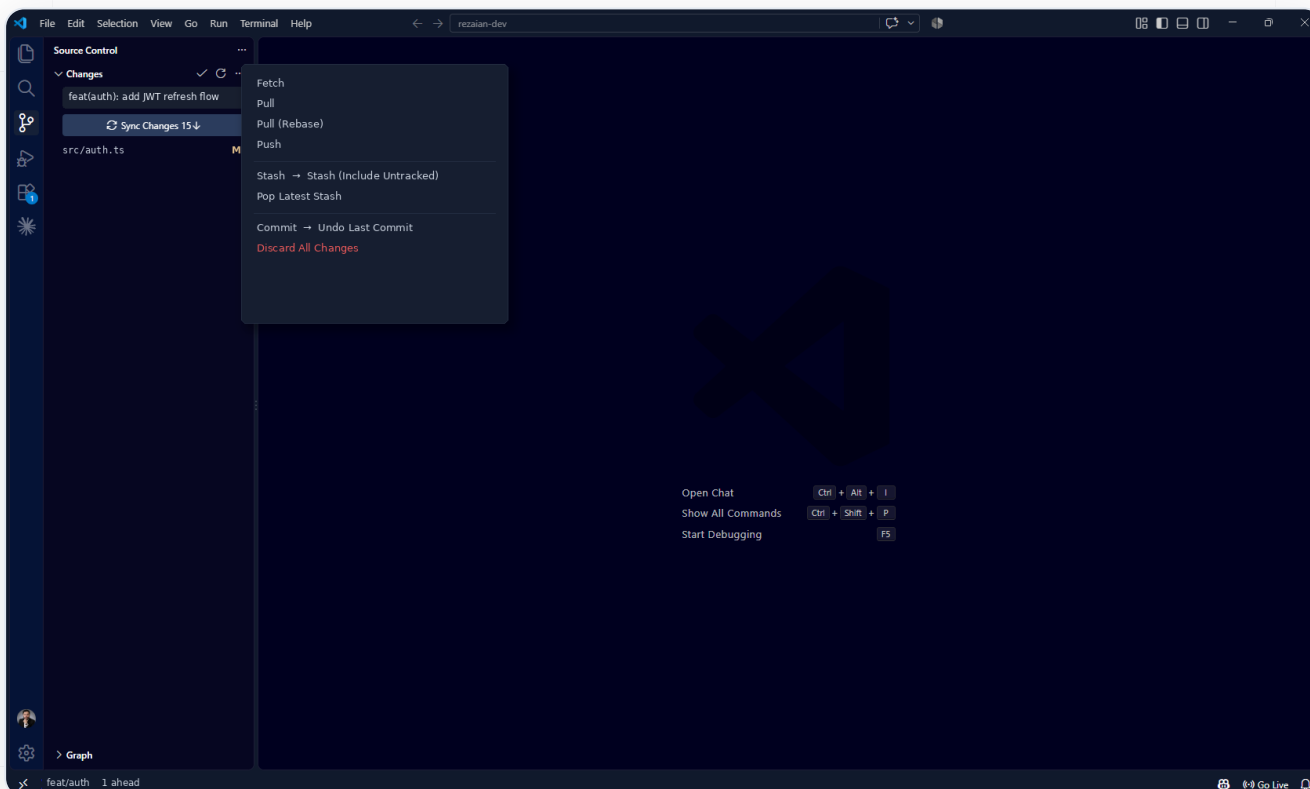
Stash → ... → Source Control

Stash (Include Untracked) ♦

Stash All ♦

لیست Apply Latest Stash / Pop Latest Stash ♦

stashها در همان منو دیده می‌شود.



شکل ۱۳ – منوی ... در Source Control: Fetch، Pull (Rebase)، Stash، Undo Last Commit و Sync در نوار وضعیت.

⚠ stash pop تعارض

اگر تعارض آمد، stash حذف نمی‌شود (pop شکست می‌خورد). حل کن، بعد drop . apply در این مواقع شفاف‌تر است.

🌲 worktree – چند checkout واقعی

worktree یعنی یک پوشه دوم از همان پروژه، روی شاخه‌ای دیگر. فایل‌ها جدايند، اما تاریخچه (پوشه .git) مشترک است – پس کامیت در پوشه دوم در همان مخزن ثبت می‌شود.

هات‌فیکس کنار فیچر

```
# از مخزن اصلی
git worktree add ../app-hotfix -b hotfix/500 main
code -n ../app-hotfix
# بعد از اتمام
git worktree remove ../app-hotfix
git worktree list
```

قواعد:

♦ یک شاخه در دو worktree همزمان checkout نمی‌شود

♦ برای PR موازی و عامل‌های AI (فصل ۳۲) ایده‌آل است

♦ `remove` قبل از پاک کردن پوشه

VS Code هر `worktree` را به‌صورت پوشه جدا باز می‌کند — Source Control مستقل، تاریخچه مشترک.

partial clone و sparse-checkout

در monorepo که ۱۰ گیگ است، کل درخت را لازم نداری.

فقط پوشه خودت

```
git clone --filter=blob:none --sparse git@github.com:org/mono.git
cd mono
git sparse-checkout set apps/web packages/ui
```

مایکروسافت) همین ایده را برای ویندوز بزرگ خودکار می‌کند — فصل ۳۱. `--filter=blob:none` (partial clone) blobها را تنبلانه می‌گیرد. GitHub این را پشتیبانی می‌کند. Scalar (ابزار

“ سؤالات مصاحبه (با پاسخ)

stash – Junior کجا ذخیره می‌شود؟ refهای `refs/stash` و کامیت‌های merge-مانند در `objects`.

Mid – کی `worktree`، کی `stash`؟ برای دقیقه؛ `worktree` برای ساعت و روز و PR موازی.

Senior – چرا شاخه در دو `worktree` ممنوع است؟ چون هر دو HEAD و Index می‌خواهند همان ref را حرکت دهند — مسابقه.

چک‌لیست فهم

☐ stash با پیام می‌سازی نه `stash@{7}` بی‌نام

☐ یک `worktree` واقعی باز کرده‌ای

☐ فرق partial clone و sparse را بلدی

فصل بعد: `remote` — `origin`، `fetch`، `pull`، `push`.

14 fetch، pull، push و Remote tracking

Remote فقط یک نام برای آدرس مخزن روی سرور است (معمولاً **origin**). Tracking یعنی شاخه محلی تو به کدام شاخه سرور وصل است – از روی آن Git می‌گوید چند کامیت جلوتر یا عقب‌تری.

اهداف یادگیری

- ♦ **origin** را به‌عنوان نام مستعار URL توضیح دهی
- ♦ **fetch** را از **pull** جدا کنی
- ♦ **-u** و شاخه بالادست را تنظیم کنی
- ♦ **force-with-lease** را به‌جای **force** خام استفاده کنی

Remote چیست؟

دیدن و ساختن

```
git remote -v
# origin  git@github.com:rezaian-dev/git-lab.git (fetch)
# origin  git@github.com:rezaian-dev/git-lab.git (push)

git remote add origin git@github.com:rezaian-dev/git-lab.git
git remote set-url origin git@github.com:rezaian-dev/git-lab.git
```

در VS Code: **Publish Branch** اگر **origin** نباشد، ویزارد GitHub را باز می‌کند.

fetch ≠ pull

فرمان	چه می‌کند
<code>git fetch</code>	اشیای جدید را می‌آورد؛ شاخه محلی را حرکت نمی‌دهد
<code>git fetch --prune</code>	refهای حذف‌شده روی سرور را محلی پاک می‌کند
<code>git pull</code>	<code>fetch</code> + <code>merge</code> (یا <code>rebase</code> اگر تنظیم شده)
<code>git pull --ff-only</code>	فقط اگر <code>fast-forward</code> ممکن باشد – امن‌ترین <code>pull</code>

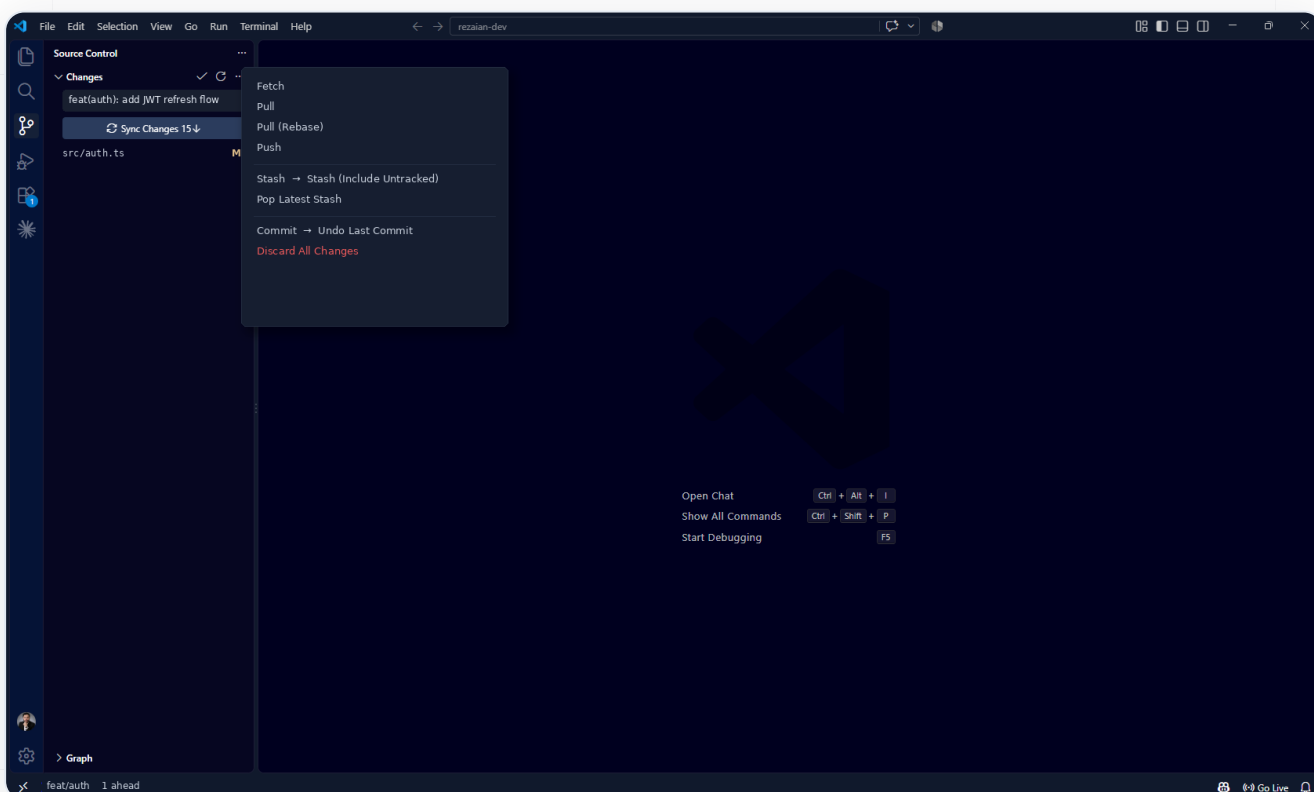
```
عادت پیشنهادی

git fetch origin
git log --oneline HEAD..origin/main   # چه چیزی می‌آید؟
git merge --ff-only origin/main
```

Terminal

```
git pull --ff-only git fetch origin
git push -u origin HEAD
```

- VS Code Git**
- ♦ **Publish Branch** اولین push و ساخت origin (با ورود GitHub)
 - ♦ **Sync** = pull سپس push
 - ♦ منوی ... جداگانه: **Pull / Push / Pull / Fetch (Rebase)**
 - ♦ نوار وضعیت: **↑1** ahead / **↓2** behind یعنی
 - ♦ در `git.autofetch: true` پس‌زمینه fetch می‌کند



شکل ۱۴ – نوار وضعیت: **↑** ahead و **↓** behind. دکمه Sync همان pull سپس push است.

بالادست

```
git switch -c feat/auth
git push -u origin HEAD      # -u = --set-upstream
git status -sb
# ## feat/auth...origin/feat/auth [ahead 1, behind 2]
```

- ♦ **ahead** = کامیت محلی که remote ندارد
- ♦ **behind** = کامیت remote که محلی نداری
- ♦ هر دو = واگرایی → باید merge یا rebase کنی قبل از push

خام روی شاخه مشترک force -- هرگز

```
git push --force-with-lease
# کرده باشد، رد می‌شود push اگر کس دیگری
```

--force کورکورانه تاریخچه سرور را عوض می‌کند. **--force-with-lease** فقط اگر remote همان چیزی باشد که تو آخرین بار دیدی.

در VS Code هنگام Push اگر rejected شدی، **Force Push** را نزن مگر روی شاخه شخصی.

✖ **force-- روی main**

در Rulesets باید ممنوع باشد. اگر نیست، همین امروز ممنوعش کن (فصل ۲۷).

“ سؤالات مصاحبه (با پاسخ)

origin – Junior اسم خاصی است؟ قرارداد است؛ می‌تواند **github** باشد. فقط نام مستعار URL است.

Mid – چرا ff-only؟ جلوی merge های اتفاقی و تاریخچه دنداندار را می‌گیرد.

Senior – lease چه چیزی را می‌سنجد؟ هش **refs/heads/...** روی remote را با remote-tracking محلی مقایسه می‌کند.

چک لیست فهم ✓ | |

- ☐ fetch را بدون ترس می‌زنی
- ☐ ahead/behind را می‌خوانی
- ☐ -u عادت شده
- ☐ force خام را کنار گذاشته‌ای

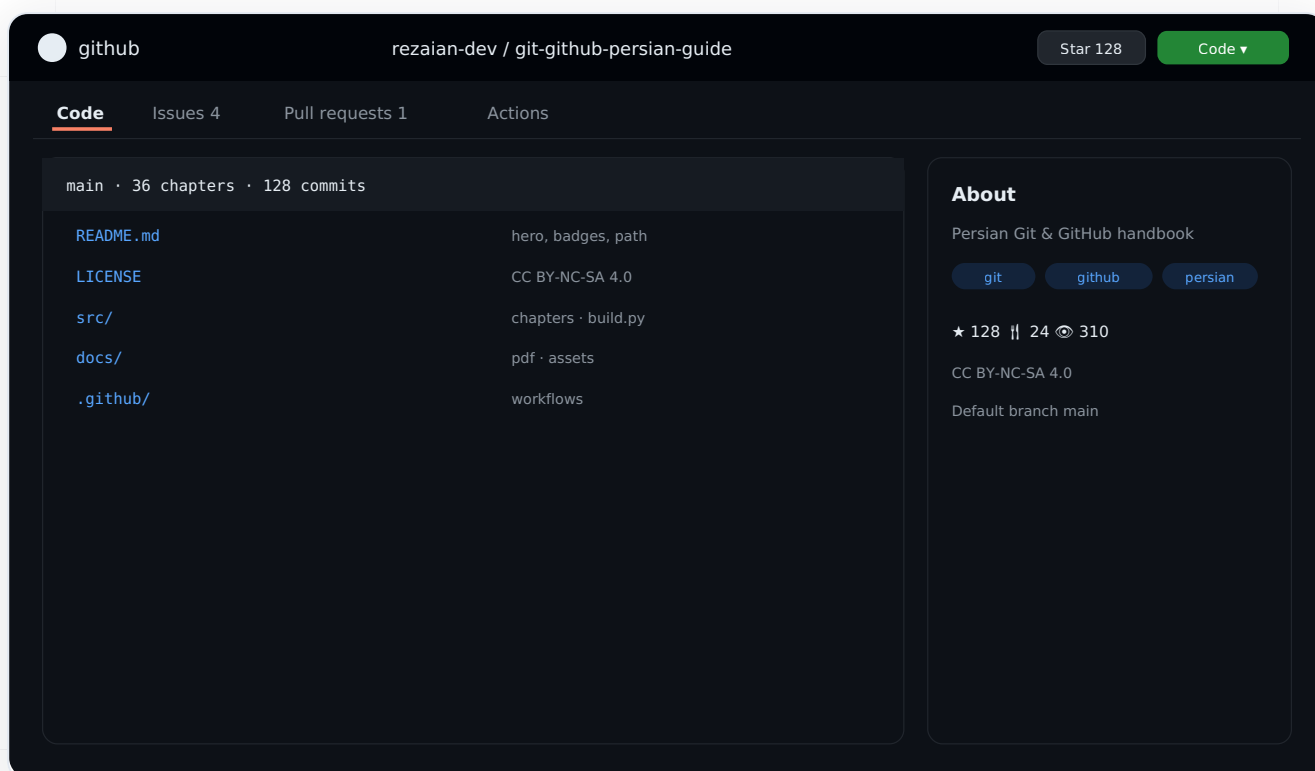
فصل بعد: GitHub به‌عنوان محصول — ریپو، Topics، README.

15 GitHub – ریپو، README و چهره عمومی پروژه

صفحه مخزن روی GitHub اولین چیزی است که همکار یا استخدام‌کننده می‌بیند. README باید در چند خط بگوید پروژه چیست، چطور اجرا می‌شود، و چطور مشارکت کنی.

اهداف یادگیری

- ♦ ریپو را از وب، `gh` یا VS Code بسازی
- ♦ `visibility` و `README`، `Topics`، `About` را درست بچینی
- ♦ تفاوت `public`، `private` و `internal` را بدانی
- ♦ فایل‌های اجتماعی مخزن را بشناسی



شکل ۱۳ – `About`، `Code`، ستاره، شاخه `main` و `README`؛ وب‌ترین پروژه.

ساخت ریپو

سه راه

```
# 1. وب: New repository – README, gitignore, License
# 2. CLI
gh repo create git-lab --public --source=. --remote=origin --push
# 3. VS Code: Publish to GitHub
```

تنظیمات حیاتی هنگام ساخت:

- ◆ **Default branch:** `main`
- ◆ **Add README:** برای مخزن خالی مفید؛ اگر از قبل تاریخچه داری نزن (تعارض)
- ◆ **gitignore:** قالب زبان
- ◆ **License:** برای عمومی، `SPDX` واضح (MIT، Apache-2.0، یا CC BY-NC-SA مثل راهنماهای فارسی)

README که Senior می‌نویسد

حداقل‌ها:

۱. یک جمله: پروژه چیست
 ۲. نشان‌ها (build, license, version)
 ۳. شروع سریع در کمتر از ده فرمان
 ۴. لینک اسناد، PDF، دمو
 ۵. نحوه مشارکت
- از HTML راست‌چین (`dir="rtl"`) برای فارسی نترس — GitHub آن را رندر می‌کند. تصویر hero را در `docs/` `assets` بگذار نه ریشه.

About، Topics، Social

در سمت راست صفحه ریپو:

- ◆ Description یک خطی
- ◆ Website (GitHub Pages)
- ◆ Topics: `git` , `github` , `persian` , `vscode` , `guide`
- ◆ Include in home — برای پروفایل

فایل	نقش
LICENSE	مجوز
CODE_OF_CONDUCT.md	رفتار
CONTRIBUTING.md	چگونه PR بفرستند
SECURITY.md	گزارش آسیب‌پذیری
.github/ISSUE_TEMPLATE/	قالب باگ/فیچر
.github/PULL_REQUEST_TEMPLATE.md	چک‌لیست PR
CODEOWNERS	مالک مسیر

✦ پروفایل README

ریپویی با نام دقیقاً برابر username (`rezaian-dev/rezaian-dev`) روی پروفایل سنجاق می‌شود. ویتترین شغلی است.

Visibility

نوع	چه کسی می‌بیند	نکتهٔ ۲۰۲۶
Public	جهان	Actions رایگان محدود؛ برای نمونه کار عالی
Private	تو + همکار دعوت‌شده	Copilot و Rulesets کامل
Internal	فقط Enterprise	بین orgهای یک شرکت

“ سؤالات مصاحبه (با پاسخ)

README – Junior اجباری است؟ از نظر Git نه؛ از نظر استخدام بله.

Mid – فرق squash merge در تنظیمات ریپو؟ Settings → General → Pull Requests: اجازهٔ merge commit / squash / rebase. تیم باید یکی را استاندارد کند.

CODEOWNERS – Senior چگونه merge را قفل می‌کند؟ اگر «Require review from Code Owners» Ruleset روشن باشد، تغییر در مسیر بدون Approve مالک نمی‌آید.

✓ چک لیست فهم

- ☐ ریپو با gh ساخته‌ای
 - ☐ README یک‌جمله‌ای + شروع سریع دارد
 - ☐ SECURITY و LICENSE وجود دارند
 - ☐ Topics تنظیم شده
- فصل بعد:** Fork، clone و رابطه با upstream.

16 Clone، Fork و Upstream – ورود به کد دیگران

Clone = کپی مخزن روی رایانه تو (با تاریخچه). Fork = کپی مخزن زیر حساب GitHub خودت. PR را باید از fork خودت به مخزن اصلی بفرستی؛ اگر فقط clone کرده باشی و حق push نداشته باشی، PR ساخته نمی‌شود.

اهداف یادگیری

- clone را با عمق و فیلتر درست انجام دهی
- fork را با `origin` و `upstream` بچینی
- شاخه را به‌روز نگه داری تا PR تمیز بماند

clone

هوشمند clone

```
git clone git@github.com:rezaian-dev/javascript-persian-guide.git
git clone --depth 1 <url> # فقط نوک - سریع، تاریخچه ناقص
git clone --filter=blob:none <url> # partial: blob تنبل
gh repo clone rezaian-dev/javascript-persian-guide
```

Terminal

```
git clone git@github.com:org/repo.git
gh repo fork gh repo clone org/repo
org/repo --clone
```

VS Code + GitHub

- Clone → Welcome یا Git: Clone :Palette
- اگر افزونه **GitHub Pull Requests** وصل است: **Clone from GitHub** و لیست ریپوهایت
- پوشه را انتخاب کن → **Open**
- Fork از **Command Palette**: **GitHub Pull Requests: Fork**

depth 1 و بعدش

shallow clone برای CI عالی است. برای کار روزانه `bisect` و `blame` را ضعیف می‌کند. با `git fetch --unshallow` کاملش کن.

چیدمان استاندارد

```
gh repo fork rezaian-dev/javascript-persian-guide --clone=true
cd javascript-persian-guide
git remote -v
# origin    تو (fork)
# upstream  ریپوی اصلی

git fetch upstream
git switch main
git merge --ff-only upstream/main
git push origin main
```

قانون طلایی:

- ♦ **origin** = ریپوی تو (حق push)
- ♦ **upstream** = ریپوی صاحب پروژه (معمولاً read-only)
- ♦ PR از **origin/feat/x** به **upstream/main**

در GitHub دکمه **Sync fork** همین merge را گرافیکی می‌کند. در VS Code افزونه PR ریموت‌ها را می‌شناسد.

مشارکت بدون fork – مدل تیمی

در سازمان، معمولاً مستقیم روی ریپو دسترسی **write** داری:

شاخه روی origin

```
git switch -c feat/docs origin/main
git push -u origin HEAD
gh pr create --base main
```

Fork برای OSS و پیمانکار خارجی است؛ برای همکار تمام‌وقت گام اضافه است.

“سؤالات مصاحبه (با پاسخ)”

Junior – فرق **clone** و **download ZIP**؟ بدون **git**. است؛ تاریخچه و شاخه ندارد.

Mid – چرا دو **remote**؟ تا بتوانی از **upstream** بگیر و به **origin** هل بدهی بدون حق نوشتن روی اصلی.

Senior – fork network چیست؟ گراف ریپوهای مشتق‌شده؛ **GitHub object**‌ها را تا حدی به اشتراک می‌گذارد. حذف **fork** تاریخچه PR را پیچیده می‌کند.

چک لیست فهم ☒ | |

clone با SSH انجام داده‌ای ☐

را درست نام‌گذاری می‌کنی `upstream / origin` ☐

Sync fork را قبل از PR تازه می‌زنی ☐

فصل بعد: Pull Request سطح Senior.

17 Pull Request – دروازه کیفیت تیم

Pull Request یعنی درخواست ادغام شاخه تو در شاخه هدف (معمولاً `main`). قبل از Merge، دیگران diff را می بینند، نظر می دهند و CI باید سبز باشد.

اهداف یادگیری

- ♦ PR را با `gh` و از VS Code باز کنی
- ♦ Draft، Checks و reviewers را درست به کار ببری
- ♦ استراتژی merge را آگاهانه انتخاب کنی
- ♦ PR بزرگ را بشکنی

The screenshot shows a GitHub Pull Request (PR) interface. At the top, the title is "feat(auth): add JWT refresh flow". Below the title, there's a status bar showing "Open", "#42", and "rezaian-dev wants to merge 4 commits into main from feat/auth". The interface is divided into several sections: "Conversation" (selected), "Commits 4", "Checks 3", and "Files changed 3". The "Conversation" section shows a green message: "This branch has no conflicts with the base branch" and "Merging can be performed automatically." Below this, it says "All checks have passed" with three green checkmarks for "CI / lint", "CI / test", and "CI / build". At the bottom of the conversation section, it says "1 approval · 0 requested changes · conversations resolved" and a green button labeled "Squash and merge". On the right side, there's a "Reviewers" section showing a green checkmark for "teammate". Below that, there's a "Labels" section with a green button labeled "feat". At the bottom of the right side, it shows "+128 -24" and a list of files: "src/auth.ts", "src/api.ts", and "README.md".

شکل ۱۴ – Conversation، Files، Checks سبز، Able to merge.

از ترمینال

```
git push -u origin HEAD
gh pr create --draft --fill --reviewer teammate
gh pr view --web
gh pr ready # خارج شو draft از
```

Terminal + gh 🖥️

```
gh pr create git push -u origin HEAD
gh pr checkout 42 --draft --fill
gh pr merge 42 --squash
```

افزونه GitHub Pull Requests 📄

۱. بعد از Push، نوار آبی **Create Pull Request**
۲. یا **Create Pull** → Activity Bar → GitHub آیکون → **Request**
۳. **Draft** را تیک بزن، **Reviewer** انتخاب کن
۴. **Checkout** روی PR دیگران برای تست محلی
۵. **Submit** → Files Changed → کامنت اینلاین → **Review**
۶. **Merge** از همان پنل (Squash/Merge/Rebase) طبق تنظیم تیم)

بدنه خوب:

۱. چرا این تغییر لازم است (لینک Issue)
۲. چه عوض شد – در سطح محصول نه خطبه خط
۳. چگونه تست شد
۴. اسکرین شات اگر UI است
۵. ریسک و rollback

✦ اندازه

هدف: زیر ۴۰۰ خط منطقی. اگر بزرگ است، **stacked PR** یا چند PR متوالی. ریویوکننده خسته **Approve** دروغین می‌زند.

Checks

هر Push روی شاخه Actions، PR را آتش می‌زند (فصل ۲۸). مربع سبز **Able to merge** یعنی:

- ♦ تعارض با base نیست
- ♦ بررسی‌های required پاس شده
- ♦ تعداد Approve کافی است
- ♦ conversation ها resolve شده‌اند (اگر اجباری باشد)

در VS Code تب Checks همان وضعیت را نشان می‌دهد.

Merge سه دکمه

روش	تاریخچه main	مناسب
Merge commit	گره دووالدی	حفظ PR به‌عنوان واحد
Squash	یک کامیت	شاخه شلوغ / WIP زیاد
Rebase and merge	خطی بدون گره merge	تیم متعهد به کامیت تمیز

تیم باید یکی را قفل کند تا `git log` یک‌شکل بماند.

Review قبل از

```
تمیزکاری
git fetch origin
git rebase origin/main      # روی شاخه شخصی
git push --force-with-lease
```

تعارض را تو حل کن نه ریویوکننده.

PR به main مستقیم از لپ‌تاپ بدون CI ⚠

حتی در پروژه شخصی Actions را روشن کن. حافظه عضله‌ای Production از همین‌جا ساخته می‌شود.

“ سؤالات مصاحبه (با پاسخ)

Draft – Junior یعنی چه؟ PR قابل merge نیست تا Ready شود؛ سیگنال «نگاه نکن هنوز».

Mid – چرا force-with-lease بعد از rebase؟ چون rebase هش عوض می‌کند؛ lease جلوی پاک کردن کار همکار روی همان شاخه را می‌گیرد.

Senior – stacked PR؟ زنجیرهٔ PR که هر کدام روی قبلی است؛ نیازمند ابزار (Graphite یا gh) و توافق تیم.

✓ | چک‌لیست فهم

☐ PR را با `--draft` باز می‌کنی

☐ چرا را در بدنه می‌نویسی نه فقط چه

☐ Checks را قبل از درخواست ریویو سبز می‌کنی

☐ استراتژی merge تیم را می‌دانی

فصل بعد: فرهنگ Code Review.

18 Code Review – فرهنگ، نه گیر دادن

Code Review یعنی قبل از Merge، حداقل یک نفر دیگر diff را بخواند. هدف پیدا کردن باگ و انتقال دانش است، نه خرد کردن نویسنده.

اهداف یادگیری

- ♦ نظر را روی **کد** بگذاری نه روی **آدم**
- ♦ از Prefixهای متعارف استفاده کنی
- ♦ Review در VS Code را Submit کنی نه فقط کامنت محلی
- ♦ بدانی Approve مسئولیت است

اصول

۱. نیت نویسنده را فرض خیر بگذار
۲. سؤال بپرس قبل از حکم («اینجا race ممکن است؟» نه «این غلط است»)
۳. nit را برچسب بزن تا با blocker قاطی نشود
۴. تحسین کن – الگوی خوب را تکرارپذیر کن
۵. خارج از scope را Issue کن نه گروگان PR

پیشوندهای رایج:

معنی	برچسب
سلیقه؛ اجباری نیست	nit:
بہتر است	suggestion:
بدون این Merge نکن	blocker:
نمی‌فهمم، توضیح بده	question:
این را بیشتر انجام بده	praise:

از ۲۰۲۵ Copilot Code Review هم نظر می‌دهد؛ آن را **لایه اول** بدان نه جایگزین انسان. نظر AI را بدون خواندن Approve نکن.

ریویو در VS Code

۱. PR را Checkout کن و واقعاً اجرا کن – نه فقط diff
۲. روی خط **Add Comment** – می‌توانی suggestion بگذاری (```suggestion`)
۳. چند کامنت pending بمانند
۴. **Submit Review** با یکی از: Comment / Approve / Request Changes

⚠ کامنت بدون Submit

Pending فقط روی ماشین تست، نویسنده چیزی نمی‌بیند تا Submit کنی.

✓ چک‌لیست نویسنده قبل از درخواست ریویو

- ☐ خودت diff را از بالا تا پایین خواندی
- ☐ فایل debug و `.env` نیست
- ☐ تست جدید برای رفتار جدید
- ☐ CI سبز
- ☐ توضیحات PR کامل است

✓ چک‌لیست ریویوکننده

- ☐ صحت: آیا باگ منطقی هست؟
- ☐ امنیت: تزریق، رمز، secret، XSS
- ☐ طراحی: آیا API پایدار می‌ماند؟
- ☐ تست: آیا فقدان پوشش هست؟
- ☐ مشاهده‌پذیری: لاگ و خطا

“ سؤالات مصاحبه (با پاسخ) ”

Mid – Request Changes یعنی چه؟ Merge معمولاً قفل می‌شود تا همان نفر یا سیاست re-request برگردد.

Senior – LGTM بدون نگاه؟ بدهی اعتماد. در حادثه، امضای تو روی تاریخچه است. Ruleset «dismiss stale reviews» را روشن کن تا push جدید Approve کهنه را باطل کند.

ریویو مهارت اجتماعی + فنی است. ابزار را در فصل ۱۱ دیدی؛ فرهنگ را اینجا. بدون هر دو، CI فقط تئاتر است.

فصل بعد: GitHub CLI – قدرت ترمینال.

19 GitHub CLI – gh به جای کلیک اضافه

gh برنامه رسمی GitHub برای ترمینال است. با آن می‌توانی بدون باز کردن مرورگر ریپو بسازی، PR باز کنی، و وضعیت Actions را ببینی.

🎯 اهداف یادگیری

- با gh چرخه کامل PR را طی کنی
- Issue و Release بسازی
- خروجی را با --json و jq اسکریپت کنی
- از gh api برای چیزهایی که فرمان آماده ندارند استفاده کنی

⚙️ ورود و پیش فرض

و تنظیم auth

```
gh auth login
gh auth setup-git          # credential helper
gh config set editor "code --wait"
gh alias set pv "pr view --web"
```

🔄 چرخه PR

روزانه

```
gh pr checkout 42
gh pr status
gh pr checks
gh pr diff
gh pr comment --body "nit: نام تابع را rotateTokens بگذار"
gh pr merge 42 --squash --delete-branch
gh pr list --author "@me" --state open
```

gh pr create --fill از عنوان کامیت‌ها بدنه می‌سازد – بعد دستی ویرایش کن.

Issue و پروژه 📋

Issue

```
gh issue create -t "PDF build fails on Windows" -b "steps..." -l bug
gh issue develop 18 --checkout # issue شاخه از روی
gh issue close 18 --reason completed
```

Actions و Release 🚀

CI عرضه و

```
gh release create v1.0.0 --generate-notes --title "v1.0.0"
gh run list --branch main
gh run watch
gh run view --log-failed
```

اسکرپت‌سازی 🧪

JSON

```
gh pr list --json number,title,reviewDecision,statusCheckRollup
gh api repos/:owner/:repo/actions/runs --jq '.workflow_runs[0].html_url'
gh api -X PUT repos/:owner/:repo/actions/permissions -f enabled=true
```

CLI و Codespaces ➦

`gh cs ssh / gh cs create` محیط ابری را بدون UI می‌سازد – مفید وقتی لپ‌تاپ ضعیف است یا از موبایل فقط ترمینال داری.

در VS Code ترمینال یکپارچه همان `gh` را اجرا می‌کند؛ افزونه PR و CLI مکمل‌اند نه رقیب.

“سؤالات مصاحبه (با پاسخ)”

gh – Junior جایگزین `git` است؟ خیر. `git` تاریخچه؛ `gh` API گیت‌هاب.

Mid – --json چرا مهم است؟ خروجی انسانی ناپایدار است؛ JSON برای CI پایدار است.

Senior – app token در برابر `user token` GitHub App برای ربات و Actions امن‌تر است؛ `least privilege` و `audit` بهتر.

چک لیست فهم | |

PR را بدون مرورگر merge کرده‌ای ☐

را دیده‌ای `gh run watch` ☐

یک `alias` شخصی ساخته‌ای ☐

فصل بعد: Discussions و Issues، Projects.

20 Discussions و Issues، Projects – کار تیمی روی GitHub

Issue برای ثبت باگ و کار است، Project برای برد کارهای تیم، Discussion برای سؤال باز. این‌ها روی GitHub زندگی می‌کنند، نه داخل پوشهٔ `.git`.

🎯 اهداف یادگیری

- ◆ Issue خوب بنویسی (بازتولید، انتظار، واقعیت)
- ◆ برچسب، مایل‌استون و قالب را بچینی
- ◆ GitHub Projects v2 را به PR وصل کنی
- ◆ بدانی Discussion کی بهتر از Issue است

🐛 آناتومی Issue

```
ایده - github/ISSUE_TEMPLATE/bug.yml
## شرح
## گام بازتولید
## انتظار
## واقعیت
## نسخه / محیط
## لاگ
```

قانون: یک Issue = یک مشکل. «همچنین این را هم درست کنید» را Issue جدا کن و لینک بده (`blocked by` #12).

کلیدواژه‌های جادویی در PR:

- ◆ Fixes #12 – با Issue، merge بسته می‌شود
- ◆ Refs #12 – فقط لینک

🏷 لیبل‌های مفید

bug، feat، docs، good first issue، help wanted، security، wontfix. رنگ مهم نیست؛ قرارداد مهم است.

Projects جدید روی جدول/بورد/زمان‌بندی است و فیلد سفارشی دارد (تخمین، ناحیه). می‌توانی با Actions وقتی PR باز شد کارت بسازی.

```
ایده اتوماسیون
# در workflow: gh project item-add
```

در سازمان‌ها Projects جایگزین Trello داخلی می‌شود چون به commit و PR وصل است.

Issues در برابر Discussions

Issues	Discussions
ضعیف	✓
✓	سؤال باز
گاهی	✓
✓	باگ مشخص
	RFC / ایده
	قابل بستن با PR

Security advisory

آسیب‌پذیری را Issue عمومی نکن. Security → Advisories یا SECURITY.md . افشای مسئولانه اعتبار می‌سازد.

“ سؤالات مصاحبه (با پاسخ) ”

Junior – Fixes #n کجا کار می‌کند؟ در پیام کامیت پیش‌فرض شاخه یا بدنه PR، وقتی به همان ریپو merge شود.

Senior – Projects v2 داده را کجا نگه می‌دارد؟ جدا از Issue؛ GraphQL API. Issue می‌تواند در چند Project باشد.

چک‌لیست فهم

- ☐ قالب Issue داری
- ☐ Fixes # را امتحان کرده‌ای
- ☐ Discussion را برای سؤال باز نگه می‌داری

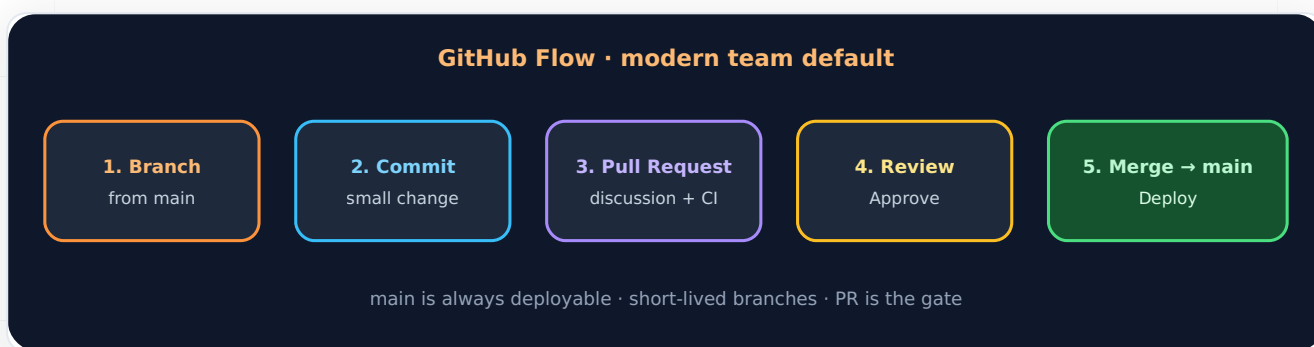
فصل بعد: استراتژی شاخه‌بندی – GitHub Flow و دوستان.

21 استراتژی شاخه – GitHub Flow تا Trunk-Based

استراتژی شاخه یعنی تیم توافق کند کد از کجا شروع شود، چطور به `main` برسد، و چه شاخه‌ای حق زنده ماندن طولانی دارد. شاخهٔ چندهفته‌ای با `main` واگرا می‌شود و `merge` را سخت می‌کند.

اهداف یادگیری

- ♦ GitHub Flow را پیاده کنی
- ♦ بدانی Git Flow کجا هنوز معنی دارد
- ♦ Trunk-based و feature flag را توضیح دهی
- ♦ سیاست شاخه را با Ruleset قفل کنی



شکل ۱۵ – شاخهٔ کوتاه از `main`، `PR`، `CI`، `Review`، `Merge`، `Deploy`

GitHub Flow – پیش‌فرض این کتاب

۱. `main` همیشه قابل استقرار است
 ۲. شاخهٔ کوتاه‌عمر از `main`
 ۳. `PR + Checks` اجباری
 ۴. `Merge → deploy` خودکار
 ۵. اگر خراب شد: `revert` سریع نه `freeze` طولانی
- مناسب وب، SaaS، اپ‌هایی با `feature flag`.

Git Flow – نسخه‌های سنگین

شاخه‌های `develop`، `release/*`، `hotfix/*`، `main`. برای نرم‌افزار با نسخهٔ شماره‌دار و چرخهٔ QA طولانی (امبدد، موبایل استور با ریویو یک‌هفته‌ای) هنوز زنده است. برای استارت‌آپ وب سربار است.

Trunk-Based 🚂

همه به `main` (trunk) با شاخه‌های چندساعته یا مستقیم. فیچر ناتمام پشت `feature flag`. نیاز به تست عالی و deploy ممتد دارد. گوگل و بسیاری از تیم‌های بزرگ این‌گونه‌اند.

مدل	طول شاخه	پیچیدگی	سرعت تحویل
GitHub Flow	روز	کم	بالا
Git Flow	هفته	زیاد	متوسط
Trunk	ساعت	کم در Git، زیاد در flag	خیلی بالا

⚠ شاخهٔ `feat/ali`سه‌ماهه`

هر روز rebase سخت‌تر می‌شود. اگر کار بزرگ است، آن را پشت flag به main تکه‌تکه تحویل بده.

در VS Code `git.branchProtection: ["main"]` جلوی کامیت مستقیم را می‌گیرد – لایهٔ محلی قبل از Ruleset سرور.

“ سؤالات مصاحبه (با پاسخ)

Mid – چرا Git Flow با GitHub Actions درد دارد؟ چون دو شاخهٔ بلند باید همیشه همگام شوند؛ CI دوگانه و cherry-pick hotfix.

Senior – environment protection چگونه با Flow جور می‌شود؟ `main` → staging خودکار؛ production نیاز به Approve در Environment دارد نه شاخهٔ جداگانهٔ اجباری.

✓ چک‌لیست فهم

- ☐ مدل تیم را در یک جمله می‌گویی
- ☐ main را مقدس می‌دانی
- ☐ شاخهٔ طولانی را بو می‌کشی

فصل بعد: Conventional Commits – زبانی برای ماشین و انسان.

22 Conventional Commits – پیام به عنوان API

Conventional Commits یعنی پیام را با قالب ثابت بنویسی: نوع (محدوده): خلاصه. مثال: `feat(auth): add` `refresh token`. ابزارها از روی نوع می‌فهمند نسخه و `changelog` را چگونه عوض کنند.

اهداف یادگیری

- ◆ مشخصات Conventional Commits 1.0.0 را رعایت کنی
- ◆ `feat!` و BREAKING CHANGE را درست علامت بزنی
- ◆ `commitlint` را در ذهن داشته باشی (پیاپی در فصل ۲۵)
- ◆ در VS Code پیام را بدون درد بنویسی

مشخصات

فرمول

`type(scope)!: description`

`[body]`

`[footer]`

انواع استاندارد: `feat`، `fix`، `docs`، `style`، `refactor`، `perf`، `test`، `build`، `ci`، `chore`، `revert`.

نمونه‌ها

```
feat(auth): add rotating refresh tokens
fix(pdf): embed Vazirmatn to avoid tofu glyphs
docs(readme): add PDF badge
feat(api)!: drop v1 endpoints
```

BREAKING CHANGE: clients must use /v2

:Semantic Release

◆ PATCH (1.2.3 → 1.2.4) → `fix`

◆ MINOR (1.2.3 → 1.3.0) → `feat`

◆ `!` یا BREAKING CHANGE → MAJOR (1.2.3 → 2.0.0)

افزونه **Conventional Commits** فرم می‌دهد. یا Snippet:

```
snippets

{
  "conv": {
    "prefix": "cc",
    "body": "${1|feat,fix,docs,chore,refactor,test,ci|}(${2:scope}): ${3:summary}"
  }
}
```

پیام را انگلیسی نگه دار اگر تیم بین‌المللی است؛ برای مخزن فارسی آموزشی می‌توانی بدنه را فارسی و type را انگلیسی بگذاری – قرارداد را در CONTRIBUTING بنویس.

bad	good
update / asdf / WIP / fixed stuff files	fix(login): handle expired JWT on 401 فعل امری، محدوده، نتیجه.

✦ چند تغییر = چند کامیت

اگر README و منطق auth را با هم عوض کردی، دو کامیت بساز (`add -p`). changelog دروغ نگوید.

“ سؤالات مصاحبه (با پاسخ)

type – Junior اجباری است؟ برای Git نه؛ برای انسان و ربات در تیم‌های مدرن عملاً بله.

Senior – چرا ! و BREAKING هر دو؟ ! در عنوان برای انسان و parser سریع؛ footer برای توضیح مهاجرت. ابزارها هر دو را می‌فهمند.

✓ چک‌لیست فهم

☐ ۱۰ کامیت آخرت Conventional است

☐ breaking را علامت می‌زنی

☐ WIP را قبل از ریویو می‌شکنی/squash می‌کنی

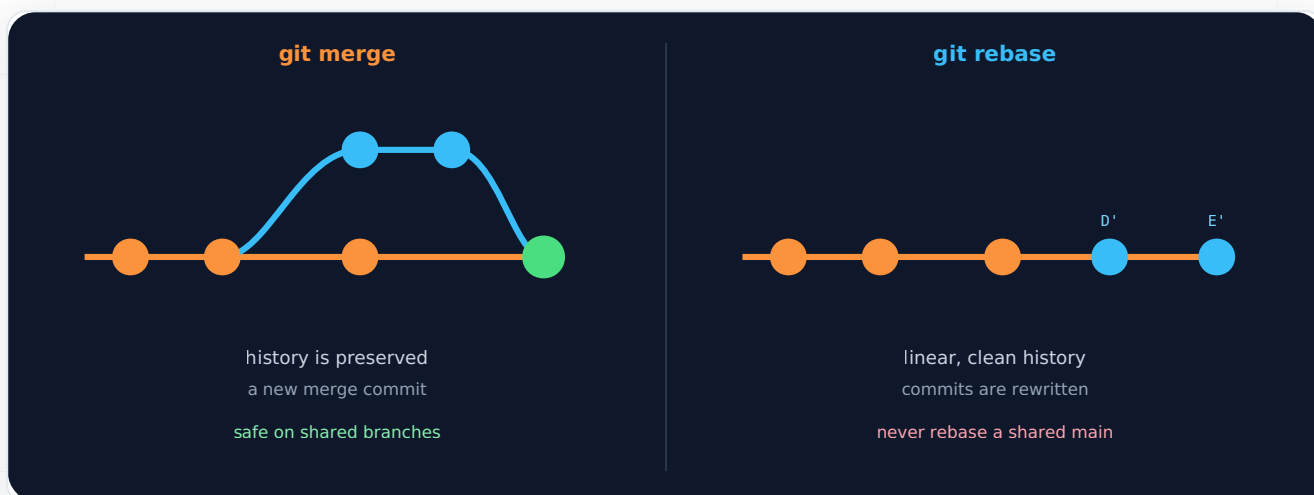
فصل بعد: بخش سوم – rebase تعاملی؛ جایی که Senior از Mid جدا می‌شود.

23 Rebase تعاملی – بازنویسی تاریخچه مثل مجسمه‌ساز

Rebase کامیت‌های شاخه تو را برمی‌دارد و یکی‌یکی روی نوک یک شاخه دیگر دوباره اعمال می‌کند. نتیجه تاریخچه خطی است، ولی هش هر کامیت عوض می‌شود. فقط روی شاخه‌ای که دیگران از آن استفاده نمی‌کنند این کار را بکن.

اهداف یادگیری

- ♦ rebase را به‌عنوان «کامیت‌ها را روی پایه جدید دوباره پخش کن» توضیح دهی
- ♦ `rebase -i` را برای `squash` و `reword` به کار ببری
- ♦ `--autosquash` و `fixup!` را بلد باشی
- ♦ قانون طلایی را هرگز نشکنی



شکل ۱۶ – rebase کامیت‌های جدید می‌سازد (D و E)؛ هش عوض می‌شود.

rebase دقیقاً چه می‌کند؟

Git هر کامیت شاخه تو را که در `main` نیست برمی‌دارد، `main` را HEAD می‌کند، و وصله‌ها را یکی‌یکی اعمال می‌کند. نتیجه: تاریخچه خطی. هزینه: هویت جدید برای هر کامیت.

به‌روز کردن شاخه شخصی

```
git fetch origin
git switch feat/auth
git rebase origin/main
git push --force-with-lease
```

Terminal

```
git rebase origin/ git fetch origin  
git rebase git rebase --continue main  
git push --force-with-lease --abort
```

VS Code Git

- ◆ Pull (Rebase) → ... برای به‌روز کردن شاخه
- ◆ Git: Rebase Branch... :Palette → origin/main
- ◆ تعارض: Stage Editor → Merge
- ◆ Git: Rebase Continue
- ◆ نوار زرد بالای ادیتور یعنی rebase ناتمام است –
- ◆ Git: Rebase Abort برای پشیمانی

interactive – اتاق عمل

پنج کامیت آخر را ویرایش کن

```
git rebase -i HEAD~5
```

ویرایشگر (در این کتاب VS Code با `core.editor="code --wait"`) لیستی می‌آورد:

todo

```
pick a11 feat: add login form  
pick b22 fix: typo in label  
pick c33 wip  
pick d44 feat: wire submit  
pick e55 docs: readme
```

فرمان‌ها:

فرمان	معنی
pick	نگه دار
reword	پیام را عوض کن
edit	بایست؛ فایل را درست کن
squash	با قبلی یکی کن؛ پیام‌ها را ترکیب کن
fixup	با قبلی یکی کن؛ پیام را دور بریز
drop	حذف

ترتیب خطوط = ترتیب تاریخچه. جابه‌جا کن تا داستان منطقی شود.

✖ هرگز rebase روی کامیت عمومی

اگر `origin/main` یا شاخه‌ای که همکاری checkout کرده شامل آن هش‌هاست، `rebase` = دو تاریخچه موازی و `force-push` جنگی.

⚡ autosquash و fixup

«address comments» بعد از ریویو، بدون کامیت

```
git commit --fixup a11
git rebase -i --autosquash origin/main
```

`--fixup` پیامی مثل `fixup! feat: add login form` می‌سازد؛ `autosquash` آن را زیر هدف می‌چسباند. در Git جدید `git commit --squash` هم هست.

از Git ۲.۳۸+ پرچم `--update-refs` شاخه‌های محلی دیگری که به کامیت‌های بازنویسی‌شده اشاره می‌کردند را هم حرکت می‌دهد – نجات از شاخه‌های یتیم.

rebase تعارض وسط ✨

نجات

```
# کن Stage حل و VS Code فایل را در
git rebase --continue
git rebase --abort          # پشیمانی کامل
git rebase --skip           # این وصله را دور بینداز - خطرناک
```

rerere (فصل ۲۴) تعارض تکراری را به خاطر می‌سپارد.

VS Code ✦

وضعیت rebase را نوار زرد بالای ادیتور نشان می‌دهد. Merge Editor همان UI تعارض فصل ۷ است. تا نوار زرد هست، کامیت عادی نزن.

“سؤالات مصاحبه (با پاسخ)”

Mid – چرا بعد از rebase «جدید» به نظر می‌رسد؟ چون هش عوض شده؛ GitHub فایل‌ها را مقایسه می‌کند نه هش را. Force-with-lease لازم است.

Senior – rebase در برابر merge برای به‌روز کردن شاخه؟ rebase تاریخچه PR را خطی و خوانا می‌کند؛ merge یک گره همگام‌سازی اضافه می‌کند. تیم باید یکی را انتخاب کند.

✓ چک‌لیست فهم

☐ rebase فقط روی شاخه خودت

☐ autosquash + fixup را بلدی

☐ --abort عضله‌ای شده

☐ force خام نمی‌زنی

فصل بعد: revert، cherry-pick، پیشرفته و rerere.

24 Cherry-pick، revert پیشرفته و rerere

Cherry-pick یک کامیت مشخص را روی شاخه فعلی کپی می‌کند (هش جدید). Revert یک کامیت خنثی‌کننده می‌سازد. rerere تعارضی را که قبلاً حل کرده‌ای به خاطر می‌سپارد.

اهداف یادگیری

- ♦ cherry-pick را با درک هش جدید به کار ببری
- ♦ revert یک merge را با `-m` درست انجام دهی
- ♦ `rerere` را روشن کنی تا تعارض تکراری تکرار نشود
- ♦ محدوده کامیت (`A..B`) را در cherry-pick بفهمی

cherry-pick

وصله یک کامیت را روی HEAD فعلی اعمال می‌کند و کامیت تازه می‌سازد (هش متفاوت، معمولاً همان پیام).

بیاور main یک هات‌فیکس را به

```
git switch main
git cherry-pick 9f3a1c
git cherry-pick -x 9f3a1c      # در پیام: (cherry picked from ...)
git cherry-pick -n 9f3a1c      # بدون کامیت؛ فقط Index/Working
git cherry-pick feat/auth~3..feat/auth  # چند کامیت (بدون خود ~3)
```

در VS Code Graph: راست‌کلیک روی گره → **Cherry Pick**.

⚠ دو بار merge کردن همان منطق

اگر بعداً کل شاخه را merge کنی، cherry-pick قبلی ممکن است تعارض یا تکرار ایجاد کند. `-x` ردیابی را آسان می‌کند. برای جریان منظم، hotfix را روی main بزن و به شاخه فیچر rebase کن.

revert merge

ادغام‌شده PR خنثی کردن

```
git revert -m 1 <merge-commit>
# -m 1 خط اصلی است (معمولاً main) یعنی والد اول
```

اگر دوباره همان شاخه را بخواهی merge کنی، Git فکر می‌کند تغییرات قبلاً آمده‌اند. راه‌حل رایج: revert آن revert، یا rebase شاخه روی main جدید.

در GitHub دکمهٔ Revert روی PR ادغام‌شده یک PR معکوس می‌سازد – برای تیم‌ها امن‌تر از revert دستی.

rerere – Reuse Recorded Resolution

یک‌بار روشن کن، سال‌ها سود ببر

```
git config --global rerere.enabled true
git config --global rerere.autoUpdate true
```

وقتی تعارضی را حل می‌کنی، Git جفت «قبل/بعد» را ذخیره می‌کند. اگر همان تعارض در rebase بعدی آمد، خودش اعمال می‌کند.

بازرس

```
ls .git/rr-cache
git rerere diff
```

♦ تیم‌های rebase-heavy

بدون rerere، هر rebase هفتگی همان تعارض CSS را دوباره حل می‌کنی. با rerere، ماشین حافظه می‌شود.

محدوده و خالی

Git جدید کامیت‌هایی که بعد از اعمال خالی‌اند را رد می‌کند. برای picking بازه روی شاخهٔ تقریباً همگام مفید است.

“ سؤالات مصاحبه (با پاسخ)

Junior – cherry-pick کپی است؟ کپی تغییر است نه کپی شیء؛ کامیت جدید با والد جدید.

Mid – چرا 1 -m ؟ merge دو درخت والد دارد؛ باید بگویی نسبت به کدام diff را معکوس کنیم.

Senior – rerere با conflict style diff3؟ پیش‌نمایش غنی‌تر است؛ حافظه rerere روی نتیجه staged کار می‌کند نه روی نشانه‌های فایل.

✓ | چک‌لیست فهم

☐ یک cherry-pick با -x زده‌ای

☐ revert merge را فهمیده‌ای

☐ rerere جهانی روشن است

فصل بعد: Hooks، commitlint و دروازه کیفیت محلی.

25 Hooks و کیفیت کامیت – lint قبل از push

Hook اسکریپتی است که Git در لحظه‌های مشخص (مثلاً قبل از commit) اجرا می‌کند. اگر اسکریپت با خطا خارج شود، آن commit یا push انجام نمی‌شود.

اهداف یادگیری

- hook محلی را از hook سمت سرور جدا کنی
- بدانی `.git/hooks` پوشیدنی نیست
- Husky / pre-commit framework را برای تیم بچینی
- محدودیت hook را بشناسی (قابل دور زدن است)

Git hooks کجایند؟

`.git/hooks/` نمونه‌های `*.sample` دارد. فعال‌سازی = حذف پسوند و اجرایی کردن.

```
pre-commit - تست شده ساده

cat > .git/hooks/pre-commit << 'EOF'
#!/usr/bin/env bash
set -euo pipefail
files=$(git diff --cached --name-only || true)
if printf '%s\n' "$files" | grep -Eq '^(|/)\.env$'; then
    echo "do not commit .env" >&2
    exit 1
fi
exit 0
EOF
chmod +x .git/hooks/pre-commit
```

⚠️ `set -e` و `grep`

اگر بنویسی `echo "$files" | grep ... && exit 1` بدون `if`، وقتی `.env` در stage نیست `grep` کد ۱ برمی‌گرداند و `set -e` کل هوک را شکست می‌دهد – یعنی هیچ کامیتی رد نمی‌شود. الگوی `if grep; then` `exit 1; fi` را استفاده کن.

مشکل: این فایل در مخزن نیست. همکار تو hook ندارد.

ابزار	مناسب
lint-staged + Husky	اکوسیستم JS
pre-commit (Python framework)	چندزبانہ، YAML
lefthook	سریع، Go، موازی
GitHub Rulesets	اجبار سمت سرور – دورزدنی نیست

package.json – ایڈہ Husky

```
{
  "scripts": {
    "prepare": "husky"
  }
}
```

.husky/commit-msg

```
npx --no -- commitlint --edit "$1"
```

commitlint.config.js

```
export default { extends: ["@commitlint/config-conventional"] };
```

.husky/pre-commit

```
npx lint-staged
```

lint-staged فقط روی فایل‌های staged اجرا می‌شود – سریع.

سمت سرور 🛡️

Hook محلی با `--no-verify` دور زده می‌شود. برای قانون واقعی:

◆ Ruleset: Conventional؟ خیر – پیام را GitHub اجباری نمی‌کند مگر Status Check

◆ Status Check: job `commitlint` روی PR

◆ Push rules: جلوگیری از `--force` روی main

no-verify-- در عجله ⚠

برای hotfix وسوسه‌انگیز است. اگر لازم شد، در PR توضیح بده و CI را اجباری بگذار تا دروازه دوم کار کند.

VS Code 🛠

وظایف `lint-staged` هنگام Commit از UI هم اجرا می‌شوند چون همان `git commit` را صدا می‌زنند. اگر hook شکست بخورد، VS Code خطا را در Git Output نشان می‌دهد.

“ سؤالات مصاحبه (با پاسخ) ”

hook – Junior را چطور به **GitHub می‌فرستم؟** نمی‌فرستی؛ نسخه را در ابزار تیمی (Husky) داخل ریپو می‌گذاری.

hook – Senior **server-side** در **GitHub؟** hook `GitHub.com` سفارشی Git ندارد؛ از Actions، webhooks، و Rulesets استفاده کن. `GitHub Enterprise Server` گاهی `pre-receive` دارد.

چک‌لیست فهم ✓

- ☐ می‌دانی hook محلی قابل دور زدن است
 - ☐ `commitlint` روی پیام Conventional است
 - ☐ CI همان قانون را تکرار می‌کند
- فصل بعد:** امضا، `passkey`، 2FA و رازها.

26 هویت و راز – امضا، passkey، 2FA، secret

امضای کامیت ثابت می‌کند این شیء با کلید ثبت‌شدهٔ تو ساخته شده. 2FA جلوی ورود با رمز تنها به حساب GitHub را می‌گیرد. secret‌های لو رفته را `git rm` پاک نمی‌کند – در تاریخچه می‌مانند.

🎯 اهداف یادگیری

- ♦ کامیت Signed و Verified را از هم تشخیص دهی
- ♦ 2FA و passkey را روی حساب GitHub اجباری کنی
- ♦ PAT، SSH و GitHub App را مقایسه کنی
- ♦ secret را از تاریخچه پاک کنی اگر فاجعه شد

✅ Verified یعنی چه؟

GitHub کامیت را Verified می‌گذارد اگر:

۱. با کلید ثبت‌شده (SSH یا GPG یا S/MIME) امضا شده باشد

۲. نویسنده با حساب منطبق باشد

در فصل ۲ SSH signing را چیدیم. برای تگ انتشار:

تگ امضاشده

```
git tag -s v1.0.0 -m "Git GitHub Persian Guide 1.0.0"
git push origin v1.0.0
```

Ruleset می‌تواند **Require signed commits** را روی `main` اجباری کند.

حساب GitHub در ۲۰۲۶ 🔒

حداقل‌ها:

- ♦ **Passkey / 2FA** اجباری برای org‌های جدی
- ♦ **Sudo mode** برای کارهای حساس
- ♦ **Session** را در دستگاه عمومی قطع کن
- ♦ **SSH keys** را سالی یک‌بار مرور کن
- ♦ **Authorized applications** OAuth را هرس کن
- ♦ **Fine-grained PAT** به‌جای classic، با انقضای کوتاه و دسترسی ریزپوی مشخص

ذخیره نکن Git را در PAT

```
# ترجیح این کتاب است SSH
gh auth token # نشان بده؛ در فایل پروژه نگذار
```

✖ PAT داخل ریپو

Secret scanning گیت‌هاب بسیاری از توکن‌ها را push را رد می‌کند. اگر رد نشد، فوراً revoke کن — rotate، نه فقط delete فایل.

اگر secret وارد تاریخچه شد 🧹

برداشتن کافی نیست HEAD فایل را از

```
git rm --cached .env
echo .env >> .gitignore
git commit -m "chore: stop tracking env"
# !تاریخچه هنوز راز را دارد
```

برای پاک کردن تاریخچه:

- ♦ **git filter-repo** (جانشین filter-branch)
- ♦ BFG Repo-Cleaner
- ♦ GitHub: **Remove sensitive data** + حتماً توکن را revoke

Force-push همه کلون‌ها را باطل می‌کند. تیم باید دوباره clone کند.

در VS Code هیچ دکمه‌ای «پاک کردن تاریخچه» ندارد — و نباید داشته باشد.

VS Code و هویت

Accounts → GitHub sign in (برای PR و Copilot) ♦

`git.enableCommitSigning` ♦

هرگز token را در `settings.json` نگذار ♦

“سؤالات مصاحبه (با پاسخ)”

Junior – Verified زرد/خاکستری؟ کلید با حساب یکی نیست یا فرمت امضا اشتباه است.

Mid – fine-grained در برابر **classic PAT**؟ محدود به ریپو، انقضا، permission جداگانه؛ classic practically root است.

Senior – چرا revoke مهم‌تر از filter-repo است؟ مهاجم ممکن است از قبل clone کرده باشد. پاک کردن تاریخچه عمومی جلوی استفاده توکن زنده را نمی‌گیرد.

چک‌لیست فهم

☐ 2FA/passkey روشن است

☐ کامیت‌ها Verified اند

☐ PAT کلاسیک نداری

☐ می‌دانی `rm --cached` راز را از گذشته پاک نمی‌کند

فصل بعد: Rulesets – سیاست مدرن شاخه.

27 Rulesets – سیاست شاخه در سطح سازمان

Ruleset مجموعه قوانین روی شاخه است – مثلاً push مستقیم به `main` ممنوع، CI باید سبز باشد، Merge فقط بعد از Approve. از Branch Protection قدیمی کامل تر است.

🎯 اهداف یادگیری

- ◆ Ruleset را به جای یک تیک ساده طراحی کنی
- ◆ review، required checks و linear history را قفل کنی
- ◆ Environments را برای production بفهمی
- ◆ CODEOWNERS را به سیاست وصل کنی

🏠 چرا Rulesets؟

Rulesets	Branch protection
تعداد سیاست	یکی per branch
چند لایه قابل ترکیب	نام شاخه
هدف	الگو (<code>release/**</code>) + tagging
Bypass	ضعیف
نقش / تیم / App مشخص	محدود
سازمان	ارثبری Enterprise → Org → Repo
ارزیابی	–
تاریخچه ارزیابی و بینش	

حداقل سیاست `main` در ۲۰۲۶:

۱. Restrict deletions
۲. Block force pushes
۳. Require a pull request (۱–۲ Approve)
۴. Require status checks to pass (CI)
۵. Require conversation resolution
۶. Require signed commits (اگر تیم آماده است)
۷. Dismiss stale reviews on new push
۸. Require review from Code Owners

```
.github/CODEOWNERS

*                @rezaian-dev
/src/build.py    @rezaian-dev
/docs/          @docs-team
*.yaml          @platform-team
```

آخرین قانون مطابق برنده است. بدون «code owners Ruleset»، این فایل فقط مستندات است.

Environments

برای Actions:

♦ staging – بدون دروازه

♦ production – required reviewers + wait timer + restricted secrets

حتی اگر شاخه آزاد باشد، deploy به production بدون Approve انسانی نمی‌رود. از ۲۰۲۵ **Immutable Releases** هم دارایی و تگ عرضه را قفل می‌کند.

VS Code اثر روی

اگر مستقیم روی **main** کامیت کنی:

♦ محلی: **git.branchProtection** هشدار می‌دهد

♦ push: سرور رد می‌کند با پیام Ruleset

پیام خطا را بخوان؛ معمولاً لینک به کدام rule است.

♦ Repository ruleset برای تگ

♦ v* را قفل کن تا کسی تگ نسخه را جابه‌جا نکند. همراه **Immutable Releases**، عرضه قابل‌اثبات می‌شود.

“سؤالات مصاحبه (با پاسخ)”

Mid – linear history یعنی چه؟ فقط fast-forward؛ merge commit روی main ممنوع. با squash یا rebase merge جور است.

Senior – bypass چه کسی باید داشته باشد؟ فقط bot حادثه و چند admin. اگر همه bypass دارند، سیاست تأثیر است.

چک لیست فهم ✓ | |

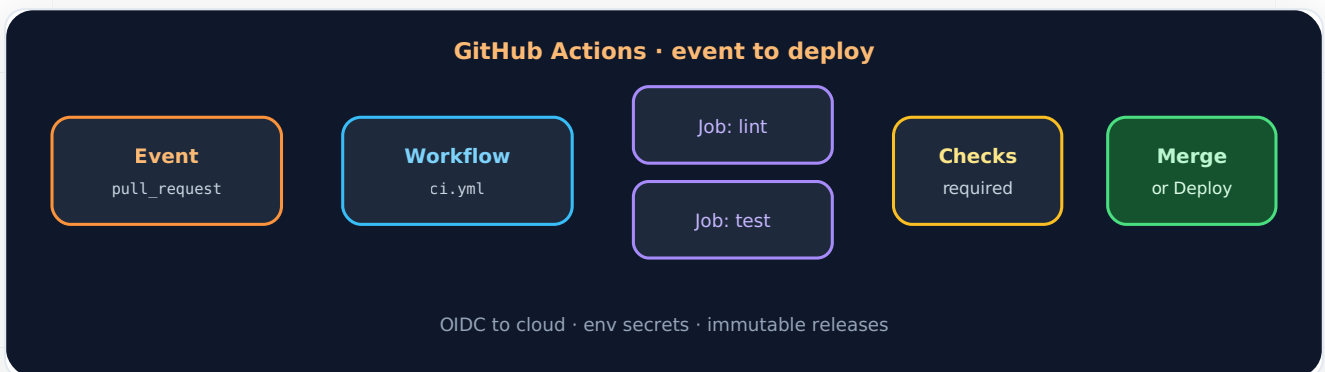
- ☐ force push روی main بسته است
 - ☐ CI required است نه تزئینی
 - ☐ production Environment Approve دارد
 - ☐ CODEOWNERS زنده است
- فصل بعد:** GitHub Actions از صفر تا OIDC.

28 GitHub Actions – از رویداد تا استقرار

GitHub Actions یعنی با هر رویداد (مثلاً push یا PR) یک گردش کار YAML روی سرور GitHub اجرا شود: تست، lint، ساخت. Secret را در مخزن ذخیره می‌کنی نه داخل کد.

اهداف یادگیری

- ♦ workflow، job، step و runner را جدا کنی
- ♦ CI را روی PR اجباری کنی
- ♦ secret را از Environment بخوانی نه از لاگ
- ♦ OIDC را به جای Access Key ابری ترجیح دهی



شکل ۱۷ – Jobs → Workflow → Event موازی → Merge/Deploy → Checks.

CI .github/workflows/ci.yml · pull_request

Success Run #128 · feat/auth · 1m 12s

✓ **lint**
12s

✓ **test**
38s

✓ **build**
22s

required checks
Ruleset on main

```

job: test

✓ Set up job
✓ actions/checkout@v4
✓ actions/setup-python@v5
✓ pip install -r src/requirements.txt
✓ python src/build.py --html
HTML written (243 KB)

✓ Complete job

```

شکل ۱۸ – صفحه run: جاب‌های lint/test/build و لاگ سبز.

اولین workflow

.github/workflows/ci.yml

```

name: CI
on:
  pull_request:
  push:
    branches: [main]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.13"
      - run: pip install -r src/requirements.txt
      - run: python src/build.py --html

```

قواعد:

◆ **pin** اکشن‌ها حداقل روی major (`@v4`)؛ برای عرضه حساس روی SHA

♦ `pull_request` از fork به secret های مخزن دسترسی ندارد (امنیت)

♦ `concurrency` را برای لغو run های کهنه روی همان شاخه بگذار

Reusable و matrix

از ۲۰۲۵ می توان workflow های reusable را تو در تو صدا زد (تا سقف مستند GitHub). ماتریس برای چند نسخه Node/Python:

```
matrix:
  strategy:
    fail-fast: false
  matrix:
    node: [20, 22]
```

`workflow_dispatch` حالا تا ۲۵ ورودی دارد — برای ops دستی بهتر از JSON هکی.

OIDC و Secrets

```
ابری بدون کلید ماندگار

permissions:
  id-token: write
  contents: read
steps:
  - uses: aws-actions/configure-aws-credentials@v4
    with:
      role-to-assume: arn:aws:iam::123:role/gha
      aws-region: eu-central-1
```

GitHub یک JWT کوتاه عمر به ابر می دهد. اگر ریپو لو برود، کلید AWS در Git نیست.

echo secret ✕

`echo ${{ secrets.X }}` در لاگ می آید مگر mask شود. حتی mask هم کامل نیست. secret را چاپ نکن.

VS Code در Actions

افزونه GitHub Actions: workflow ها، run ها، لاگ. `gh run watch` در ترمینال یکپارچه. روی PR تب Checks همان نتیجه را نشان می دهد.

“ سؤالات مصاحبه (با پاسخ)

Junior – runner چیست؟ ماشین (GitHub-hosted یا self-hosted) که step ها را اجرا می‌کند.

Mid – تفاوت pull_request و pull_request_target؟ دومی در زمینهٔ base با secret اجرا می‌شود – خطرناک اگر checkout روی کد fork باشد.

Senior – OIDC چه حمله‌ای را می‌بندد؟ کلیدهای بلندمدت ذخیره‌شده در secrets که با log یا memory dump می‌لکد؛ همچنین چرخش دستی.

✓ | چک‌لیست فهم

☐ CI روی هر PR اجرا می‌شود

☐ production از Environment با Approve می‌آید

☐ اکشن‌ها pin شده‌اند

☐ OIDC را حداقل مفهومی بلدی

فصل بعد: امنیت زنجیرهٔ تأمین روی GitHub.

29 امنیت زنجیره تأمین – Dependabot، CodeQL، Secret Scanning

امنیت زنجیره تأمین یعنی بدانی وابستگی‌ها، توکن‌ها و تگ انتشار قابل جعل یا تعویض نباشند. Dependabot، CodeQL و Secret Scanning همین لایه‌اند.

اهداف یادگیری

- ◆ Dependabot را برای آپدیت و هشدار CVE راه بیندازی
- ◆ CodeQL و secret scanning را بفهمی
- ◆ Immutable Releases را در مدل عرضه قرار دهی
- ◆ attestations را در یک جمله توضیح دهی

Dependabot

```
.github/dependabot.yml

version: 2
updates:
  - package-ecosystem: pip
    directory: /src
    schedule:
      interval: weekly
  - package-ecosystem: github-actions
    directory: /
    schedule:
      interval: weekly
```

دو چهره:

- ◆ **Security updates** – CVE؛ PR فوری
- ◆ **Version updates** – تازگی؛ زمان‌بندی

PRهای Dependabot را مثل کد انسان ریویو کن. گاهی breaking است.

secret scanning و CodeQL 🔍

- ◆ **CodeQL**: تحلیل معنایی؛ query برای injection، XSS، path traversal در Copilot code review ۲۰۲۵ با لینترها ترکیب شد.
- ◆ **Secret scanning**: توکن AWS، GitHub، Stripe ... را در push می‌بیند. Push protection جلوی ورود را می‌گیرد.
- ◆ **Private vulnerability reporting**: پژوهشگر بیرونی بدون Issue عمومی.

```
اسکلت - codeql.yml
on: [push, pull_request]
jobs:
  analyze:
    runs-on: ubuntu-latest
    permissions:
      security-events: write
    steps:
      - uses: actions/checkout@v4
      - uses: github/codeql-action/init@v3
        with:
          languages: python, javascript
      - uses: github/codeql-action/analyze@v3
```

Immutable Releases (GA ۲۰۲۵) 📦

تگ و دارایی release قفل می‌شوند؛ بازنویسی **v1.2.0** ممکن نیست. همراه **attestations** (SLSA-مانند) می‌توانی ثابت کنی این باینری از کدام workflow و کدام commit آمده.

```
تأیید
gh attestation verify dist/app.tar.gz --owner rezaian-dev
```

SBOM ✦

برای محصول جدی، در (CycloneDX/SPDX) workflow SBOM بساز و به release بچسبان. مشتری سازمانی می‌پرسد.

اشتباه‌های زنجیره 🔪

- ◆ اکشن `uses: some-user/action@main` — شاخه متحرک = کد متحرک
- ◆ `curl` | `bash` در CI

◆ npm package typosquatting

◆ خود-hosted runner روی ریپوی عمومی (fork می‌تواند کد اجرا کند)

“ سؤالات مصاحبه (با پاسخ)

Mid – push protection چیست؟ سرور push را رد می‌کند اگر secret شناخته‌شده در diff باشد.

Senior – attestation چه تضمینی است؟ امضای GitHub روی metadata ساخت: SHA، workflow، هویت. Tamper بعد از ساخت را نشان می‌دهد؛ باگ منطقی داخل کد را نه.

✓ | چک‌لیست فهم

☐ Dependabot برای زبان و Actions روشن است

☐ Push protection فعال است

☐ تگ نسخه Immutable است

☐ اکشن روی SHA یا تگ قفل است نه `@main`

فصل بعد: داخلی gc، packfile، Git – reftable.

30 داخلی fsck، gc، packfile و reftable

Git فایل‌ها را اول جدا و بعد فشرده در packfile نگه می‌دارد. gc زباله‌ها را جمع می‌کند. این فصل وقتی لازم می‌شود که مخزن باد کرده یا fsck خطا می‌دهد.

اهداف یادگیری

- ◆ loose object را از packfile تشخیص دهی
- ◆ gc و repack را با احتیاط اجرا کنی
- ◆ fsck را برای سلامت دیتابیس به کار ببری
- ◆ reftable را به‌عنوان backend مدرن refs بشناسی

objects

دو شکل:

- ◆ **Loose**: `.git/objects/ab/cdef...` یک فایل per شیء
- ◆ **Packed**: `pack-....pack + .idx` — دلتا فشرده، سریع برای fetch

نگاه

```
git count-objects -vH
git verify-pack -v .git/objects/pack/*.idx | sort -k 3 -n | tail
git cat-file -t 9f3a1c
git cat-file -s 9f3a1c
```

`git gc` loose pack را می‌کند، unreachable را بعد از grace period دور می‌ریزد. `git gc --prune=now` تهاجمی است.

⚠ gc وسط کار دیگران

روی سرور شلوغ gc را در maintenance پنجره بزن. `git maintenance start` زمان‌بندی امن‌تری است.

reftable refs: files

سنت: هر شاخه یک فایل در `.git/refs/heads` + `packed-refs` . با صدها هزار (Gerrit, monorepo) ref می‌کند می‌شود.

مخزن آزمایشی

```
git init --ref-format=reftable /tmp/rt
```

Reftable در مسیر production-ready Git ۳.۰ است. GitHub در مقیاس خودش از سیستم‌های مشابه بهره می‌برد. برای پروژه معمولی files کافی است.

recover و fsck

سلامت

```
git fsck --full
git fsck --lost-found
```

dangling commit اغلب کامیت‌های rebase شده‌اند – گاهی نجات با `git show <hash>` و شاخه ساختن. اگر `index.lock` ماند (کریش VS Code): فقط وقتی هیچ فرایند Git/Code روی مخزن نیست حذفش کن.

تاریخچه سنگین

چه چیزی حجم را خورده؟

```
git rev-list --objects --all \
| git cat-file --batch-check='%<objecttype> %<objectname> %<objectsize> %<rest>\' \
| awk '/^blob/' | sort -k3 -n | tail
```

بعد `git filter-repo` برای حذف باینری accidental. همه باید re-clone کنند.

“سؤالات مصاحبه (با پاسخ)”

Mid – چرا clone دوم سریع‌تر است؟ pack از قبل دلتا شده؛ looseها در مسیر critical نیستند.

Senior – unreachable چه زمانی می‌میرد؟ reflog expire (پیش‌فرض ~90 روز) بعد prune. تا آن موقع --reset --hard قابل برگشت است.

چک لیست فهم ✓ | |

☐ `count-objects -vH` را خوانده‌ای

☐ می‌دانی `gc unreachable` را بعد از `reflog` می‌کشد

☐ `reftable` را با `SHA-256` قاطی نمی‌کنی (دو محور جدا)

فصل بعد: مخازن بزرگ، `monorepo`، `Scalar`، `partial clone`.

31 مخازن بزرگ – monorepo و partial clone، Scalar

در مخزن خیلی بزرگ، کل تاریخچه و همه فایل‌ها را یکجا clone نکن. Partial clone محتوا را تنبلانه می‌آورد؛ sparse-checkout فقط پوشه‌های لازم را روی دیسک می‌گذارد.

اهداف یادگیری

- ♦ partial clone و sparse-checkout را ترکیب کنی
- ♦ LFS را از partial clone جدا کنی
- ♦ بدان کی monorepo و کی polyrepo
- ♦ Scalar و Git protocol v2 را بشناسی

Partial clone

ها blob بدون همه

```
git clone --filter=blob:none git@github.com:org/mono.git
git clone --filter=tree:0 <url> # ها هم بعد tree حتی تنبل تر؛
```

وقتی فایل را checkout می‌کنی Git blob را می‌گیرد. نیاز به پروتکل v2 و سرور پشتیبان (GitHub دارد).

Sparse checkout cone

فقط دنیای خودت

```
git sparse-checkout init --cone
git sparse-checkout set apps/web packages/ui
```

--cone الگوهای پوشه‌ای سریع‌تر از globهای پراکنده است.

Scalar

ابزار مایکروسافت (همراه Git for Windows / قابل نصب جدا): `git maintenance`، `partial clone`، `fsmonitor` را یکجا برای رپیوهای عظیم ویندوز تنظیم می‌کند. اگر ۳۰ checkout ثانیه طول می‌کشد، Scalar را امتحان کن نه «لپ‌تاپ نو».

Polyrepo در برابر Monorepo 🏢

Polyrepo	Monorepo
چند PR هماهنگ	یک PR
	تغییر چندپروژه
	CI
	باید path filter شود وگرنه می‌سوزد
راحت‌تر per-team	همه همه‌چیز را می‌بینند مگر sparse/server
	دسترسی
Actions ساده	Nx, Bazel, Turborepo
	ابزار

Actions در path filter ➦

```
YAML
on:
  pull_request:
    paths: ["apps/web/**", "packages/ui/**"]
```

بدون این، هر تغییر README کل مزرعه runner را آتش می‌زند.

VS Code multi-root برای polyrepo؛ برای monorepo همان workspace file + sparse که فقط پوشه‌های لازم را نشان می‌دهد.

“سؤالات مصاحبه (با پاسخ)”

Senior – blob:none در برابر LFS؟ LFS باینریهای مشخص را خارج می‌کند. Partial clone همه blobهای Git را تنبل می‌کند. اغلب با هم استفاده می‌شوند.

✓ چک‌لیست فهم

☐ clone فیلترشده را تجربه کرده‌ای

☐ CI را path-aware کرده‌ای

☐ می‌دانی monorepo بدون ابزار ساخت جهنم است

فصل بعد: Copilot، Coding Agent، AGENTS.md در ۲۰۲۶.

32 Copilot و Git – Coding Agent در عصر عامل‌ها

Copilot Coding Agent می‌تواند از روی Issue شاخه بسازد، کد بزند و Pull Request پیش‌نویس باز کند. Merge باید با Approve انسان باشد؛ عامل را مثل همکار تازه‌کار ریویو کن.

اهداف یادگیری

- سه سطح Copilot را جدا کنی: agent، complete، در coding agent، IDE ابری
- PR عامل را مثل PR پیمانکار ریویو کنی
- AGENTS.md و دستورالعمل ریویو را بنویسی
- خطر تاریخچه تولیدشده توسط AI را بشناسی

سه لایه

لایه	کجا	Git چه می‌شود
Inline / Chat	VS Code	تو کامیت می‌کنی
Agent mode IDE	VS Code، JetBrains، ...	چند فایل، دستور ترمینال، worktree جدا
Coding agent ابری	gh / GitHub.com	VM، شاخه خودش، draft PR، لاگ نشست

عامل ابری (GA از ۲۰۲۵): Issue را assign می‌کنی به Copilot. ماشین مجازی کلون می‌کند، کد می‌نویسد، به draft PR پوش می‌کند. تو باید **Approve** کنی. سیاست‌ها:

- فقط به شاخه‌ای که خودش ساخته push می‌کند
- نویسنده درخواست نمی‌تواند همان PR را Approve کند (اگر required review باشد)
- workflowهای Actions روی PR عامل تا تأیید انسان اجرا نمی‌شوند
- اینترنت محدود به فهرست قابل‌پیکربندی

github/copilot-instructions.md

- Conventional Commits
- نكن commit را env. هرگز
- جديد بنويس feat را برای vitest تست
- بساز python src/build.py را با PDF

از ۲۰۲۵ Universe فایل **AGENTS.md** می‌تواند عامل سفارشی با ابزار و مرز مشخص تعریف کند. MCP (Model Context Protocol) به Issue، جست‌وجوی کد و سیستم‌های داخلی وصل می‌شود. در ۲۰۲۶ VS Code پنجره Agents: چند نشست، worktree جدا، دیف کنار چت. هر نشست Git هویت خودش را دارد — مثل همکار موازی.

قانون طلایی انسان

۱. هرگز PR عامل را بدون خواندن diff ادغام نکن
۲. Secret و مجوز را در دستورالعمل ببند
۳. کامیت‌های عامل را squash کن اگر نويز است
۴. تاریخچه باید برای انسان قابل‌خواندن بماند
۵. Ruleset و CI برای عامل همان است که برای کارآموز

✖ عامل + --force + --no-verify

اگر به عامل اجازهٔ push مستقیم به main بدهی، مدل ذهنی این کتاب را دور ریخته‌ای. عامل هم باید از PR رد شود.

“سؤالات مصاحبه (با پاسخ)”

Mid — چرا draft PR برای عامل مهم است؟ چون CI تولید را تا بازبینی انسان آتش نمی‌زند.

Senior — مسئولیت حقوقی کامیت AI؟ نویسنده/Approve کننده انسان است. امضا، review و attestation هنوز روی آدم است.

✓ چک لیست فهم

□ PR عامل را مثل کد خارجی ریویو می‌کنی

□ دستورالعمل ریپو نوشته‌ای

□ main برای عامل هم قفل است

فصل بعد: سی نکته طلایی روزانه.

33 نکته و ترفند طلایی Git و GitHub

سی نکته کوتاه برای کار روزانه. هر مورد یک عادت است؛ لازم نیست همه را یکروزه حفظ کنی.

🎯 اهداف یادگیری

- ♦ سی عادت را در کار روزانه بشناسی
- ♦ بدانی کدام ترفند تاریخچه را بازنویسی می‌کند

🖥 ترمینال

۱. `git switch` - برگشت به شاخه قبلی.
۲. `git log -1 --format='%H'` هش کامل HEAD.
۳. `git show HEAD:src/app.ts` فایل را از کامیت بخوان بدون checkout.
۴. `git add -p` جراحی hunk.
۵. `git commit --fixup + rebase --autosquash`.
۶. `git pull --ff-only` یا هیچ.
۷. `git push --force-with-lease` نه `--force`.
۸. `git fetch --prune` شاخه مرده را پاک می‌کند.
۹. `alias.lg` گراف رنگی.
۱۰. `git config --global help.autocorrect 10` فرمان نزدیک را اجرا می‌کند.

🖥 VS Code

۱. `Ctrl+Shift+G` عضله اصلی.
۲. Stage Selected Ranges به جای کل فایل.
۳. Timeline قبل از panic.
۴. روشن، Sync آگاهانه. `git.autofetch`.
۵. روی `main` `git.branchProtection`.
۶. PR را داخل ادیتور Checkout کن.
۷. Merge Editor سه‌ستونه برای تعارض سخت.
۸. `code --wait` به عنوان editor گیت.

۹. Worktree + پنجره جدید برای هات‌فیکس.

۱۰. Git Output را بخوان؛ UI خطا را قورت می‌دهد.

GitHub 🐙

۱. PR زیر ۴۰۰ خط.

۲. Fixes #id در بدنه.

۳. Draft تا CI سبز.

۴. Squash یا merge commit – یکی برای کل تیم.

۵. CODEOWNERS + Ruleset.

۶. gh pr checks قبل از ping کردن ریویوکننده.

۷. Dependabot را ignore نکن؛ زمان‌بندی کن.

۸. Release Immutable.

۹. gh alias برای کارهای تکراری.

۱۰. README فارسی با dir="rtl" شرم ندارد.

✦ کارت یادآوری Undo

→ عمومی · reset --soft → کامیت محلی · restore --staged → Index · restore → Working
revert · گمشده → reflog

“سؤالات مصاحبه (با پاسخ)”

هر سطح: یکی از این سی تا را در مصاحبه به‌عنوان عادت روزانه‌ات تعریف کن – بهتر از حفظ flag.

✓ جمع‌بندی

نکته بدون مدل ذهنی شعبده است. اگر فصل ۳ را بلدی، این سی تا ارزان‌اند.

فصل بعد: کارگاه عملی چهار مینی‌پروژه.

34 کارگاه – چهار مینی پروژه از صفر تا PR

چهار تمرین عملی از ساخت مخزن تا PR. هر تمرین را خودت اجرا کن؛ فقط خواندن کافی نیست.

🎯 اهداف یادگیری

- ♦ چهار سناریوی واقعی را کامل طی کنی
- ♦ از VS Code و `gh` با هم استفاده کنی
- ♦ خروجی قابل نمایش برای رزومه بسازی

🔧 پروژه ۱ – مخزن شخصی تمیز

هدف: ریپوی `learning-git` با `README` فارسی، `LICENSE`، و دو کامیت `Conventional`.

اسکلت

```
mkdir learning-git && cd learning-git
git init -b main
# بساز README و LICENSE
git add README.md LICENSE
git commit -m "chore: bootstrap learning repo"
gh repo create learning-git --public --source=. --remote=origin --push
```

در VS Code Publish را هم می‌توانی بزنی. معیار موفقیت: کامیت `Verified`، ریپو با `About` و `Topic`.

🔧 پروژه ۲ – فیچر با PR و تعارض عمدی

۱. شاخه `feat/hello` از `main`
 ۲. فایل `hello.md` را تغییر بده، PR باز کن (Draft)
 ۳. روی `main` همان فایل را جور دیگر عوض کن و `push` کن (با PR جدا یا مستقیم اگر آزمایش شخصی است)
 ۴. تعارض را در `VS Code Merge Editor` حل کن
 ۵. CI حداقلی (حتی `echo ok`) بگذار تا `Check` سبز شود
 ۶. `Squash merge`
- معیار: اسکرین شات `Merge Editor` در `README` کارگاه.

پروژه ۳ – accidental secret و نجات

۱. عمداً `.env` را کامیت کن
۲. `Push protection` اگر گرفت، آفرین – دورش نزن
۳. اگر نگرفت: `rm --cached`، `gitignore`، توکن ساختگی را `rotate` فرض کن
۴. توضیح بده چرا `filter-repo` لازم می‌شد اگر توکن واقعی بود

⚠ هرگز توکن واقعی تمرین نکن

فقط مقدار `SECRET=fake`.

پروژه ۴ – Actions و PDF

اگر همین کتاب را می‌سازی:

ایده

```
- python src/build.py --html
# artifact HTML را آپلود کن
```

PR که CSS را عوض می‌کند باید HTML را به‌عنوان artifact بسازد. معیار: تب Checks در VS Code سبز.

چالش اضافه (سطح Senior)

- ♦ `worktree` جدا برای `hotfix`
- ♦ برای تمیز کردن پنچ کامیت `rebase -i`
- ♦ روی `main` در ریپوی خودت (اگر حساب اجازه می‌دهد)
- ♦ بدون مرورگر `gh pr create`

➤ خروجی قابل‌اشتراک

یک gist یا بخش README با فرمان‌های استفاده‌شده و دو اسکرین: Graph قبل/بعد از `rebase`.

✓ چک لیست فهم

- ☐ چهار پروژه تیک خورده
 - ☐ حداقل یک تعارض واقعی حل شده
 - ☐ یک workflow سبز دیده‌ای
- فصل بعد:** بیست اشتباه رایج.

35 ۲۰ اشتباه رایج و راه‌حل

بیشتر اشتباه رایج (از `add` . کور تا force روی `main`) و برای هر کدام همان Undo مشخص. قبل از کپی فرمان، ستون «چه کار نکن» را بخوان.

اهداف یادگیری

- ♦ اشتباه را نام ببری قبل از اینکه فاجعه شود
- ♦ راه‌حل درست را از راه‌حل خطرناک جدا کنی

۱. `git add` کور: `.env` و `dist` می‌آید. راه‌حل: `-p` و `.gitignore`.
۲. کامیت روی `main`: `Ruleset + branchProtection` در VS Code.
۳. پیام `update`: `Conventional`.
۴. `amend` بعد از `push`: `revert` یا کامیت جدید.
۵. `--force` به `origin/main`: `lease` + ممنوعیت سرور.
۶. `rebase` شاخه مشترک: `merge` یا `revert`.
۷. `Discard` در VS Code بدون `Timeline`: `Local History` را فوراً چک کن.
۸. `clone` با `HTTPS` و بعداً گیر `token`: `remote` را به `SSH` عوض کن.
۹. `ignore` کردن فایل `track` شده: `rm --cached`.
۱۰. حل تعارض با `Accept Both` کور: تست.
۱۱. `PR ۵۰۰۰` خطی: بشکن.
۱۲. `Approve` بدون اجرای کد: حداقل تست CI اجباری.
۱۳. `secret` در پیام کامیت: `revoke`.
۱۴. `submodule` فراموش شده: `clone --recurse-submodules` (اگر تیم `submodule` دارد؛ وگرنه اجتناب).
۱۵. نام شاخه با فاصله: از `/` و `-`.
۱۶. کشیدن `ZIP` به جای `clone`: تاریخچه نداری.
۱۷. دو نفر یک `worktree/شاخه`: قفل `Git`.
۱۸. اعتماد به `Smart Commit`: خاموشش کن.
۱۹. اکشن `@main`: `pin`.
۲۰. فکر کردن که `GitHub` = بکاپ تنها: `remote` دوم یا `mirror` برای چیزهای حیاتی.

«`lease`، بعد در `PR` توضیح»

شفافیت < سرعت جعلی.

«الان `force` می‌زنم بعداً درست می‌کنم»

تاریخچه دیگران را نمی‌توانی بعداً آسان درست کنی.

Undo map · which command, which tree?

Working Tree

git restore file
VS Code Discard
restores the file

Index

git restore --staged
Unstage in VS Code
removes from staging

Last commit

git commit --amend
git reset --soft HEAD~1
if not pushed yet

public history

git revert
revert the PR
never rewrite public history

golden rule: if others saw it, revert — never reset

شکل ۱۹ — قبل از هر فرمان مخرب، این نقشه را ببین.

“ سؤالات مصاحبه (با پاسخ)

سؤال محبوب: «بدترین اشتباه Git تو چه بود؟» صادق باش + بگو از کدام ابزار (reflog، revert، Ruleset) بعدش استفاده کردی. رشد را نشان می‌دهد.

جمع‌بندی ✓

اشتباه نکردن هدف نیست؛ اشتباه غیرقابل‌برگشت نکردن هست. Git برای برگشت ساخته شده مگر force روی عمومی.

فصل بعد: مصاحبه، واژه‌نامه و نقشه راه.

36 مصاحبه، واژه‌نامه و نقشه راه بعد از کتاب

پرسش‌های مصاحبه با جواب کوتاه، جدول واژه فارسی/انگلیسی، و مسیر بعد از کتاب. اگر بتوانی سه درخت و DAG را روی کاغذ بکشی، بیشتر سؤال‌ها همان مدل‌اند.

اهداف یادگیری

- ♦ به پرسش‌های سطح‌بندی‌شده با مدل سه‌درخت جواب بدهی
- ♦ اصطلاح انگلیسی را با معادل دقیق فارسی بگویی
- ♦ مسیر یادگیری بعد از این مرجع را بدانی

پرسش‌های مصاحبه (با پاسخ کوتاه)

Junior

- ♦ Git چیست؟ سیستم نسخه‌بندی توزیع‌شده مبتنی بر snapshot و محتوا-آدرس.
- ♦ staging یعنی چه؟ پیش‌نویس کامیت؛ Index.
- ♦ تفاوت clone و fork؟ clone محلی است؛ fork کپی حساب‌محور روی GitHub.
- ♦ pull در برابر fetch؟ `pull = fetch + ادغام/rebase`.

Mid

- ♦ merge در برابر rebase؟ حفظ گره در برابر بازنویسی خطی؛ rebase فقط روی شاخه شخصی.
- ♦ `reset --soft/--mixed/--hard`؟ کدام درخت‌ها حرکت می‌کنند (فصل ۸).
- ♦ `A...B` در diff؟ از جد مشترک؛ همان نمای PR.
- ♦ چرا `commit amend` هش را عوض می‌کند؟ هویت شیء از محتواست.
- ♦ `force-with-lease` چیست؟ `force` مشروط به نخوردن کار دیگران.

Senior / Staff

- ♦ DAG و چرا مهم است؟ توپولوژی تاریخچه؛ merge دو والد، rebase گره جدید.
- ♦ revert یک merge؟ `1 -m` و پیامد merge دوباره.
- ♦ Ruleset در برابر branch protection؟ لایه‌بندی، الگو، bypass، ارث سازمانی.
- ♦ OIDC در Actions؟ هویت کوتاه‌عمر به ابر بدون کلید ماندگار.
- ♦ چگونه باگ را در ۱۰ هزار کامیت پیدا می‌کنی؟ `bisect run` + تست قطعی.
- ♦ monorepo در Git چگونه زنده می‌ماند؟ `path-filter` CI، `sparse cone`، `partial clone`، گاهی Scalar.

♦ PR عامل AI را چطور کنترل می‌کنی؟ draft، required review، دستورالعمل، CI بعد از Approve.

♦ تمرین مصاحبه

جواب را ۲ دقیقه روی کاغذ بکش نه حفظ کن. اگر سه‌درخت و DAG را کشیدی، ۸۰٪ پرسش‌ها حل است.

فارسی	English
مخزن	Repository
درخت کاری	Working Tree
نمایه / استیج	Index / Staging Area
کامیت / ثبت	Commit
شاخه	Branch
تگ	Tag
اشارهٔ HEAD	HEAD
جداشده	Detached HEAD
واکشی	Fetch
ادغام	Merge
بازچینش	Rebase
گزینش گیلان	Cherry-pick
خنثی‌سازی	Revert
بازیابی فایل	Restore
بازنشانی اشاره‌گر	Reset
قفسهٔ موقت	Stash
درخت کار دوم	Worktree
تعارض	Conflict
درخواست ادغام	Pull Request
انشعاب حساب	Fork
بالادست	Upstream
ردیابی	Tracking Branch
گراف جهت‌دار بدون دور	DAG
تودهٔ فشرده	Packfile

فارسی	English
جمع‌آوری زباله	Garbage Collection
قلاب	Hook
مجموعه قواعد	Ruleset
صاحبان کد	CODEOWNERS
گواهی ساخت	Attestation
کلون ناقص	Partial Clone
نادیده‌گیری تنک	Sparse Checkout
امضای تأییدشده	Verified Signature
عامل کدنویس	Coding Agent

جدول را از راست به چپ بخوان؛ ستون انگلیسی برای جست‌وجوی مستندات رسمی است.

📖 نقشه راه بعد از کتاب

۱. یک ماه: همه کار روزانه فقط با VS Code + **gh**؛ ترمینال برای `rebase -i`.
۲. سه ماه: Maintain یک ریپوی عمومی؛ ریویو PR دیگران؛ Actions با OIDC روی یک ابر.
۳. شش ماه: حادثه ساختگی: `secret leak` + بازیابی با `filter-repo` روی مخزن آزمایشی.
۴. یک سال: طراحی Ruleset سازمان، Immutable Release، Environments، CODEOWNERS.
۵. مطالعه بعدی: Pro Git (رسمی)، مستندات SLSA، GitHub Actions، داخلی reftable، مدل مجوز GitHub Apps.

مجموعه فارسی همراه:

- ♦ مرجع JavaScript ES2025 — [javascript-persian-guide](#)
- ♦ همین کتاب را هر فصل که Git عوض شد (۳.۰) با PR به‌روز کن

≡ جمع‌بندی فصل

اگر بتوانی روی کاغذ سه درخت، Undo، DAG عمومی/خصوصی و مسیر PR را بکشی، سطح عالی را لمس کرده‌ای. بقیه تجربه است.

پایان مسیر کتاب ✓

- ♦ مدل ذهنی را به تیمات آموزش بده – بهترین تثبیت
 - ♦ یک PR آموزشی روی ریپوی خودت باز کن و ریویو بخواه
 - ♦ ستاره و Issue روی راهنماهای فارسی یعنی این مسیر زنده می‌ماند
- حالا برو تاریخچه را درست بنویس.

پایان مرجع



حالا تاریخچه را تو می نویسی

اگر این کتاب برایتان مفید بود، آن را با تیمتان به اشتراک بگذارید. Git فقط فرمان نیست؛ مدل ذهنی زمان، همکاری و کیفیت است.

Git 2.51+

GitHub 2026

VS Code

Actions

Rulesets

گیت هاب `github.com/rezaian-dev`

مرجع JavaScript `javascript-persian-guide`

گردآوری و تدوین حرفه‌ای · نسخه ۲۰۲۶ · ویرایش ۱.۰.۰